Enhancing Weak Supervision with Foundation Models: Empowering Expert Knowledge Expression

By

Peilin Yu

DRAFT

A DISSERTATION
SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF
DOCTOR OF PHILOSOPHY
IN THE

COMPUTER SCIENCE DEPARTMENT AT BROWN UNIVERSITY

PROVIDENCE, RHODE ISLAND

Spring 2025

This dissertation by Peilin Yu is accepted in its present form by the Department of Computer Science as satisfying the dissertation requirement for the degree of Doctor of Philosophy.

Date _____ _____

Stephen H. Bach, Advisor

Recommended to the Graduate Council

Date _____ _____

Ugur Çetintemel, Reader

Date _____ _____

Frederic Sala, Reader

Approved by the Graduate Council

Date _____ _____

Thomas A. Lewis, Dean of the Graduate School

Acknowledgements

First and foremost, I want to express my deep gratitude to my advisor, Professor Stephen Bach. His guidance, consistent support, and mentorship were essential throughout my Ph.D. journey at Brown. Professor Bach not only taught me how to approach research rigorously but also showed me the importance of integrity, humility, and thoughtfulness in academic life. His mentorship has significantly influenced my development as both a researcher and a person.

I am also very grateful to the many friends, collaborators, and staff members at Brown who made my time here both intellectually engaging and personally rewarding. In the following paragraphs, individuals within groups are listed alphabetically by last name. Please know that I appreciate each person mentioned for their specific help and support.

I would like to thank my labmates in BATS (Bach's Awesome Team of Students). Working alongside Reza Esfandiarpoor, Yeganeh Kordi, Aidan LaBella, Nihal Nayak, Francisco Piedrahita-Vélez, Jasper Solt, Zheng-Xin Yong, and Max Zuo, as well as our postdocs Cristina Menghini and Elaheh Raisi, has been a wonderful experience and greatly enriched my research.

I also had the opportunity to collaborate with many bright master's and undergraduate students in our group: Ross Briden, Elise Carman, Andy Delworth, Tiffany Ding, Chace Hayhurst, Berkan Hiziroglu, Tom Liu, Zhengke Liu, Justin Long, Top

Piriyakulkij, Esteban Safranchik, Dylan Sam, Gaurav Sharma, Avi Trost, Andrew Yuan, and Jiayu Zheng. Your curiosity and hard work were always motivating.

To my friends from SuperLab—Amina Abdullahi, Catherine Chen, Nate Gillman, Michal Golovenvevsky, Apoorv Khandelwal, Michael Lepori, Calvin Luo, Jack Merullo, Samuel Musker, William Rudman, Aalok Sathe, Zitian Tang, Shijie Wang, Megan Wei, Tian Yun, Yuan Zang, Zilai Zeng, and Ruochen Zhang—thank you for the many insightful discussions and collaborations that helped sharpen my thinking and broaden my research perspective.

To many more friends at Brown CS—Mete Tuluhan Akbulut, Binhao Chen, Haotian Fu, Kweku Kwegyir-Aggrey, Shihang Li, Jason Xinyu Liu, Sam Lobel, Alessio Mezzetto, Lishuo Pan, Benedict Quartey, Rafael Rodriguez Sanchez, Saket Tiwari, Kai Wang, Kevin Wang, Zhuo Wang, Brandon J Woodard, Shiyun Zou, Yuxuan Zhao, Kaiyu Zheng, and George Zerveas—thank you for your encouragement and for making the department such a supportive place. The informal chats, brainstorming, and shared experiences truly made it feel like a second home.

I also want to extend my appreciation to the excellent administrative staff (AStaff) at Brown CS, especially Genie, John, and Lauren, as well as the technical staff (TStaff) and the team at the Center for Computation and Visualization (CCV), for their reliable support and help whenever needed.

I am particularly thankful to my roommates and close friends who were there for me through the difficult times of the pandemic and everything else: Qiran Gong, Da Huo, Jingyi Ji, Yifan Jiang, Yaxi Lei, Mingxuan Li, Yuanqi Li, Yi Mi, Weizhi Xiao, Wenhao Yang, and Xiling Zhang. Your support, patience, and humor made even the toughest days manageable.

The conversations, laughter, and friendships were essential in getting me through this journey. I couldn't have done it without all of you.

I would also like to express my sincere appreciation to my committee members, Professor Ugur Çetintemel and Professor Frederic Sala, for their valuable feedback and guidance. Your insights significantly helped refine the core ideas presented in this dissertation.

Finally, I extend my deepest love and gratitude to my family. To my parents, Shiqin Yu and Ling Xia—thank you for your unwavering belief in me and your selfless support over the years. To my grandparents, Yuanzhen Lai, Youqiao Liu, Lihua Xia, and Haizhong Yu—your constant love and encouragement have always been my source of strength. This dissertation rests on the foundation of care and resilience you have given me.

Contents

CHAPTER 1

Introduction

Human knowledge and expertise are critical for developing computational systems capable of making accurate decisions at scale. Expressing and encoding this knowledge into computational frameworks has been a central research challenge since the inception of artificial intelligence in the 1950s (McCarthy, 1959; Newell et al., 1959). Knowledge expression in computation systems has evolved from expert-crafted rules in early expert systems to the careful labeling of training datasets that power modern deep learning systems, where each label represents an expert's judgment about the correct output for a given input.

Despite recent advances in machine learning, effectively capturing domain expertise remains challenging due to the reliance on high-quality labeled data. This dependency poses significant obstacles in specialized fields like medicine or law, where labeling is costly, time-consuming, and often limited by the availability of domain experts. **Weak supervision** frameworks provide a systematic approach for generating labeled datasets from noisy, incomplete sources through probabilistic modeling (Ratner et al., 2016a, 2017). These frameworks enable domain experts to express knowledge through programmatic labeling functions while accounting for noise and uncertainty in their outputs.

Foundation models (Bommasani et al., 2021b), pre-trained on massive datasets, present new opportunities to enhance how experts express their knowledge within weak supervision frameworks. These models provide capabilities for flexible interfacing via natural language **prompting**, semantic understanding, and cross-task generalization that can expand the expressiveness of expert knowledge capture. By integrating foundation models with weak supervision, we can potentially create more flexible and intuitive interfaces for domain experts while maintaining the systematic noise handling and uncertainty quantification that makes weak supervision robust.

This thesis focuses on advancing weak supervision by leveraging the capabilities of foundation models. I aim to develop mechanisms that enable domain experts to express their knowledge naturally and comprehensively, while ensuring scalability and robustness for real-world applications. This work seeks to develop more effective mechanisms for expert knowledge transfer, enabling efficient approaches for capturing and representing domain-specific knowledge within machine learning systems.

1.1 The Evolution of Knowledge Expression: from Expert Systems to Probabilistic Models

1.1.1 The Era of Expert Systems

Expert systems emerged in the 1970s as artificial intelligence's first significant success in practical applications. These systems aimed to capture human expertise through carefully crafted rules, typically expressed as "if-then" statements.

A major landmark system, DENDRAL (Feigenbaum et al., 1970) was developed to identify molecular structures from mass spectrometry data. While successful in its narrow domain, DENDRAL, as the authors later recognized, revealed fundamental challenges of rule-based approaches as problem complexity increased (Feigenbaum,

1984).

Another significant milestone for expert systems was the development of MYCIN (Shortliffe et al., 1975) at Stanford University. MYCIN was designed to assist in diagnosing blood infections and recommending appropriate treatments. With a knowledge base comprising approximately 600 handcrafted rules, MYCIN demonstrated the feasibility of encoding complex medical expertise into a computational system.

A typical rule in MYCIN might look like the following:

Example Rule in MYCIN System

IF (fever AND elevated white blood cells) THEN bacterial_infection (CF=0.8)

where "bacterial_infection" is the output prediction and CF stands for **certainty factors**. The **certainty factors** was an innovative attempt to handle uncertainty in rule-based reasoning (Shortliffe et al., 1975). By assigning numerical confidence levels to rules, MYCIN aimed to mimic the probabilistic judgments of medical experts. For instance, a **certainty factor** of 0.8 might indicate "strong suggestive evidence" for a bacterial infection, while a lower value could represent weaker evidence. One thing to note that **certainty factor** is hardcoded as part of the heuristics.

While early expert systems demonstrated remarkable success in specialized domains, several challenges emerged:

- **Knowledge Engineering Bottleneck:** Feigenbaum found that the knowledge engineering process itself presented a nontrivial bottleneck. Expert systems required domain specialists to explicitly express their decision-making processes as logical rules, a time-consuming and often imperfect process (Feigenbaum, 1984). The rigid rule structure can make it difficult to handle novel scenarios or capture subtle interactions between different pieces of evidence.
- **Uncertainty Handling:** MYCIN's **certainty factors** were an early attempt

to allow experts to express **partial uncertainty**, their confidence in specific logical implications, using hardcoded values. For example:

> IF (fever AND elevated white blood cells) THEN bacterial_infection (CF = 0.8)

The certainty factor of 0.8 enabled an expert to express, "I am confident but not certain that these symptoms indicate a bacterial infection." This was an important step in moving beyond towards more nuanced representation of uncertainty for expertise expression.

However, real-world expertise requires addressing a deeper challenge: **source uncertainty**, or the varying reliability of different knowledge sources based on their expertise and experience. Consider two medical experts providing conflicting rules:

- **Specialist Doctor A:**

  > IF (fever $>$ 38.5°C) THEN bacterial_infection (CF = 0.8)

- **Generalist Doctor B:**

  > IF (fever $>$ 38.5°C) THEN viral_infection (CF = 0.7)

While certainty factors quantified each rule's confidence, expert systems like MYCIN lacked the ability to differentiate between the reliability of Expert A, a specialist with focused expertise, and Expert B, a generalist. This inability to explicitly model **source uncertainty** or the trustworthiness of knowledge sources limited the system's ability to effectively resolve conflicting rules and weigh competing sources of expertise.

Expert systems like MYCIN pioneered the encoding of human expertise in automated

systems through certainty factors, demonstrating that computers could reason with degrees of confidence. While this approach only captured partial uncertainty through static values, it established uncertainty handling as essential for expressing expert knowledge. The need to combine knowledge from multiple experts with varying levels of reliability motivated the development of probabilistic frameworks like Dawid-Skene model (Dawid and Skene, 1979a) that could explicitly model source uncertainty.

1.1.2 Transition to Probabilistic Frameworks

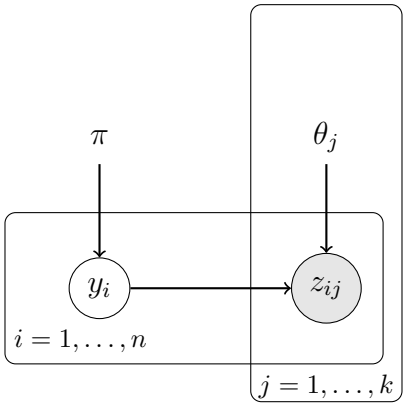

Figure 1.1: Plate diagram of the Dawid-Skene model (Dawid and Skene, 1979a) for aggregating expert annotations. The latent true labels y_i are generated from a prior distribution π, while observed expert annotations z_{ij} depend on both the true label y_i and expert-specific reliability parameters θ_j. The plates denote replication over n instances and k experts. Shaded nodes represent observed variables.

With the need to model source uncertainty becoming clear, researchers developed probabilistic frameworks that could explicitly reason about annotator reliability without access to the underlying true labels. One of the earliest and most influential contributions in this domain is the model proposed by Dawid and Skene (1979a), which formalized the aggregation of noisy annotations from multiple experts to infer latent true labels. This approach assumes a probabilistic relationship between the true label $y_i \in \mathcal{Y}$ for an instance x_i and the observed labels, represented as votes z, provided by expert annotators. Formally, let $\mathcal{X} = \{x_1, \ldots, x_n\}$ be a dataset with n instances and k experts, where the observed labels are represented as $\mathcal{Z} = \{z_{ij} \mid z_{ij} \in \mathcal{Y}\}$, where z_{ij}

denotes the label provided by the j-th expert for the i-th instance.

The model assumes that each true label y_i is drawn independently from a categorical distribution with parameter π, representing the prior probability of each class in $\mathcal{Y}$. Given the true label y_i, each expert j provides their label z_{ij} according to an expert-specific confusion matrix θ_j, which captures their labeling accuracy and error patterns. The probabilistic model aims to estimate the true labels by maximizing the joint likelihood:

$$p(Y, \mathcal{Z}) = \prod_{i=1}^{n} p(y_i \mid \pi) \prod_{j=1}^{k} p(z_{ij} \mid y_i, \theta_j), \tag{1.1}$$

where $p(y_i \mid \pi)$ represents the prior probability of label y_i, and $p(z_{ij} \mid y_i, \theta_j)$ models the probability of expert j providing label z_{ij} given the true label y_i and their reliability parameters θ_j.

The introduction of probabilistic frameworks that explicitly model annotator uncertainty has laid a robust theoretical foundation for modern crowdsourcing. These frameworks leverage patterns of agreement and disagreement among annotators to denoise the collected data, offering mathematically sound methods for aggregating judgments and mitigating the impact of noisy or inconsistent annotations. Over time, significant advances, such as Bayesian networks for modeling complex causal relationships (Pearl, 1988) and Hidden Markov models for sequential data (Rabiner, 1989), have extended these methods to represent dependencies and dynamically update beliefs based on new evidence.

The adoption of such probabilistic models has been instrumental in powering modern crowdsourcing platforms, such as Amazon Mechanical Turk, which democratize data annotation by enabling large-scale collaboration with non-expert annotators. These platforms have supported major AI advancements in recent years, including the creation of landmark datasets like ImageNet (Deng et al., 2009).

However, the reliance on crowdsourced human annotators, many of whom are amateur workers (Snow et al., 2008), introduces several practical challenges:

- **Annotation Quality:** Labels from non-expert annotators are often noisy and inconsistent (Ipeirotis et al., 2010; Gao et al., 2013), which can reduce the reliability of the aggregated output.
- **Scalability:** Recruiting and managing a large pool of annotators is expensive and time-intensive (Quinn and Bederson, 2011; Mason and Suri, 2012), particularly for large-scale datasets or domains requiring specialized knowledge (Krishna et al., 2016; Wang et al., 2012). The cost per label can make large-scale annotation projects prohibitively expensive.
- **Privacy and Safety:** Certain types of data are subject to strict privacy and safety constraints, such as those mandated by the Health Insurance Portability and Accountability Act (HIPAA) (Hamburg and Collins, 2012). These regulations prevent outsourcing annotation tasks involving sensitive data to external platforms (Breaux, 2013). Medical records, personally identifiable information, and proprietary data must be handled within secure and compliant environments, which limits the feasibility of crowd-based annotation (Kittur et al., 2013).

1.1.3 Programmatic Weak Supervision

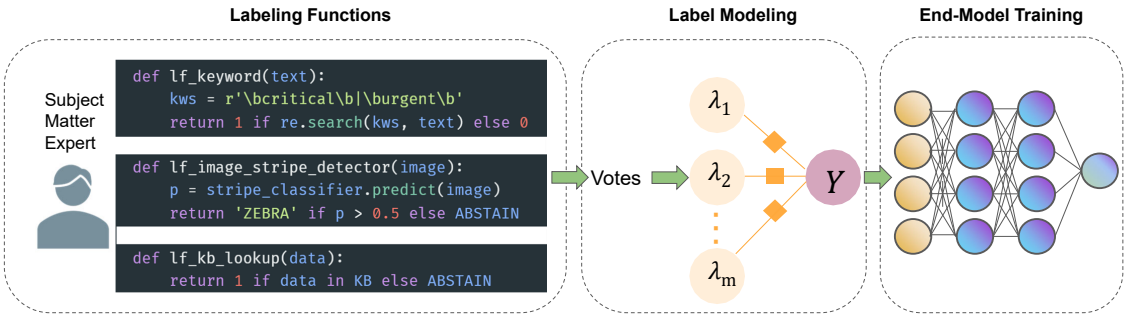

Figure 1.2: An overview of the programmatic weak supervision workflow. Subject matter experts encode their domain knowledge with multiple *labeling functions* $(\lambda_1, \lambda_2, \ldots, \lambda_m)$ such as keyword matching, attribute detection using classifiers, and knowledge base lookups. These labeling functions represent different ways of capturing heuristic rules or domain-specific knowledge. Each labeling function can vote for one label or abstain. The noisy votes are aggregated by a probabilistic *label model* to infer consensus probabilistic labels. These probabilistic labels are then used to train an end model, such as a neural network classifier.

Instead of relying on crowdworkers, Programmatic weak supervision enables domain experts to encode their knowledge through *labeling functions* (LFs). Frameworks like Snorkel (Ratner et al., 2016b, 2017) leverage these LFs to encode heuristic rules, pre-trained classifiers, or external knowledge bases, enabling domain experts to programmatically express task-specific knowledge.

Let us formally define the problem of knowledge expression in programmatic weak supervision. Consider a dataset $\mathcal{X} = \{x_1, ..., x_n\}$ with unobserved true labels $Y = \{y_1, ..., y_n\}$ where $y_i \in \mathcal{Y}$. The traditional supervised learning approach would require a complete set of labeled examples $\{(x_i, y_i)\}_{i=1}^n$. Instead, domain experts with relevant knowledge can help classify these examples but may not be able to directly provide labels for all instances.

In the traditional weak supervision setup, domain experts will create m labeling functions $\lambda_1, ..., \lambda_m$, where:

$$\lambda_j : \mathcal{X} \to \mathcal{Y} \cup \{\varnothing\} \tag{1.2}$$

These functions generate a matrix of noisy labels $\Lambda \in (\mathcal{Y} \cup \{\varnothing\})^{n \times m}$. The key challenge is to estimate the true labels Y given the noisy observations Λ. This is typically modeled as a probabilistic graphical model (Koller, 2009) and Ratner et al. describes the probabilistic label model as a factor graph:

$$p(Y, \Lambda) = \frac{1}{Z} \exp(\sum_{i=1}^{n} \sum_{j=1}^{m} \phi_{ij}(\Lambda_i^j, y_i; \theta)) \tag{1.3}$$

where ϕ_{ij} captures dependencies between labeling functions and true labels, and θ represents model parameters.

Once the label model parameters θ are learned, we can estimate probabilistic labels $\tilde{Y}$ for the entire dataset:

$$\tilde{y}_i = p(y_i | \Lambda_i; \theta) \tag{1.4}$$

These probabilistic labels can then be used to train any discriminative model $f_w : \mathcal{X} \to \mathcal{Y}$ such as ResNet (He et al., 2016) or Vision Transformers (Dosovitskiy et al., 2021) for image classification tasks, or BERT (Devlin et al., 2019) and RoBERTa (Liu et al., 2019b) for text classification tasks. The end model is trained by minimizing the expected loss where the expectation is over the probabilistic label distribution $\tilde{y}_i$:

$$\mathcal{L}(w) = \sum_{i=1}^{n} \mathbb{E}_{y \sim \tilde{y}_i}[\ell(f_w(x_i), y)] \tag{1.5}$$

where ℓ is an appropriate loss function, typically a cross-entropy objective function.

Programmatic weak supervision effectively bridges the gap between expert knowledge expression and modern deep learning architectures. In practice, it can enable the creation of sophisticated models even in scenarios where hand-labeled training data is scarce or unavailable. Programmatic weak supervision presents exciting opportunities for accelerating expert knowledge expression, enabling more efficient and scalable ways

to transfer expertise into machine learning systems.

1.1.4 Foundation Models: A Paradigm Shift in Knowledge Expression

Recent advances in foundation models (Bommasani et al., 2021b) have fundamentally transformed how we can capture and express domain expertise in machine learning systems. Large Language Models such as GPT (Brown et al., 2020), Llama (Touvron et al., 2023), and Claude (Anthropic, 2024), along with Vision-Language Models like CLIP (Radford et al., 2021a) and Florence (Yuan et al., 2021) and LLaVA (Liu et al., 2024), enable a more natural and flexible approach to knowledge expression. These models, pre-trained on massive datasets including The Pile (Gao et al., 2020), Common Crawl (Raffel et al., 2020), and internet-scale text-image pairs (Radford et al., 2021a), learn generalized representations that bridge the gap between human expertise and machine understanding.

One key innovation of foundation models lies in their ability to process and understand natural language instructions. This capability has evolved from simple prompt-response patterns to sophisticated reasoning techniques. In-context learning allows models to adapt to new tasks through examples (Brown et al., 2020), while chain-of-thought prompting enables step-by-step reasoning (Wei et al., 2022). Advanced approaches like Tree of Thoughts further enhance reasoning capabilities by exploring multiple solution paths (Yao et al., 2024). For visual tasks, CLIP demonstrates how natural language supervision can enable zero-shot recognition of novel concepts (Radford et al., 2021a), expanding the scope of knowledge expression to multi-modal domains.

Through prompting, domain experts can express intricate domain concepts, complex reasoning patterns, and specialized knowledge using familiar natural language. This capability dramatically lowers barriers to knowledge transfer - medical experts can

describe diagnostic criteria through detailed explanations, scientists can articulate experimental protocols step-by-step, and financial analysts can specify complex trading rules conversationally. The result is a more natural and comprehensive way to capture domain expertise compared to traditional programmatic approaches.

Despite these powerful capabilities, foundation model outputs can be noisy, inconsistent or contain hallucinated information (Huang et al., 2023). Their integration of foundation models with weak supervision frameworks represents a promising direction for enhancing their capabilities. Weak supervision's principled probabilistic framework can bolster the reliability of foundation model outputs while preserving their flexible knowledge expression advantages. This combination creates a powerful synergy - weak supervision provides systematic noise handling and uncertainty quantification, while foundation models enable natural language knowledge expression and complex reasoning.

The integration of foundation models with weak supervision frameworks can offer a few promising advantages:

- **Robust Knowledge Integration:** Weak supervision frameworks systematically aggregate foundation model outputs from various prompting mechanisms or external knowledge sources, calibrating their reliability while maintaining the benefits of natural language expression.
- **Multi-modal Knowledge Capture:** Multi-modal models like vision-language models expand knowledge expression to visual domains, with weak supervision providing principled noise handling across modalities.
- **Reduced Technical Barriers:** Natural language interaction removes programming requirements while weak supervision ensures reliability, making machine learning more accessible to domain experts.

Recent research validates this approach's potential for enhancing expert knowledge

expression. Studies demonstrate how language models can generate effective labeling functions (Smith et al., 2022) and how their outputs can strengthen weak supervision signals (Chen et al., 2022). By combining the flexibility of natural language prompts with robust frameworks for handling noise, this approach helps experts express their knowledge more effectively and in ways that can be applied to real-world problems. This integration represents a promising step toward creating tools that allow domain experts to share their insights easily while maintaining the reliability needed for machine learning systems to perform well in practice.

1.2 Improving Expert Knowledge Incorporation

Thesis Contributions and Organization

This dissertation aims to empower the expression of expert knowledge in machine learning systems by advancing weak supervision through the integration of foundation models. The core contributions are organized around three central directions:

1. Enhancing Weak Supervision via More Expressive Generative Models. Traditional weak supervision frameworks require labeling functions to output single-class predictions, limiting their expressiveness and failing to capture the natural ambiguity of domain expertise. In **Chapter 2**, I introduce the *Noisy Partial Label Model (NPLM)*, a novel probabilistic generative framework that allows labeling functions to output subsets of class labels. This formulation captures partial uncertainty in supervision and supports more nuanced and reusable knowledge sources. In this chapter, I provide theoretical identifiability guarantees, a efficient learning algorithm for NPLM, and empirical validation across text and vision classification tasks.

2. Efficient Adaptation of Foundation Models. While foundation models offer impressive generalization capabilities, their adaptation to downstream tasks can

be resource-intensive, and existing fine-tuning techniques often require storing and accessing full model parameters or additional adaptation modules during inference. **Chapter 3** proposes *Compositional Soft Prompting* ($\mathbb{CSP}$), a method that enables compositional zero-shot generalization in vision-language models by representing attributes and objects as learnable prompt tokens. This allows recognition of unseen attribute-object pairs without exhaustive supervision. To further improve efficiency, **Chapter ??** introduces *Neural Selective Fine-Tuning (NSFT)*, a parameter-efficient adaptation method that selectively fine-tunes submodules—specifically sub-Multilayer Perceptrons (MLPs)—within transformers. NSFT achieves strong downstream task performance while significantly improving inference-time efficiency, making it particularly suited for deployment in resource-constrained or latency-sensitive settings.

3. Synergizing Weak Supervision and Foundation Models through Prompted Weak Supervision. Despite their flexibility, current weak supervision systems often require domain experts to encode their knowledge into rigid, code-based labeling functions, which can pose a significant barrier for non-technical experts. This may increase manual workload and limit the integration of expert knowledge. Foundation models can enable supervision through natural language prompts, offering a new pathway to capture expert knowledge without requiring programming expertise. **Chapter 4** introduces **Alfred**, a prompted weak supervision system that leverages natural language interfaces to simplify training data creation. By allowing domain experts to encode their knowledge using natural language prompts instead of programming, **Alfred** lowers the technical barriers and enhances the usability of weak supervision frameworks. **Chapter 5** presents a structure-refining module that models inter-prompt dependencies via embedding similarities computed directly from foundation models, improving the accuracy of label aggregation. To standardize evaluation and promote future research, **Chapter 6** introduces *Alfred-Apps*, a comprehensive benchmark suite for prompted weak supervision, and *Agent Alfred*, a prototype agentic system that au-

tomates prompted supervision workflows using LLMs, with the goal of further reducing human effort in large-scale data annotation tasks.

Together, these contributions establish a novel framework for expert knowledge transfer in machine learning by combining the systematic noise modeling of weak supervision with the expressive capabilities of foundation models. The methods proposed herein have been validated in diverse domains, including visual recognition, text classification, and multimodal tasks, especially in settings with limited labeled data. This work advances the theory and practice of weak supervision and contributes to the construction of more adaptive, scalable, and accessible machine learning systems.

CHAPTER 2

Learning from Multiple Noisy Partial Labelers

In this chapter, I introduce the Noisy Partial Label Model (NPLM), a novel probabilistic generative framework that generalizes traditional weak supervision by allowing labeling functions to output subsets of class labels rather than single-class predictions. This capability enables the incorporation of more nuanced expert heuristics that were previously unsupported. The model supports a broader class of labeling functions—partial labeling functions (PLFs)—and provides theoretical guarantees and an efficient algorithm for learning from them. This work was presented at International Conference on Artificial Intelligence and Statistics 2022 with Tiffany Ding and Stephen H. Bach.

2.1 Introduction

The need for large-scale labeled datasets has driven recent research on methods for *programmatic weak supervision* (PWS), such as data programming (Ratner et al., 2016a, 2017), adversarial label learning (Arachie and Huang, 2021), learning rules from

labeled exemplars (Awasthi et al., 2020a), and weak supervision with self-training (Karamanolakis et al., 2021). In PWS, *labeling functions*, such as user-written rules and other heuristics, provide votes on the true labels for unlabeled examples or abstain from voting. Then, a label model, such as a probabilistic generative model or minimax game, is often used to estimate the true labels in a way that accounts for unknown differences in accuracies and other properties of the labeling functions. Finally, these estimated labels are used to train an end model on the unlabeled data to generalize beyond the information contained in the labeling functions. This approach has had recent success with applications in natural language processing (Mallinar et al., 2019; Safranchik et al., 2020), computer vision (Chen et al., 2019), medicine (Fries et al., 2019; Saab et al., 2020; Fries et al., 2021), and the Web (Bach et al., 2019b). However, all of these methods assume that labeling functions cast votes for *individual* classes. In this work, we propose to generalize PWS to support labeling functions that cast votes for a *subset* of classes, called partial labels. We refer to such labeling functions as *partial labeling functions* (PLFs). Our goal is to aggregate information from multiple partial labeling functions that are noisy (i.e., have imperfect accuracy) in order to estimate labels for unlabeled data.

Incorporating partial labels into PWS would enable users to take advantage of a wider range of domain knowledge. In typical PWS frameworks, only heuristics that are specific to one class can be incorporated. As a result, creating labeling functions requires careful task-specific engineering to avoid features that are shared by more than one class. For example, consider the task of classifying images of animals with a label from the set {`HORSE, TIGER, LION, ZEBRA`}. There are many useful heuristics that can be learned from other labeled data sets, such as detectors for claws or stripes (Lampert et al., 2009). Such heuristics divide the label space with multiple partitions. A claw detector could produce two partial labels: {`TIGER, LION`} if a claw is detected and {`HORSE, ZEBRA`} if not. Likewise, a stripe detector could output {`TIGER, ZEBRA`} if

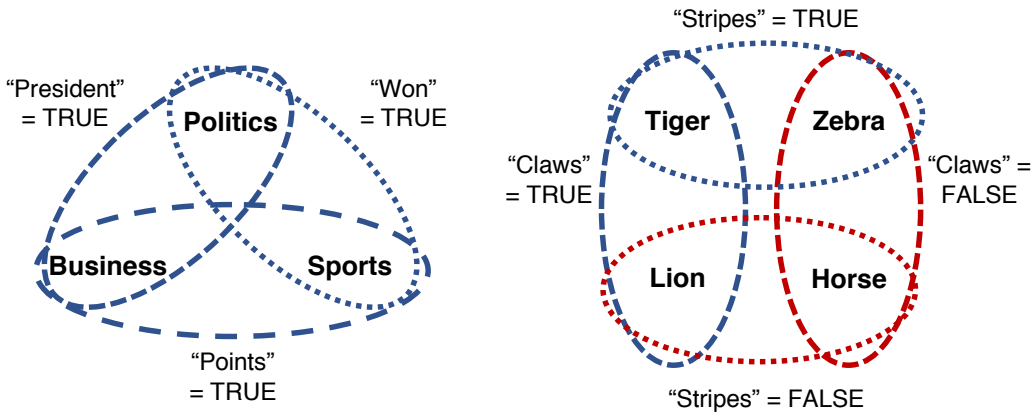

Figure 2.1: Examples of the expressivity of partial labeling functions. On the left, three functions each vote for two of three classes if they observe a particular token in a news article and abstain otherwise. On the right, two functions each vote for two of four classes if they detect a particular attribute in an image. Otherwise, they vote for the other two.

stripes are detected and {`HORSE, LION`} if not (Figure 2.1). However, these heuristics cannot be used as labeling functions in current PWS frameworks. More generally, we observe a need for partial labeling functions in many multiclass applications where users want to express heuristics that narrow down the set of possible class labels but are not specific to a single class.

Learning from multiple noisy PLFs is challenging because we must resolve ambiguity arising from three sources: (1) PLF imprecision, i.e., voting for a set of classes instead of a single class, (2) PLF inaccuracy, i.e., voting for a set of classes that does not contain the true class, and (3) conflict among multiple PLFs. A further requirement is that PWS frameworks should support labeling functions that abstain, meaning they can choose not to label certain examples. This is particularly critical for hand-engineered rules that might be highly specialized. A framework for learning from multiple noisy PLFs should therefore be able to resolve all these types of ambiguities in a principled way while also maintaining the expressive capabilities of existing PWS frameworks.

This problem setting is quite general and is related to multiple lines of work in machine learning, although each of them only addresses part of the problem considered

here. As mentioned above, previous PWS frameworks generally require labeling functions to provide a single class label (Ratner et al., 2016a, 2017; Arachie and Huang, 2021; Awasthi et al., 2020a; Karamanolakis et al., 2021; Safranchik et al., 2020). One exception is Snorkel MeTaL (Ratner et al., 2019b), which is capable of handling labeling functions with a multi-task tree structure where higher-level labeling functions are grouped into super classes that encompass fine-grained classes. This requirement of a tree structure makes modeling partial labels that divide the label space into overlapping subsets practically infeasible. There is also a wide body of work on learning from partial labels, also called superset learning (Jin and Ghahramani, 2002; Nguyen and Caruana, 2008; Luo and Orabona, 2010; Cour et al., 2011; Liu and Dietterich, 2012, 2014; Hüllermeier and Cheng, 2015; Cabannnes et al., 2020; Wang et al., 2020; Cabannes et al., 2021; Zhang et al., 2021c). In these settings, there is generally one partial label per example. The ambiguity in the imprecise labels in such settings can be resolved as a maximum likelihood or risk minimization problem. Many methods additionally learn the likely confusions between partial and true labels (Durand et al., 2019; Li et al., 2020a; Yan and Guo, 2020; Xie and Huang, 2021). However, such methods do not handle the case of multiple partial labelers that can disagree and abstain. Finally, some work on zero-shot learning (ZSL) creates attribute detectors that can be viewed as partial labelers (Farhadi et al., 2009; Lampert et al., 2009; Palatucci et al., 2009; Jayaraman and Grauman, 2014). PWS with partial labels can also be viewed as a generalization of the transductive ZSL setting (Xian et al., 2018; Wang et al., 2019), in which labelers are allowed to abstain and a class may be associated with multiple attribute values. Across all these areas, there remains a need for learning from multiple noisy partial labelers.

To address this issue, we propose a generalized PWS framework that supports partial labels and handles the additional ambiguity caused by the imprecise outputs of the PLFs. First, we introduce a probabilistic generative model that estimates the

agreement between the outputs of each partial labeling function and the latent, true label. Second, we show how to learn the model parameters efficiently for large datasets. For example, we can learn on 100k examples with 10 PLFs in one minute. Since PWS is inherently human-in-the-loop, fast iteration is crucial. Third, we prove that this model's parameters are generically identifiable up to label swapping under mild conditions on the PLFs. This result means that we can estimate the accuracy of each partial labeling function without access to ground truth labels in a principled way. Using the learned parameters, we can compute the posterior distribution over true labels for each example. These probabilistic training labels can then be used to train an end model in the same manner as other PWS frameworks.

We demonstrate this framework with experiments on three text and six object classification tasks. On the text classification tasks, we show that the additional flexibility provided by partial labelers enables heuristics that significantly improve over single-class labelers alone. We find an average 8.6 percentage point improvement in accuracy. On the object classification tasks, we find that modeling the accuracies of the PLFs explicitly enables us to achieve accuracy comparable to recent embedding-based ZSL methods using only pre-trained attribute detectors. These results provide a foundation for constructing and learning more modular, reusable knowledge sources for weak supervision.

2.2 Related Work

In the past few years, programmatic weak supervision (PWS) has emerged as a systematic approach to efficiently create labeled training data (Ratner et al., 2016a, 2017; Arachie and Huang, 2021; Awasthi et al., 2020a; Karamanolakis et al., 2021). A typical PWS framework consists of three stages. First, domain experts engineer weak supervision sources in the form of labeling functions, such as rules or classifiers related to the target task. Second, a label model, such as a probabilistic generative model, is

used to estimate the latent true labels using the labeling function outputs. Third, the estimated labels are used to train an end model that generalizes beyond the information in the supervision sources. The core of a typical PWS framework is the label modeling stage. The choice of label model determines what types of supervision sources are supported. Many frameworks are based on crowdsourcing methods (Dawid and Skene, 1979b; Nitzan and Paroush, 1982; Gao and Zhou, 2013), where providing a single label is a natural assumption. In the original data programming framework (Ratner et al., 2016a), labeling functions can output a single label or abstain. The label model is generative, meaning that each true label is a latent variable and the observed votes of the labeling functions are conditioned on the true labels. The parameters of the label model are learned by maximizing the marginal likelihood of the observed votes. Statistical dependencies such as correlations among the votes can be modeled, and methods exist to learn specific types of dependencies from unlabeled data (Bach et al., 2017a; Varma et al., 2019a).

The Snorkel MeTaL (Ratner et al., 2019b) framework extends data programming to learn across multiple, related tasks organized in a tree structure. For example, in a fine-grained named entity recognition task, one might use a set of labeling functions that vote on coarse-grained entity types and separate sets of labeling functions to further vote on the subtypes within each coarse type. The outputs of labeling functions at higher levels of the tree can be thought of as a restricted form of partial labels, in the sense that all labeling functions must follow the same tree-structured organization of the classes. In contrast, in our setting, each partial labeling function can organize the classes into its own, possibly overlapping groups.

Other PWS frameworks have approached labeling functions and the label modeling processes in different ways, but all so far assume that each labeling function votes for a single class. Adversarial label learning (Arachie and Huang, 2021), performance-guaranteed majority vote (Mazzetto et al., 2021b), adversarial multi class learn-

ing (Mazzetto et al., 2021a), and related work in semi-supervised ensemble learning (Balsubramani and Freund, 2015a, 2016b,a) solve minimax games based on assumed or estimated constraints on labeling function accuracies. Awasthi et al. (2020a) proposed learning from rules and exemplars for those rules, learning to downweight the confidence in the rules on data instances not similar to the exemplars. Karamanolakis et al. (2021) proposed integrating PWS with semi-supervised self-training.

Other work on learning with partial labels has focused on the case where there is a single partial label per example. Classifiers can be learned via maximum likelihood estimation (Jin and Ghahramani, 2002; Liu and Dietterich, 2012) or empirical risk minimization (Nguyen and Caruana, 2008; Luo and Orabona, 2010; Cour et al., 2011; Liu and Dietterich, 2014; Hüllermeier and Cheng, 2015; Cabannnes et al., 2020; Cabannes et al., 2021; Feng et al., 2020b). Many methods additionally learn the likely confusions between partial and true labels (Durand et al., 2019; Li et al., 2020a; Yan and Guo, 2020; Xie and Huang, 2021). Wang et al. (2020) proposed learning multiple partially labeled tasks simultaneously, in order to exploit structure among the tasks, but during training there is still only one partial label per prediction. Partial labels are also related to complementary labels (Ishida et al., 2017; Feng et al., 2020a), which are annotations that indicate which label the example does *not* have.

Our problem setting is also related to some forms of zero-shot learning (ZSL) (Xian et al., 2018; Wang et al., 2019). In zero-shot classification, a model learns to match semantic descriptions of classes to examples of those classes. Once learned, the model can be applied to novel classes. Many early approaches to ZSL created detectors for different attributes (Farhadi et al., 2009; Lampert et al., 2009; Palatucci et al., 2009; Jayaraman and Grauman, 2014). In the transductive setting (Xian et al., 2018; Wang et al., 2019), in which the target classes are known and unlabeled examples of them are available during model development, these detectors can be viewed as restricted partial labeling functions that always divide the label set into non-overlapping groups and

never abstain. More recently, much work on ZSL has moved away from relying entirely on attribute detectors, and recent work can be grouped into either embedding-based or generative-based methods (Pourpanah et al., 2020). Embedding-based methods align representation spaces between classes and examples in order to classify unlabeled data (Socher et al., 2013; Frome et al., 2013; Romera-Paredes and Torr, 2015; Xian et al., 2016, 2018; Wang et al., 2019). Some work, e.g., Liu et al. (2019a) and Liu et al. (2020), also learn to exploit and expand attribute-based information, but generally still do not use separate attribute detectors. On the other hand, generative-based ZSL methods generate examples of the unseen classes with deep generative models and then train a classifier with that data (Bucher et al., 2017; Verma et al., 2018; Felix et al., 2018; Sariyildiz and Cinbis, 2019; Xian et al., 2019; Narayan et al., 2020). In our experiments, we compare with transductive embedding-based methods, which are more similar to PWS because both involve trying to label a fixed, unlabeled data set. We leave incorporating zero-shot data generation into PWS for future work.

2.3 A framework for Partial Labeling Functions

Following prior work in PWS (Ratner et al., 2016a, 2017), our framework consists of three stages. First, we define partial labeling functions as sources of weak supervision (Section 2.3.1). Second, we propose and analyze a label model to aggregate their outputs (Section 2.3.2). Third, the learned label model is used to compute the posterior distribution for the true label of each unlabeled example, which is used to train a noise-aware classifier (Section 2.3.4).

2.3.1 Partial Labeling Functions

We propose generalizing labeling functions to *partial labeling functions* (PLF) in order to make use of many available weak supervision sources that are informative but not specific enough to identify a single class. PLFs can range in granularity, from

dividing the label space into two large groups down to identifying a specific class, i.e., a regular labeling function. This flexibility allows users to take advantage of many additional supervision signals, as we illustrate in our experiments.

A PLF G is a function that maps an unlabeled example to a proper subset of the possible labels or abstains by outputting the full set of all possible labels. Formally, our goal is to learn a classifier $C : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ is the space of inputs and $\mathcal{Y} = \{y_1, \ldots, y_k\}$ is the set of possible labels. A PLF is then a function $G : \mathcal{X} \to \mathcal{G} \subseteq \mathbb{P}(\mathcal{Y}) \setminus \{\emptyset\}$, where $\mathbb{P}(\mathcal{Y})$ is the power set of $\mathcal{Y}$. $G(X)$ is a partial label for $X \in \mathcal{X}$, i.e., the set of labels that the PLF indicates the example X could have (although this information could be incorrect). If $G(X) = \mathcal{Y}$, the PLF is said to abstain, because it provides no information about the true label. As described further in Section 2.3.2, a key characteristic of a PLF is its codomain $\mathcal{G}$, excluding when the PLF abstains. We denote this set of partial labels for a PLF G as $T(G) = \mathcal{G} \setminus \{\mathcal{Y}\}$. To ensure that our label model is well-defined, we will impose these conditions on $T(G)$: (1) each label $y \in \mathcal{Y}$ appears in at least one element of $T(G)$, and (2) no label $y \in \mathcal{Y}$ appears in every element of $T(G)$. These are very mild conditions that can easily be satisfied by adding a "dummy" output to the codomain $\mathcal{G}$ that the PLF might not actually produce. A PLF can be defined based on a variety of noisy supervision heuristics using domain knowledge and/or available resources, such as classifiers for related tasks. To better understand PLFs, consider the following text and object classification examples corresponding to Figure 2.1:

Example 1. Consider a news classification task where

$$\mathcal{Y} = \texttt{\{POLITICS, SPORTS, BUSINESS\}}$$

. In this task, some words can be very informative as supervision sources even if they do not narrow the example down to a specific class. For example, the word

"president" may frequently appear in both political and business contexts. We can construct a PLF G based on a simple token matcher for "president" such that $G : \mathcal{X} \to \{\{\texttt{BUSINESS}, \texttt{POLITICS}\}, \{\texttt{SPORTS}\}, \mathcal{Y}\}$. If the token "president" appears in the example X, then $G(X) = \{\texttt{BUSINESS}, \texttt{POLITICS}\}$. Otherwise, $G(X) = \mathcal{Y}$, i.e., G abstains, because the absence of the token is not enough to conclude anything about the label with high confidence. In this example, $T(G) = \{\{\texttt{BUSINESS}, \texttt{POLITICS}\}, \{\texttt{SPORTS}\}\}$. Notice here $\{\texttt{SPORTS}\}$ is a "dummy" label set to satisfy the conditions on $T(G)$ described above.

Example 2. Consider an object classification task where

$$\mathcal{Y} = \{\texttt{HORSE}, \texttt{TIGER}, \texttt{LION}, \texttt{ZEBRA}\}$$

. Following work in zero-shot learning, we can build a binary classifier for the visual attribute of having stripes by training on other classes of animals for which we already have labels. We can then use the classifier's output to define a PLF $G_1 : \mathcal{X} \to \{\{\texttt{TIGER}, \texttt{ZEBRA}\}, \{\texttt{HORSE}, \texttt{LION}\}\}$. For an example X, if the stripes detector returns a positive label, then $G_1(X) = \{\texttt{TIGER}, \texttt{ZEBRA}\}$. Otherwise, $G_1(X) = \{\texttt{HORSE}, \texttt{LION}\}$. We can similarly construct a PLF with a claw detector as $G_2 : \mathcal{X} \to \{\{\texttt{TIGER}, \texttt{LION}\}, \{\texttt{HORSE}, \texttt{ZEBRA}\}\}$.

PLFs are a generalization of the labeling functions used in prior work on PWS; traditional labeling functions can be represented as PLFs with codomain $\mathcal{G} = \{\{y_1\}, \ldots, \{y_k\}, \mathcal{Y}\}$. PLFs provide the additional flexibility to users of incorporating weak supervision heuristics with differing granularitities and ways of dividing the label space.

2.3.2 Label Model

In our framework, users provide two inputs: PLFs and unlabeled examples in $\mathcal{X}$. Like other PWS frameworks, at the core of our method is a probabilistic label

model that captures the properties of the weak supervision sources by representing the unknown ground-truth labels as latent variables. In this subsection, we propose and analyze a probabilistic label model for PLFs. Our label model is straightforward to define as a generalization of existing ones for labelers that provide a single label (Dawid and Skene, 1979b; Ratner et al., 2016a). It defines a correct output of a partial labeler as one that is consistent with the unknown, true label, i.e., that the true label is in the output set. If the partial labeler is trivially correct because it outputs the set of all possible labels, it is said to abstain, which is not counted towards its estimated accuracy.

While straightforward to define, using the model introduces new challenges. The first challenge is practical. Existing methods for optimizing the marginal likelihood of the label model in the single-label case do not work for PLFs. The matrix factorization approach proposed by Ratner et al. (2019b) exploits the fact that the matrix of agreements among labelers can be expressed as a product of the model parameters, but this does not hold in the PLF case. Bach et al. (2019b) proposed optimizing the likelihood in an auto-differentiation framework like PyTorch (Paszke et al., 2017). We find that naively applying this approach is impractically slow, and that carefully defining the computation graph leads to a 300× speedup.

The other challenge we address is theoretical. We consider when it is possible to learn the parameters of the model without access to ground truth in a principled way, i.e., when the model is identifiable. Prior theoretical work either on PWS or learning with partial labels is not applicable to our scenario. Prior work on identifiability for PWS does not consider partial labels (Ratner et al., 2019b; Safranchik et al., 2020), and new conditions and arguments are needed to handle them. Prior work on risk bounds for learning with partial labels does not address noise nor multiple labelers (Cour et al., 2011; Liu and Dietterich, 2014; Hüllermeier and Cheng, 2015; Feng et al., 2020c). Therefore, our main contributions in this section are a fast learning technique and a

theorem characterizing sufficient identifiability conditions for our model.

Setup For a classification task with input space $\mathcal{X}$ and label space $\mathcal{Y} = \{y_1, \ldots, y_k\}$, we are given m unlabeled examples $X = (X_1, \ldots, X_m)$ with unknown ground truth labels $Y = (Y_1, \ldots, Y_m)$ such that (X, Y) are i.i.d. samples from some distribution $\mathcal{D}$. We are also given n PLFs $G = (G_1, \ldots, G_n)$. We use G as shorthand for the $m \times n$ array of PLF outputs where $G_{ai} = G_i(X_a)$ when it is clear from context.

Joint Distribution We define a joint distribution $P(G, Y)$ over the outputs of the PLFs on X and the latent, true labels Y. Like prior work (Ratner et al., 2016a), we assume that the PLF outputs are conditionally independent given the true labels, i.e., the naive Bayes assumption. In practice this works well, but extending work on learning more complex distributions for other types of PWS is a potential direction for future exploration (Bach et al., 2017a; Varma et al., 2019a). Analogous to prior work (Ratner et al., 2019b), for each PLF G_i, we define parameters $\alpha_i \in [0, 1]^k$ and $\beta_i \in [0, 1]$. Each element α_{ij} is the accuracy of G_i on examples of class y_j, i.e., the probability that $y_j \in G_{ai}$ given that X_a has label y_j and G_{ai} is not $\mathcal{Y}$. β_i is the propensity of G_i voting, i.e., not abstaining. In other words, $\beta_i = P(G_{ai} \neq \mathcal{Y})$. In our framework, the class balance $P(Y)$ can either be a learned distribution or fixed. We assume that if a PLF G_i makes a mistake, it outputs an incorrect partial label from $T(G_i)$ uniformly at random.

To define the joint distribution $P(G, Y)$, for each PLF G_i we also need to refer to the sets in $T(G_i)$ that are consistent or inconsistent with each label. Let $N_{ij} = \{\mathcal{L} | y_j \in \mathcal{L} \text{ for } \mathcal{L} \in T(G_i)\}$ be the set of label sets in the codomain of G_i that contain label y_j (excluding $\mathcal{Y}$). Likewise, let $N_{ij}^{\mathrm{C}} = \{\mathcal{L} | y_j \notin \mathcal{L} \text{ for } \mathcal{L} \in T(G_i)\}$ be the set of label sets in the codomain of G_i that do not contain label y_j. Then, the joint distribution is

$$P(G, Y) = \prod_{a=1}^{m} P(Y_a) \prod_{i=1}^{n} P(G_{ai} | Y_a) \tag{2.1}$$

where

$$P(G_{ai}|Y_a = y_j) = \begin{cases} 1-\beta_i, & \text{if } G_{ai} = \mathcal{Y} \\ \frac{\beta_i \alpha_{ij}}{|N_{ij}|}, & \text{if } y_j \in G_{ai} \neq \mathcal{Y} \\ \frac{\beta_i(1-\alpha_{ij})}{|N_{ij}^{\mathrm{C}}|}, & \text{if } y_j \notin G_{ai}. \end{cases} \tag{2.2}$$

Learning Given the unlabeled examples and PLF outputs G, our goal is to estimate the parameters of $P(G, Y)$ (denoted collectively as Θ) and compute the posterior $P(Y|G)$ over the unknown labels. To estimate Θ, we maximize the marginal likelihood of the observed outputs of the PLFs:

$$\hat{\Theta} = \operatorname*{argmax}_{\Theta} P_{\Theta}(G) = \operatorname*{argmax}_{\Theta} \sum_{Y} P_{\Theta}(G, Y) \ . \tag{2.3}$$

This optimization is implemented in PyTorch (Paszke et al., 2017). The marginal log likelihood of a batch of examples is computed in the forward pass, and stochastic gradient descent is used to update the parameters. We find that the way the likelihood computation is implemented in the forward pass can lead to an orders-of-magnitude difference in training time. For every example, we need to compute its conditional likelihood for every class based on votes from every PLF. Naively, this will require three layers of for loops through examples, PLFs, and classes. We can speed up the computation by expressing the conditional log likelihood computation as a sequence of matrix operations. This optimization trades space for speed by precomputing intermediate values and taking advantage of vectorized matrix operations.

Let m be the number of instances in one batch, n be the number of PLFs and k be the number of classes. For each batch we precompute accuracy indicator matrices $\mathbf{AI} \in \{-1, 1\}^{m \times n \times k}$ and count matrices $\mathbf{N} \in \mathbb{Z}^{m \times n \times k}$ where entry $\mathbf{AI}_{a,i,j} = 1, \mathbf{N}_{a,i,j} = -\log|N_{i,j}|$ if class y_j is in the label subset output by the i-th PLF on the a-th example, and $\mathbf{AI}_{a,i,j} = -1, \mathbf{N}_{a,i,j} = -\log|N_{i,j}^{\mathrm{C}}|$ otherwise. We also precompute propensity

indicator matrices $\mathbf{PI} \in \{0,1\}^{m \times n}$ where entry $\mathbf{PI}_{a,i} = 1$ if the a-th instance received a non-abstaining vote (vote is not $\mathcal{Y}$) from the i-th PLF. Let $\mathbf{A} \in \mathbb{R}^{n \times k}$ be the log of the accuracy parameters and $\mathbf{B} \in \mathbb{R}^{n}$ be the log of the propensity parameters. We can map these parameters back to probability space as

$$\begin{aligned} \alpha_{i,j} &= \frac{\exp(\mathbf{A}_{i,j})}{\exp(\mathbf{A}_{i,j}) + \exp(-\mathbf{A}_{i,j})} \quad \text{and} \\ \beta_b &= \frac{\exp(\mathbf{B}_i)}{\exp(\mathbf{B}_i) + 1} \ . \end{aligned} \tag{2.4}$$

We extend $\mathbf{PI}$, $\mathbf{A}$, and $\mathbf{B}$ to $\mathbf{PI}^{\text{ext}}$, $\mathbf{A}^{\text{ext}}$, and $\mathbf{B}^{\text{ext}}$ in 3 dimensions, with $\mathbf{PI}$ replicated along the third axis k times, $\mathbf{A}$ replicated along the first axis m times, and $\mathbf{B}$ replicated along the first axis m times and third axis k times. Then, during each forward pass, we only need to calculate normalizing matrices $\mathbf{ZA} \in \mathbb{R}^{n \times k}$ and $\mathbf{ZB} \in \mathbb{R}^{n}$ for accuracy and propensity respectively where

$$\begin{aligned} \mathbf{ZA}_{i,j} &= -\log\left(\exp(\mathbf{A}_{i,j}) + \exp(-\mathbf{A}_{i,j})\right) \quad \text{and} \\ \mathbf{ZB}_i &= -\log(\exp(\mathbf{B}_i) + 1) \ . \end{aligned} \tag{2.5}$$

We similarly extend $\mathbf{ZA}$ and $\mathbf{ZB}$ to $\mathbf{ZA}^{\text{ext}}$ and $\mathbf{ZB}^{\text{ext}}$. During the forward pass we calculate the batch conditional log likelihood by first computing a 3-dimension tensor:

$$\underset{m \times n \times k}{\mathbf{T}} = \underset{m \times n \times k}{\mathbf{A}^{\text{ext}}} \odot \underset{m \times n \times k}{\mathbf{AI}} + \underset{m \times n \times k}{\mathbf{N}} + \underset{m \times n \times k}{\mathbf{B}^{\text{ext}}} + \underset{m \times n \times k}{\mathbf{ZA}^{\text{ext}}}$$

and then summing over the n PLFs:

$$\underset{m \times k}{\log P(G|Y)} = \sum_{n} \left(\underset{m \times n \times k}{\mathbf{ZB}^{\text{ext}}} + \underset{m \times n \times k}{\mathbf{PI}^{\text{ext}}} \odot \underset{m \times n \times k}{\mathbf{T}} \right) \tag{2.6}$$

where $\odot$ is element-wise multiplication. The marginal likelihood is then combuted by summing over the k possible classes. This modification allows us to remove for loops in our code from the computation graph, and instead use only optimized matrix

operations. This approach leads to a 300× speedup in training time compared to a naive approach. This speedup makes the framework practical for iterative PLF development. For example, learning with 100k examples and 10 PLFs on an Intel i5-6600k CPU takes one minute.

Identifiability An important theoretical question is whether it is reasonable to try to learn the parameters of $P(G, Y)$ even though Y is never observed. We answer this question affirmatively by showing that as long as the codomains of the PLFs are sufficiently targeted or diverse, it is possible to determine the parameters of the label model (up to label swapping) using only the distribution of PLF outputs $P(G)$, except for on a measure zero subset of the space of possible parameter values. This property is the strongest useful notion of identifiability for models with latent variables (Allman et al., 2015). A model whose parameters can be determined except for on a measure zero subset is called *generically identifiable*. *Label swapping* refers to the fact that unobserved classes in a latent variable model can be relabeled without changing the observed distribution. This means that the map going from the observed distribution of a label model with k classes to parameter values is at best $k!$-to-one and cannot be one-to-one even under ideal conditions. In practice, label swapping is not an issue because most PLFs are more accurate than random guessing. We state a condition on the PLF codomains that is sufficient for identifiability in the following theorem.

Theorem 1. The parameters of the model P(G, Y) described in Section 2.3.2 are generically identifiable up to label swapping provided that the collection G of partial labeling functions can be partitioned into three disjoint non-empty sets S_1, S_2, and S_3 such that, for sets $j = 1, 2$ and all classes $y \in \mathcal{Y}$, we can choose label sets $t_i \in T(G_i)$ satisfying $\bigcap_{G_i \in S_j} t_i = \{y\}$.

2.3.3 Proof of Theorem 1

Theorem 1 provides sufficient conditions for the generic identifiability of the label model described in Section 2.3.2. In this section, we prove this theorem. Our proof is non-trivial because our label model yields probability distributions that are a measure-zero subset of the distributions considered by Allman et al. (2009). Allman et al. (2009) allows each entry of the class-conditional distributions to be any value in the interval $[0, 1]$ such that the entries sum to 1, whereas our label model imposes additional algebraic constraints on the entries. Allman et al. (2009) establishes identifiability except for on a measure-zero subset of the distributions that they consider, but we are unable to directly apply their results because our family of distributions might be contained in the measure-zero subset they exclude. It is therefore necessary to establish that the set of distributions for which identifiability does not exist is of measure zero with respect to the distributions that can be produced by our label model, or, equivalently, the set of values of the accuracies $\alpha_{i,j}$, propensities β_i, and class balance $P(Y)$ for which identifiability does not exist has measure zero with respect to the set of all possible parameter values.

Background The key tool that we use in our proof is Kruskal's unique factorization theorem, which relies on the concept of Kruskal rank (Kruskal, 1977). The *Kruskal rank* of a matrix is defined to be the largest integer n such that every set of n rows is linearly independent. A useful fact is that a matrix with full row rank also has full Kruskal rank. Kruskal's theorem says that if, for $u = 1, 2, 3$, we have a $k \times r_u$ matrix M_u with Kruskal rank I_u, and these I_u satisfy

$$I_1 + I_2 + I_3 \geq 2k + 2 \tag{2.7}$$

then, given only the three-dimensional tensor M where entry (a, b, c) is given by

$$M(a, b, c) = \sum_{j=1}^{k} M_1(j, a) M_2(j, b) M_3(j, c) \tag{2.8}$$

we can recover the original matrices M_u.

Allman et al. (2009) makes the connection that the probability distribution of a latent variable model with three observed variables and one latent variable that takes on a finite set of values can be described by the matrix M. Each M_u can be intrepreted as a conditional probability matrix where row c is a probability distribution over the possible values of feature u given that the latent variable has value c.

In situations where there are more than three observed variables, variables can be combined to form "grouped" variables that satisfy the theorem conditions, if needed. In our case, it is possible that individual PLFs do not have codomains that are large and diverse enough to satisfy the Kruskal rank requirement. In these situations, the condition can be satisfied by amalgamating multiple PLFs to form a grouped PLF, which can be viewed as an observed variable with a codomain that is the Cartesian product of the codomains of its member PLFs.

We show that the conditions of Theorem 1 ensure that, for a generic choice of parameters in a k-class model, there is a tripartion of the PLFs such that two of the corresponding conditional probability matrices have full Kruskal rank (k) and the third conditional probability matrix has a Kruskal rank of at least 2. Thus, the conditions of Kruskal's unique factorization theorem are satisfied, so we can recover the conditional probability matrices, from which the accuracies $\alpha_{i,j}$, propensities β_i, and class balance $P(Y)$ can be computed by solving a system of equations.

Proof of Theorem 1. S_1, S_2, and S_3 partition the PLFs into three disjoint subsets. For $u = 1, 2, 3$, define some ordering to the PLFs in subset S_u so that $S_{u,i}$ gives

the i-th PLF in the subset. We will treat S_u as a "grouped" PLF with codomain $\mathcal{G}(S_u) = \{(t_1, t_2, ..., t_{|S_j|}) \mid t_1 \in \mathcal{G}(S_{u,1}), t_2 \in \mathcal{G}(S_{u,2}), ..., t_{|S_u|} \in \mathcal{G}(S_{u,|S_u|})\}$, where $\mathcal{G}(G)$ denotes the codomain of PLF G. Let M_u denote the $k \times |\mathcal{G}(S_u)|$ conditional probability matrix for the combined output of all PLFs in subset S_u, where each entry is a product containing some combination of β_i, $(1 - \beta_i)$, $\alpha_{i,j}$, $(1 - \alpha_{i,j})$, and normalizing constants. We assume that the class balance $P(Y)$ has positive entries and all PLFs have non-zero propensities β_i, because any extraneous class labels or non-voting PLFs would be removed. Define $\tilde{M}_1 = \mathsf{diag}(P(Y))M_1$, where $\mathsf{diag}(\mathbf{v})$ denotes the matrix with the entries of vector $\mathbf{v}$ along its main diagonal and zeros elsewhere. $P(G)$, the observed distribution of PLF outputs, corresponds to the three-dimensional tensor obtained from applying Equation 2.8 to $\tilde{M}_1$, M_2, and M_3. We will consider the Kruskal ranks of $\tilde{M}_1$, M_2, and M_3, which we respectively denote I_1, I_2, and I_3.

We first consider M_2. The (row) rank of a matrix A is equal to the largest integer n for which there exists an $n \times n$ submatrix of A that has a nonzero determinant. The determinant of such a submatrix is called an n-minor. M_2 has less than full row rank if and only if all of its k-minors are zero. This condition can be expressed as the nonvanishing of a polynomial in the entries of M_2, which are themselves functions of the label model parameters. In other words, the set of parameter values for which M_2 does not full row rank is the zero set of this polynomial. As described in Allman et al. (2009), so long as the polynomial is not identically zero, the parameter values yielding less than full row rank is a measure-zero subset of the full parameter space. To show that this polynomial is not zero for all values in the parameter space, it is sufficient to show that there exists at least one set of parameter values for which the polynomial is nonzero, or, equivalently, that there is a set of parameter values for which M_2 has full row rank.

The values of the propensities β_i and class balance $P(Y)$ do not affect row rank as long as they are nonnegative, as assumed above. We now show that there is a setting

of the accuracies $\alpha_{i,j}$ for which the Kruskal rank of M_2 is k. Set all $\alpha_{i,j} = 1$. By the conditions of Theorem 1, for each class c, there is an output in the codomain of S_2 for which c appears in all of the individual PLF outputs and no other class appears in all outputs. This implies the following two statements about the column in M_2 that is associated with this output: (1) the c-th entry of this column does not contain $(1-\alpha_{i,j})$ in its product, and (2) all other entries are products containing at least one $(1-\alpha_{i,j})$. When $\alpha_{i,j} = 1$, these entries containing $(1-\alpha_{i,j})$ are all zero. In other words, M_2 has k columns that are all zero except for a single entry, and the row containing this entry is different across the k columns. These columns form the basis of a column space of dimension k. For any matrix A, $\dim(\text{Col } A) = \dim(\text{Row } A) =$ row rank of A. Thus, the row rank of M_2 is k. Since M_2 has full row rank when all $\alpha_{i,j} = 1$, it also has full Kruskal rank. This shows that the polynomial whose nonvanishing determines whether M_2 has full row rank is not identically zero, so M_2 generically has full row and Kruskal rank.

We now consider $\tilde{M}_1$. The arguments applied to M_2 can be applied exactly to M_1, but we are interested in $\tilde{M}_1 = \mathsf{diag}(P(Y))M_1$. However, since we assumed that $P(Y)$ contains only positive entries, and multiplying each row of a matrix by a nonzero scalar does not change its row rank, the same arguments can be applied to $\tilde{M}_1$. We conclude that $\tilde{M}_1$ also generically has full Kruskal rank.

Finally, we consider M_3. The Kruskal rank of a matrix is less than two only if there are two rows that are scalar multiples of each other. This can happen in our model only when the class conditional accuracies for two classes are exactly equal, which corresponds to a measure zero subset of the parameter space. Thus, we can generically assume that M_3 has a Kruskal rank of $I_3 \geq 2$.

Since, generically, M_1 and M_2 have Kruskal ranks of k and M_3 has a Kruskal rank of at least 2, we have $I_1 + I_2 + I_3 \geq 2k + 2$, so Kruskal's unique factorization theorem tells us that we can recover $\tilde{M}_1$, M_2, and M_3 from $P(G)$, the observed distribution of

PLF outputs. Once $\tilde{M}_1$, M_2, and M_3 are known, the accuracies $\alpha_{i,j}$, propensities β_i, and class balance $P(Y)$ can be computed using algebraic manipulations. □

This theorem tells us that it is reasonable to try to estimate the PLF accuracies even though the true class labels are never observed. Our proof adapts ideas presented in Theorem 4 of Allman et al. (2009), which uses Kruskal's unique factorization theorem and feature grouping to establish conditions for the generic identifiability of a naive Bayes model with arbitrary parameters. Since the space of models we consider is equivalent to a measure zero subset of the parameters in an arbitrary naive Bayes model, an additional proof is needed to show that these parameters are generically identifiable. We develop a novel argument to show that the above is a sufficient condition for identifiability. We show that for any distribution satisfying the condition described in Theorem 1, we can construct matrices representing factors of the joint distribution over observations that generically have full Kruskal rank. This is sufficient to satisfy the conditions in Kruskal's theorem.

In words, the condition described in Theorem 1 requires that for each class y, we can select a label group from the codomain of each PLF in S_1 such that the intersection of these label groups contains only the class y. This condition also applies to S_2. One way to satisfy this condition is to create PLFs that produce single-class label groups. For example, if PLF G_i contains $\{1\}, \{2\}, ..., \{k\}$ in its codomain, then any set S_j that contains G_i will satisfy the Theorem 1 condition. However, even if no PLFs output any single-class label sets, it is still possible for the label model parameters to be identifiable because the condition can also be satisfied by using multiple PLFs with different codomains. Suppose that we want to show that the condition is satisfied for class 1 and we have $\{1, 2, 3\} \in T(G_1)$, $\{1, 3, 4\} \in T(G_2)$ and $\{1, 2, 4\} \in T(G_3)$. The intersection of these sets is $\{1\}$.

2.3.4 Noise-Aware Classifier

The final stage of our framework is to train a classifier. After $P(G, Y)$ is estimated with unlabeled data, we compute the posterior $P(Y|G)$. Then, we minimize the expected empirical risk with respect to this distribution. For classifiers that output probabilistic predictions, the loss function becomes the cross-entropy loss weighted by the posterior over true labels. As in other PWS frameworks (Ratner et al., 2016a, 2017, 2019b; Safranchik et al., 2020), many off-the-shelf neural networks can be chosen based on the task.

2.4 Experimental Results

We demonstrate benefits of incorporating partial labels into PWS on applications in text and object classification. In Section 2.4.1, we compare our framework with baselines that (1) use only traditional labeling functions and (2) heuristically aggregate partial labels without a probabilistic model. Our proposed approach significantly improves accuracy over both baselines. In Section 2.4.2, we use pretrained visual attribute detectors as PLFs for classifying unseen objects. Our framework achieves accuracy that is competitive with recent embedding-based transductive ZSL methods. While our framework is not designed specifically for ZSL, we present this comparison to demonstrate its flexibility and show another scenario where modeling the noise of multiple partial labeling functions can significantly improve performance relative to a heuristic approach. It shows that discrete attribute detectors can be competitive with recent ZSL approaches. These two sets of experiments are complementary, showing that our framework benefits both hand-engineered rules and partial labeling schemes defined in prior work.

We make available the code for our framework[1] and experiments.[2]

[1] `github.com/BatsResearch/nplm`
[2] `github.com/BatsResearch/yu-aistats22-code`

	SciCite		TREC-6		AG-News	
	ACC	F1	ACC	F1	ACC	F1
Supervised (Dev. Set)	78.5±1.1	75.5±1.5	88.3±1.6	76.2±2.0	80.1±1.7	79.9±1.9
LFs Only (w/o End)	65.1	44.7	22.0	23.8	33.0	25.1
NC (w/o End)	73.2	69.2	29.6	32.6	46.1	43.5
NPLM (w/o End)	71.5	69.4	38.2	43.0	51.1	49.4
LFs Only	78.6±0.7	76.7±0.8	67.2±0.9	69.3±1.0	79.8±0.9	79.6±0.8
NC	80.2±0.6	77.7±0.6	67.8±1.5	56.5±0.3	78.0±1.0	77.2±1.0
NPLM	81.4±1.3	79.5±1.3	85.0±0.8	85.7±0.7	85.0±0.5	84.7±0.5
NPLM vs. LFs Only	↑ 2.8	↑ 2.8	↑ 17.8	↑ 16.4	↑ 5.2	↑ 5.1
NPLM vs. NC	↑ 1.2	↑ 1.8	↑ 17.2	↑ 29.2	↑ 7.0	↑ 7.5

Table 2.1: Results for text classification with mean accuracy (ACC), macro F1 (F1) and 95% CIs.

2.4.1 Text Classification

We first evaluate the benefit of incorporating PLFs into text classification with hand-engineered rules.

Datasets We consider three datasets. First, SciCite (Cohan et al., 2019) is a citation classification dataset sourced from scientific literature. The corresponding task is to classify a citation as referring to either background information, method details, or results. Second, TREC-6 (Li and Roth, 2002) is a question classification dataset containing open-domain questions. The task is to classifiy each question as asking about one of six semantic categories. Finally, AG-News (Gulli, 2005) is a large-scale news dataset. The task is to classify each example as one of four topics.

```
def first_word_PLF(instance):
    '''
    Label by first word of question.
    ABBR - Abbreviation
    DESC - Description and concepts
    ENTY - Entities
    HUM - Human beings
    LOC - Locations
    NUM - Numeric values
    '''
    word = instance.split()[0].lower()
    if word == "who": return [HUM]
    elif word == "where": return [LOC]
    elif word == "when": return [NUM]
    elif word == "why": return [DESC]
    elif word == "how": return [DESC, NUM]
    elif word == "name": return [ENTY, HUM]
    else: return [ABBR, DESC, ENTY,
                  HUM, LOC, NUM]  # Abstain
```

Figure 2.2: A partial labeling function developed for the TREC-6 question-type task. It uses the first word of the sentence to possibly narrow down the set of labels.

PLF Development For text tasks, we develop PLFs by inspecting examples from the development set for each dataset (916 examples for SciCite, 389 for TREC-6, and 500 for AG-News). We implement heuristic rules as Python functions that take as input the example text and any available metadata (such as the name of the section in which the sentence appears for SciCite). Most rules rely on checking for keywords or other surface patterns in the text. Figure 2.2 shows an example for TREC-6, in which we use the first word of a question to vote on what type of question it is. This example illustrates some of the utility of PLFs. Some words like "who" are sufficient to reliabily identify a single class. Others like "how" greatly narrow the set of possible lables, even though they do not specify a single label.

Methods We evaluate methods for aggregating the outputs of the PLFs to train an end model. First, as a baseline, we consider using only the PLFs that are equivalent to traditional labeling functions, i.e., they always output one label or abstain. We call this baseline **LFs Only**. Second, as another baseline we use a heuristic called **Nearest Class (NC)**, which chooses the class with the highest number of compatible partial labels. This baseline is a generalized majority vote heuristic for PLFs. Finally, our method, called **Noisy Partial Label Model (NPLM)**, is our label model from

Section 2.3.2. In all cases, we use the estimated labels to fine-tune a pretrained BERT (Devlin et al., 2019) base uncased English model with a three-layer classification head. Following prior work (Ratner et al., 2016a, 2017), we use the expected cross entropy w.r.t. $P(Y|G)$ as our training loss. As ablations, we also report performance using the aggregated PLF outputs directly as predictions, without training an end model, denoted **(w/o End)**.

Results We report mean micro-averaged accuracy and macro-averaged F1 of the compared methods in Table 2.1 on the standard test sets. Results using the end model are shown with 95% confidence intervals obtained using five different random seeds. NPLM consistently improves F1 and accuracy relative to LFs Only (8.6 and 8.1 percentage points on average, respectively) and NC (8.5 and 12.8 percentage points on average, respectively). The performance advantage over LFs Only demonstrates the benefits of additional weak supervision that can be expressed as PLFs, and the advantage over NC demonstrates that the proposed label model is learning useful information. The ablated versions of the methods significantly underperform their counterparts, showing that in all cases the end model learns to generalize beyond the information contained in the weak supervision heuristics. Many of the errors are on examples for which all supervision sources abstain, where a label is chosen arbitrarily or according to the class prior $P(Y)$. For context, we also report the performance of the end model trained on the development set with ground-truth labels. NPLM significantly outperforms it in most cases. On TREC-6, the supervised baseline has a higher accuracy but much lower macro-averaged F1, indicating that our method does significantly better on the rarer classes.

	LAD (ACC)					
	Animals	Fruit	Vehicles	Electronics	Hairstyles	Avg.
ConSE—Norouzi et al. (2013)	36.9	29.8	37.5	28.3	24.6	31.4
ESZSL—Romera-Paredes and Torr (2015)	50.2	37.2	45.8	32.8	31.8	39.6
SynC—Changpinyo et al. (2016)	61.6	51.4	54.9	43.0	29.1	48.0
VCL—Wan et al. (2019)	75.4± 0.8	35.0± 1.0	62.4± 0.5	36.7± 0.5	33.8± 0.7	48.7± 0.3
QFSL—Song et al. (2018)	-	-	-	-	-	-
WDVSc—Wan et al. (2019)	97.2± 0.8	43.3± 1.3	82.1± 0.6	54.8± 1.1	31.1± 2.6	61.7± 0.6
NC (w/o End)	65.8	31.2	60.3	40.3	39.1	47.3
NPLM (w/o End)	86.0	38.7	73.5	51.8	45.9	59.2
NC	71.9±1.2	36.2±0.6	65.3±1.2	48.0±0.7	40.9±0.5	52.5±0.3
NPLM	87.6± 0.2	42.4± 0.8	77.0± 0.2	57.7± 0.7	46.9± 0.9	62.3± 0.2
NPLM vs. NC	↑ 15.7	↑ 6.2	↑ 11.7	↑ 9.7	↑ 6.0	↑ 9.8

Table 2.2: Results for object classification on LAD. We report mean accuracy (ACC) with 95% CIs across the five standard splits for each of the five subtasks.

	AwA2 (MCA)		
	U	S	$\mathcal{H}$
ConSE—Norouzi et al. (2013)	0.5	90.6	1.0
ESZSL—Romera-Paredes and Torr (2015)	77.8	5.9	11.0
SynC—Changpinyo et al. (2016)	90.5	10.0	18.0
VCL—Wan et al. (2019)	21.4	89.6	34.6
QFSL—Song et al. (2018)	66.2	93.1	77.4
WDVSc—Wan et al. (2019)	76.4	88.1	81.8
NC (w/o End)	47.7	-	-
NPLM (w/o End)	68.2	-	-
NC	43.1± 1.2	91.8± 0.2	58.6± 1.1
NPLM	71.1± 0.6	91.9± 0.1	80.1± 0.3
NPLM vs. NC	↑ 28.0	-	↑ 21.5

Table 2.3: Results for object classification on AwA2. We report mean class accuracy (MCA) with 95% CIs. We evaluate AwA2 in a generalized setting: S and U denotes MCA on the seen and unseen classes respectively. $\mathcal{H}$ is the harmonic mean.

2.4.2 Object Classification

In this task, we show how our framework can be used to model discrete visual attribute detectors, and that this approach can achieve results competitive with recent embedding-based ZSL methods. Although they have not been used often in recent ZSL work, discrete attribute detectors have benefits such as modularity and interpretability.

These experiments show that modeling them as PLFs with our unsupervised label model can lead to good accuracy.

Datasets We consider the Large-Scale Attribute Dataset (LAD) (Zhao et al., 2019) and Animals with Attributes 2 (AwA2) (Xian et al., 2018), which both provide class-level discrete visual attributes. LAD is a recently proposed attribute-based dataset with 78k instances that organizes common objects into five sub-datasets: electronics, vehicles, fruits, hairstyles and animals. For each sub-dataset, the classes are divided into five folds of seen and unseen classes, and average performance over all tasks is used as a benchmark for ZSL. AwA2 is a popular ZSL animal classification dataset consisting of ∼30k instances with 85 binary attributes, 40 seen classes, and 10 unseen classes.

PLF Development Following early work on zero-shot object classifcation (Farhadi et al., 2009; Lampert et al., 2009; Palatucci et al., 2009; Jayaraman and Grauman, 2014), we model each visual attribute in the datasets with a binary classifier. In all cases, the classifiers are trained on the seen classes for that task or fold, and the unseen classes are not used at all, not even as validation data. To create classifiers for LAD, we extract features from a ResNet-50 (He et al., 2016) pretrained on ILSVRC (Russakovsky et al., 2015), in order to compare fairly with prior work. For AwA2, we fine-tune a pretrained ResNet-101 on the seen classes. Each class is trained with respect to the class-wise attribute annotations on the training sets of the seen classes. We define the PLFs according to the provided attribute annotations from the respective datasets.

Methods We incorporate PLFs into the NC and NPLM methods as described in Section 2.4.1. We use examples of the unseen classes as our unlabeled data. In all cases, our end models are three-layer perceptrons trained on the extracted features (ResNet-50 for LAD and ResNet-101 for AwA2). As with the text data, we use the expected cross entropy with respect to $P(Y|G)$ as our training loss. Following the

literature, for LAD we evaluate in a strict zero-shot setting, meaning that the model is only evaluated on unseen classes. For AwA2, we evaluate in a generalized zero-shot setting, meaning that the model is evaluated on both seen and unseen classes, so we mix the unseen classes with estimated labels and seen classes with given labels during training. We compare with three recent transductive, embedding-based ZSL methods: QFSL (Song et al., 2018), VCL (Wan et al., 2019), and WDVSc (Wan et al., 2019). For context, we also report results from three standard inductive methods, ConSE (Norouzi et al., 2013), ESZSL (Romera-Paredes and Torr, 2015), SynC (Changpinyo et al., 2016), although they are at a disadvantage because they do not access the unlabeled data nor any information about the unseen classes. For LAD, we replicate and report the results of WDVSc and VCL using the same features.

Results We report the average results and 95% confidence intervals based on five random seeds in Table 2.2,2.3. Similar to the text classification tasks, NPLM significantly outperforms NC (an average of 9.8 percentage points on LAD and 21.5 percentage points on AwA2), and the ablations show that the end model generalizes beyond the PLFs. NPLM is also competitive with WDVSc, the top-performing ZSL method, either slightly underperforming or outperforming.

2.5 Limitations and Broader Impact

Our work expands the space of supervision sources that can be incorporated into PWS systems. Weak supervision is complementary to many other techniques, such as semi-supervised learning (Chapelle et al., 2009; Van Engelen and Hoos, 2020), transfer learning (Pan and Yang, 2009; Zhuang et al., 2020), active learning (Cohn et al., 1996; Settles, 2012), and zero-shot data generation (Bucher et al., 2017; Verma et al., 2018; Felix et al., 2018; Sariyildiz and Cinbis, 2019; Xian et al., 2019; Narayan et al., 2020). A limitation of our work is that exploring how partial labeling functions interact with

these techniques is left as future work. The same is true for complementary techniques within weak supervision, such as adversarial label learning (Arachie and Huang, 2021), learning rules from labeled exemplars (Awasthi et al., 2020a), and weak supervision with self-training (Karamanolakis et al., 2021). Additionally, while PWS can enable more rapid development, its dependence on heuristics introduces the potential for bias. For this reason, auditing any created models for potential negative impacts is as important, if not more important, in PWS as in traditional supervised learning (Saleiro et al., 2018; Mehrabi et al., 2019).

2.6 Dataset Information

SciCite (Cohan et al., 2019) is a citation purpose classification dataset containing 8243 train, 916 development and 1861 test instances of 3 categories sourced from scientific literatures and it is publicly available under Apache License 2.0.

TREC-6 (Li and Roth, 2002) is a publicly available dataset for research use and it is a question classification dataset containing a broad amount of open-domain questions from 6 semantic categories. Since the original dataset lack a validation/development set, we sample 389 instances from the training set to make a train/dev/test size split of 5063/389/500 respectively.

AG-News (Gulli, 2005; Zhang et al., 2015) is a publicly available dataset for research use. It is a large-scale news topic classification dataset containing 4 categories. We similarly sample 500 training instances as our development set. The train/dev/test size are 119.5k/500/7600 respectively.

LAD (Zhao et al., 2019) is a publicly available dataset for research use. It has approximately 78k instances that organizes common objects into five sub-datasets: electronics, vehicles, fruits, hairstyles and animals. Each sub-dataset is associated with 5 different seen/unseen class splits.

AwA2 (Xian et al., 2018) is a publicly available dataset for research use. It has 85 binary attributes for 50 animal classes. For our experiment, following the dataset authors, we adopt the proposed split that divides the 50 classes into 40 seen classes and 10 unseen classes.

All datasets we use are publicly available standard research datasets. These datasets generally do not contain personally identifiable information. Public figures are sometimes mentioned in the text datasets.

2.7 Additional Experiment Details

For the PLFs development and label modeling stage of the text classification task, the experiments are set on a local PC with Intel i5-6600k CPU and 32 GB of RAM. For the discriminative modeling and PLFs development that involves neural network inference/training for the object classification task, we perform our experiments on virtual computing instances with Intel Xeon E5-2698 v4 CPU, 1 NVIDIA V100 GPU or 1 NVIDIA RTX 3090 GPU and 32 GB of RAM.

2.7.1 Text Classification

For both LFs Only and NPLM, following prior practices in programmatic weak supervision (**?**Bach et al., 2019b), we filter the training instances by only retaining ones with at least one PLFs/LFs votes and the filtered instances are used for LF-Only/NPLM label and end model training. For the optimization of LFs Only and NPLM, we use a initial learning rate of 0.01 and a reduce-learning-rate-on-plateau learning rate scheduler with decreasing factor of 0.1. We train the NPLM/LFs Only Label models for 5 epochs.

For the end model, we adopt a pretrained(English) BERT-base model (Devlin et al., 2019) (bert-base-uncased) with a 3-layer classification head. The model is implemented with AllenNLP (Gardner et al., 2017). The 3-layer classification head has a hidden size

of dimension 256 with LeakyReLU as activation, batch normalization and a dropout layer with 50% probability. For all of the end models, we use AdamW (ADAM with weight decay) (Loshchilov and Hutter, 2019) optimizer and we train the models with a 16 training batch size. For AG-News, we train 20 epochs and a starting learning rate of 5e-7 and we set the gradient clipping threshold to 2.0. For TREC-6, we train with a total of 25 epochs with a 2.0 gradient clip and 3e-5 initial learning rate. For SciCite, we train the model with a total of 20 epochs with a 2.0 gradient clip and 3e-6 initial learning rate. The best end model is picked based on the best validation macro-averaged F1. Please refer to <dataset>_pipeline.ipynb, run_end_model_<dataset>.py, and /end/backbones/text_classifiers.py in the experiment code repository for corresponding code.[3]

2.7.2 Object Classification

We use AwA2 for our generalized zero-shot experiments and the sub-tasks of LAD for zero-shot evaluations. For AwA2, we follow the *proposed splits* and we follow the seen/unseen class split guide noted by the original authors for LAD. We adopt previous practices and guidelines in evaluating generalized AwA results, using average per class top-1 accuracy (or mean class accuracy, MCA) as the main performance metric for both unseen and seen classes at test and then report the harmonic mean. For LAD, we follow the authors' practices to report the average accuracy over the 5 sub-categories, each with 5 different seen/unseen class splits.

While the train/validation split among the seen classes are given in AwA2, LAD does not supply a train/validation split among the seen classes. We randomly sample at least one and at most 10% of the seen classes as validation classes for the detector and the rest seen data as training.

For both AwA2 and LAD, we train detectors for each attribute with a 3 layer MLP.

[3] `github.com/BatsResearch/yu-aistats22-code`

We consider setting the hidden dimensions to either 512 or 1024. We select the size that gives the higher minimum per-class accuracy. (In other words, whichever improves the worst-scoring class the most.) We use ILSVRC-pretrained ResNet-50 features for LAD and seen class-finetuned ResNet-101 features on AwA2. The minority class is balanced by oversampling. We also apply batch normalization and 50% dropout during training at each layer. The activation function used is LeakyReLU. For the optimization, we adopt a Adam optimizer with initial learning rate from 1e-4 and a multi-step learning rate scheduler. We train the detector with respect to {100, 300, 500} epochs with a learning rate scheduling step size of {30, 80, 200} respectively. Best model is selected based on best validation accuracy measured on the held-out seen classes.

For the NPLM label model, we use Adam optimizer with a reduce-learning-rate-on-plateau learning rate scheduler. The full set of hyperparameters can be found in the experiment code repository. The end model is a 3-layer MLP with both hidden layers size being 1024. We apply batch normalization and 50% dropout at each layer and it is activated with LeakyReLU. We optimize the end discriminative model with a initial learning rate of 1e-4 and we adopt a reduce-learning-rate-on-plateau learning rate scheduler with decreasing factor of 0.1. For the generalized task on AwA2, we train the model for 11 epochs. For LAD, we pick the best model with the lowest training loss. Same as the text tasks, the training objective is soft cross entropy.

2.8 PLFs for Text Classification

In this section, we list the detailed partial labeling functions involved in our experiments. Corresponding code for PLFs can be also found in the supplementary code as ipython notebook files.

For text experiments, for faster and more convenient PLFs development, we developed an abstraction class **WeakRule**(exec_module, label_maps) for each PLFs, which

has two crucial arguments/elements: exec_module that is the decision function programmatically defined and label_maps that map the result from the decision function to a partial label group.

To make development process of some rules even simpler, we develop a further abstraction inherited from class **WeakRule**, **BinaryRERule**. **BinaryRERule** will exam if specified regular expression match a given sentence and give either positive/negative feedback for each partial label groups or positive/abstain feedback if it is a unipolar Binary PLF (meaning that it only cast supervision signals onto a single partial label group or abstains).

For SciCite (10 PLFs):

Label Index Mapping:

0 - Background 1 - Method 2 - Result

```
def df1(instance):
    sen = instance['preprocessed_string'].lower()
    if ' we ' in sen
        or ' our ' in sen
        or ' by us ' in sen:
        return 1
    return -1
firstperson_rule = WeakRule(
    exec_module=df1,
    label_maps={0:[0], 1:[1,2]})

def sectionTitleRule(instance):
    results_pat = 'result|discussion|Lconclusion|observation'
    method_pat = 'method|approach|experiment|evaluation'
    intro_pat = 'introduction|background'
    title = str(instance['sectionName']).lower()
    if re.search(method_pat, title):
        return 1
    elif re.search(intro_pat, title):
        return 0
    elif re.search(results_pat, title):
        return 2
    return -1
sectionTitle_rule = WeakRule(
    exec_module=sectionTitleRule,
    label_maps={0:[0], 1:[1], 2:[0, 2]})

def df2(instance):
    sen = instance['preprocessed_string'].lower()
    if ' result ' in sen
       or ' results ' in sen:
        return 1
    return 0
result_rule = WeakRule(
    exec_module=df2,
    label_maps={0:[0,1], 1:[2]})
```

```
def length_citation(instance):
    m = instance['citation']
    if len(m.split(';')) > 2:
        return 1
    return -1
cit_len_rule = WeakRule(
    exec_module=length_citation,
    label_maps={0:[2], 1:[0,1]})

def df3(instance):
    def result_related(inst):
        patterns = ['equivocal result',
                    'similar result',
                    'same result',
                    'different result',
                    'expected result']
        for pattern in patterns:
            if pattern in inst:
                return 1
        return -1
    return result_related(instance['preprocessed_string'].lower())
res_rule = WeakRule(
    exec_module=df3,
    label_maps={0:[1], 1:[0,2]})

re_patt_0 = 'using|measuring|used|the method|' +
'data|state-of-art|calculated|applied|' +
'according to|approach'
simple_re_rule_1 = BinaryRERules(
    re_pattern=re_patt_0,
    preproc=lambda inst:inst['string'],
    label_maps={0:[0,2], 1:[1]}, unipolar=False)

def df4(instance):
    def comparison(inst):
        patterns = [
            'in line with',
            'discordant with',
            'consistent with',
            'keeping with',
            'accordance with',
            'agreement with',
            'similar with',
            'compared to',
            'contrast to',
            'contrary to',
            'comparable to',
            'contradict to',
            'affirmed by',
            'supported by',
            'in support of',
        ]
        for pattern in patterns:
            if pattern in inst:
                return 1
        return -1
    return comparison(instance['preprocessed_string'].lower())
resultp_rule = WeakRule(
    exec_module=df4,
    label_maps={0:[0,1], 1:[2]})

def df5(instance):
    def match(inst):
        patterns = [
            'has been',
            'in order to',
```

```
            'considered to',
            'initially',
            'even if',
            'have shown',
            'has shown'
        ]
        for pattern in patterns:
            if pattern in inst:
                return 1
        return -1
    return match(instance['preprocessed_string'].lower())
add_rule = WeakRule(
    exec_module=df5,
    label_maps={0:[2], 1:[0,1]})

re_patt_1 = 'our result|this is in keeping|' +
'with previous|this study differ|' +
'this is in agreement with|this conclusion|' +
'this finding|' +
'similar result.*(found|observed|obtained)'
re_tb = BinaryRERules(
    re_pattern=re_patt_1,
    preproc=lambda inst:inst['string'],
    label_maps={0:[0,1], 1:[2]}, unipolar=True)

re_patt_2 = 'we (employ|utilize)|iap-as|' +
'metaanalyses|temporal transition|' +
'was estimated|quantitative (analyses|analysis)' +
'|this procedure|implementation|sequence analysis' +
'|regularization method|were analysed|we adopt|' +
'bayesian|were sampled|quantitative method|fracture|' +
simulating|this design|algorithm|developed|' +
'model performance|(was|were) evaluated|as control' +
'|scheme|control management|(was|were|is|are) measured' +
'|the rats|the pigs|we appl'
re_tm = BinaryRERules(
    re_pattern=re_patt_2,
    preproc=lambda inst:inst['string'],
    label_maps={0:[0,2], 1:[1]}, unipolar=True)
```

For TREC-6 (16 PLFs):

Label Index Mapping:

0 - Abbreviation 1 - Description and concepts 2 - Entities 3 - Human beings 4 - Locations

5 - Numeric values

```
def first_word(instance):
    st = instance['string']
    word = st.split()[0].lower()
    if word == "who":
        return 0
    elif word == "where":
        return 1
    elif word == "when":
        return 2
    elif word == "why":
        return 3
    elif word == "how":
        return 4
    elif word == "name":
        return 5
    else:
        return -1
```

```
first_word_rule = WeakRule(
    exec_module=first_word,
    label_maps={
        0: [3],
        1: [4],
        2: [5],
        3: [1],
        4: [1, 5],
        5: [2, 3],
        6: [0]
    })

def called(instance):
    st = instance['string'].lower().split()
    if "called" in st:
        return 1
    return -1
r1 = WeakRule(
    exec_module=called,
    label_maps={
        0: [0, 1, 5],
        1: [2, 3, 4]
    })

def mean(instance):
    st = instance['string'].lower().split()
    if "mean" in st or "meaning" in st:
        return 1
    return -1
r2 = WeakRule(
    exec_module=mean,
    label_maps={
    0: [2, 3, 4, 5],
    1: [0, 1]
    })

def abbre(instance):
    st = instance['string'].lower()
    if "stand for" in st or "abbreviat" in st:
        return 1
    return -1
r3 = WeakRule(
    exec_module=abbre,
    label_maps={
        0: [1, 2, 3, 4, 5],
        1: [0]
    })

def desc(instance):
    st = instance['string'].lower()
    tokens = st.split()
    if "definition" in tokens or \
            "come from" in st or \
            "origin" in tokens:
        return 1
    return -1
r4 = WeakRule(
    exec_module=desc,
    label_maps={
        0: [0, 2, 3, 4, 5],
        1: [1]
    })

def enty(instance):
    subject = get_subject(instance['string'])
    if subject in ("animal", "body",
```

```
                    "color", "creative",
                    "currency", "disease",
                    "event", "food", "instrument",
                    "language", "letter", "plant",
                    "product", "religion",
                    "sport", "substance", "symbol",
                    "technique", "term",
                    "vehicle", "word"):
            return 1
    return -1
r5 = WeakRule(
    exec_module=enty,
    label_maps={
        0: [0, 1, 3, 4, 5],
        1: [2]
    })

def loc(instance):
    subject = get_subject(instance['string'])
    if subject in ("city", "country", "mountain", "state", "capital"):
        return 1
    return -1
r7 = WeakRule(
    exec_module=loc,
    label_maps={
        0: [0, 1, 2, 3, 5],
        1: [4]
    })

def num(instance):
    st = instance['string'].lower()
    if "what year" in st:
        return 1
    return -1
r8 = WeakRule(
    exec_module=num,
    label_maps={
        0: [0, 1, 2, 3, 4],
        1: [5]
    })

def num1(instance):
    st = instance['string'].lower()
    if "how many" in st or 'how much' in st or 'how old' in st:
        return 1
    return -1
r9 = WeakRule(
    exec_module=num1,
    label_maps={
        0: [0, 1, 2, 3, 4],
        1: [5]
    })

whatmean = BinaryRERules(
    name='what_*_mean',
    re_pattern='what.*mean',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[0, 2, 3, 4,5], 1:[1]}, unipolar=True)

desc_r1 = BinaryRERules(
    name='what_*_',
    re_pattern='what.*use of|what.*origin of|why do',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[0, 2, 3, 4,5], 1:[1]}, unipolar=True)
```

```
numpattern1 = BinaryRERules(
    name='num_patt',
    re_pattern='how far|what.*birthday|how long|how deep|'+
    'when did|when was|how tall|what month|population|toll'+
    '|how big|how long|what year',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[0,1, 2, 3, 4], 1:[5]}, unipolar=True)

descpatt_1 = BinaryRERules(
    name='num_patt',
    re_pattern='what is the origin|what is the history'+
    '|what.*mean|how do you buy|what is the difference'+
    '|how can I|how do I|what effect',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[0, 2, 3, 4,5], 1:[1]},
    unipolar=True)

descpatt_2 = BinaryRERules(
    name='num_patt',
    re_pattern='how.*tell|how d.*affect|how do.*work'+
    '|how do you fix|how do you get|how do you find'+
    '|how do I find|how.*made',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[0, 2, 3, 4,5], 1:[1]}, unipolar=True)

newhow = BinaryRERules(
    name='num_patt',
    re_pattern='how do|how was|how are|how is'+
    '|how was|how could|how can',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[0, 2, 3, 4,5], 1:[1]}, unipolar=True)

wsf = BinaryRERules(
    name='what_*_sf',
    re_pattern='what.*stand for',
    preproc=lambda inst:inst['string'].lower(),
    label_maps={0:[1, 2, 3, 4,5], 1:[0]}, unipolar=True)
```

For AG-News (11 PLFs):

Label Index Mapping: 0 - World 1 - Sports 2 - Business 3 - Science/Technology

```
def df1(instance):
    entity_doc = instance['sen_ner']
    if 'EVENT' in  entity_doc and 'GPE' in entity_doc:
        return 1
    return -1
gpe_event_title = WeakRule(
    name='gpe+event_title',
    exec_module=df1,
    label_maps={0:[0,2,3], 1:[1]})

def df2(instance):
    entity_doc = instance['title_ner']
    if 'LOC' in entity_doc:
        return 1
    return -1
loc_title_rule = WeakRule(name='loc_title',
    exec_module=df2, l
    abel_maps={0:[1,2,3], 1:[0]})

sp0 = BinaryRERules(
```

```
    name='sports_names',
    re_pattern='kelvim escobar|red sox|'+
    'formula one|grand prix|svetlana kuznetsova'+
    '|billy wagner| kobe | beckham|johnny damon'+
    '|robin ventura|olivier panis',
    preproc=lambda inst:inst['sen_lemma'].lower(),
    label_maps={0:[0,2,3], 1:[1]}, unipolar=True)

def bexclusive_lemmas(instance):
    business_keywords = {'profit', 'bankrupt', 'yen', 'financial'}
    doc = instance['sen_lemma'].lower().split()
    for word in doc:
        for keyword in business_keywords:
            if word.startswith(keyword):
                return 1
    return -1
bexclusive_lemmas_rule = WeakRule(
    name='bus_lemma',
    exec_module=bexclusive_lemmas,
    label_maps={0:[0,1,3], 1:[2]})

tech_patt = 'space.com|space station|'+
    'network authentication|(python|java|matlab|c) developer|'+
    'application|virus|browser hijack|search engine|internet-based|'+
    'windows update|smart phone|source code|mangement software'+
    '|software.*develop|internet connection|interactive gam'+
    '|game console|transfer datum|internet security|g network'+
    '|internet company|storage capacity|music player|microsystem'+
    '|comsumer electronic|operat.*system|wireness network|motherboard'+
    '|spacecraft|malicious program|video game'
techr = BinaryRERules(name='tech_terms',
    re_pattern=tech_patt,
    preproc=lambda inst:inst['sen_lemma'].lower(),
    label_maps={0:[0,1,2], 1:[3]}, unipolar=True)

def exclusive_lemmas(instance):
    world_pol_keywords = {'mideast', 'iraq',
                          'baghdad', 'pakistan',
                          'afghan', 'kurd', 'arab',
                          'egypt', 'iran', 'turkey',
                          'syria', 'bahrain',
                          'israel', 'jordan',
                          'kuwait', 'lebanon',
                          'oman', 'palestine',
                          'qatar', 'saudi', 'uae',
                          'yemen' ,'chechnya'}
    world_pol_keywords |= {'al-qaeda', 'taliban'}
    world_pol_keywords |= {'hostage', 'abduct',
                           'hijack'}
    world_pol_keywords |= {'invasion', ' coup ',
                           'curfew', 'army', 'troop',
                           'peace', 'militant', 'missile'}
    world_pol_keywords |= {'murder', 'death'}
    sports_keywords = {'baseball', 'football', 'soccer',
                       'hockey', 'basketball',
                           'tennis', 'golf'}
    sports_keywords |= {'stadium', 'arena'}
    sports_keywords |= {'season', 'playoff', 'tournament'}
    sports_keywords |= {'mlb', 'nfl', 'nba', 'mls',
                        'nhl', 'ncaa', 'league', 'racing'}
    sports_keywords |= {'premiership'}
    sports_keywords |= {'quarterback', 'centerback',
```

```
                        'fullback', 'pitcher'}
    business_keywords = {'profit', 'bankrupt', 'financial'}
    bt_keywords = {'robot', 'robotic'}
    bt_keywords |= {'web', 'internet'}
    bt_keywords |= {'linux'}
    bt_keywords |= {'stem-cell', 'biotechnology'}
    bt_keywords |= {'xbox', 'playstation'}
    bt_keywords |= {'microsoft'}
    bt_keywords |= {'space', 'nasa'}
    bt_keywords |= {'adobe', 'ipod', 'apple', 'xerox', 'ibm'}
    doc = instance['sen_lemma'].lower().split()
    for word in doc:
        for keyword in bt_keywords:
            if word.startswith(keyword):
                return 3
    for word in doc:
        for keyword in world_pol_keywords:
            if word.startswith(keyword):
                return 0
    for word in doc:
        for keyword in sports_keywords:
            if word.startswith(keyword):
                return 1
    for word in doc:
        for keyword in business_keywords:
            if word.startswith(keyword):
                return 2
    return -1
exclusive_lemmas_rule = WeakRule(
    exec_module=exclusive_lemmas,
    label_maps={0:[0], 1:[1], 2:[2], 3:[2,3]})

def df3(instance):
    entity_doc = instance['title_ner']
    if 'PERSON' in  entity_doc and 'GPE' in entity_doc:
        return 1
    return -1
person_gpe_title_rule = WeakRule(
    name='p+gpe_title',
    exec_module=df3,
    label_maps={0:[2, 3], 1:[0, 1]})

def df4(instance):
    entity_doc = instance['sen_ner']
    if 'PERSON' in  entity_doc and 'EVENT' in entity_doc:
        return 1
    return -1
person_event_sen_rule = WeakRule(
    name='person+event_sen',
    exec_module=df4,
    label_maps={0:[2,3], 1:[0,1]})

def df5(instance):
    entity_doc = instance['title_ner']
    if  'PRODUCT' in entity_doc:
        return 1
    return -1
sen_prod_t_rule = WeakRule(
    exec_module=df5,
    label_maps={0:[0,1], 1:[2,3]})

dpw2 = BinaryRERules(
    name='dpw2',
    re_pattern='minister|chairman',
    preproc=lambda inst:inst['sen_lemma'].lower(),
```

```
    label_maps={0:[1,3], 1:[0,2]}, unipolar=True)

intnr = BinaryRERules(
    name='tech',
    re_pattern='internet',
    preproc=lambda inst:inst['sen_lemma'].lower(),
    label_maps={0:[1,2], 1:[0,3]}, unipolar=True)
```

2.9 Conclusion

In this chapter, I addressed the limitation of limited expressiveness of labeling functions in conventional weak supervision frameworks, which restrict experts to single-class outputs and force them to simplify their nuanced knowledge into rigid rules. I introduced a novel probabilistic generative model that allows noisy labeling functions to output subsets of possible class labels, thereby capturing the natural complexity of domain expertise more effectively. Our empirical evaluations demonstrate that this approach improves the performance of weak supervision on text classification tasks and enables pre-trained attribute detectors to perform competitively in transductive zero-shot learning for object classification. By expanding the expressiveness of labeling functions, we reduce the need for exhaustive rule creation and empower domain experts to convey their knowledge more flexibly. This advancement lays a robust foundation for more efficient and scalable weak supervision systems towards the goal of enhancing expert knowledge integration in machine learning models.

CHAPTER 3

Learning to Compose Soft Prompts for Compositional Zero-Shot Learning

This chapter presents Compositional Soft Prompting ($\mathbb{CSP}$), a technique designed to enable vision-language models to generalize to unseen attribute-object combinations in a zero-shot setting. By learning distinct prompt embeddings for attributes and objects, $\mathbb{CSP}$ allows models like CLIP to handle novel compositions without requiring exhaustively labeled examples for every combination. This work is presented at the International Conference on Learning Representations 2023 with Nihal V. Nayak and Stephen H. Bach.

3.1 Introduction

Compositionality is the long-standing goal of artificial intelligence of creating new concepts by combining existing primitive concepts (Chomsky, 1956; Fodor and Pylyshyn, 1988; Hupkes et al., 2020; Lake and Baroni, 2018; Marcus, 2003). The practical advantage of compositionality for deep neural networks lies in the ability to build new classifiers by combining existing classifiers. In this work, we consider compositional

zero-shot learning, a classification task where the model learns to predict unseen or novel compositions of primitive concepts (Naeem et al., 2021; Nagarajan and Grauman, 2018; Purushwalkam et al., 2019). Research on compositional zero-shot learning in language and vision focuses on attribute-object compositions such as `old tiger` and `young tiger`, where `tiger` is the object category described by the attributes `old` and `young`.

Existing methods for compositional zero-shot learning typically map attributes and objects to pretrained word embeddings and use a pretrained image encoder backbone to jointly align the image and the attribute-object text representations to learn compositionality (Li et al., 2020b; Mancini et al., 2021a,b; Misra et al., 2017; Naeem et al., 2021; Nagarajan and Grauman, 2018; Purushwalkam et al., 2019; Xu et al., 2021). However, the pretraining of the word embeddings and image encoder is disjoint and isolated from each other, i.e., these methods learn to align image and text representations from scratch. These task-specific architectures also are limited in flexibility. For example, to adapt these methods to higher-order compositions with multiple attributes and objects such as `small furry cat` or `old white tiger`, the original architecture needs to be modified. The ability to generalize beyond the original training length is a key test for compositionality (Hupkes et al., 2020).

In contrast, we propose to build on large-scale pretrained vision-language models (VLMs), which are trained on massive amounts of aligned images and text (Jain et al., 2021; Jia et al., 2021; Li et al., 2021; Radford et al., 2021b). We focus on CLIP (Radford et al., 2021b), a powerful vision-language model pretrained on 400 million image-text pairs. CLIP has two main components: the image encoder and the text encoder that produce vector representations for images and text in a multi-modal embedding space. The text encoder accepts a textual input, or a prompt such as `A photo of dog` to produce a vector representation for the class `dog`. Taking the cosine similarity with all the class prompts and the image, we get a compatibility score for the classes and pick

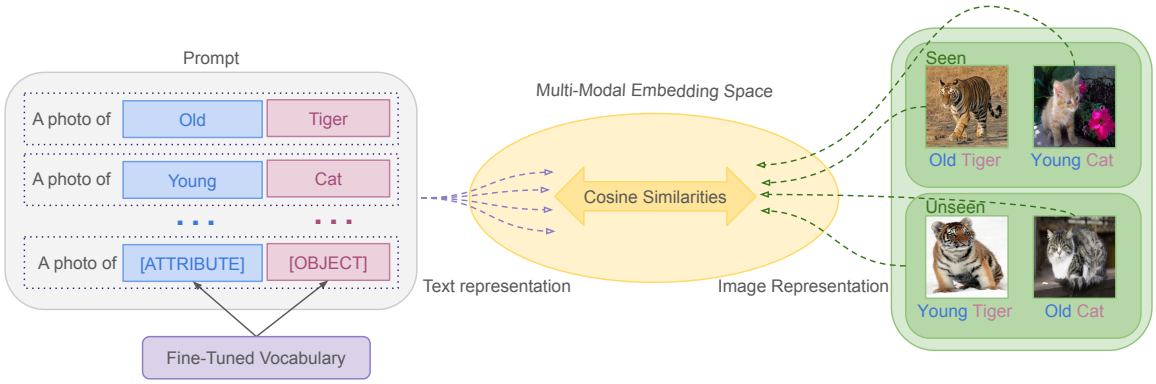

Figure 3.1: An overview of compositional zero-shot learning with ℂ𝕊ℙ. We fine-tune the vocabulary for attributes and objects on the seen classes. Then we compose novel soft prompts to test on the unseen classes.

the one with the highest score. However, CLIP without any fine-tuning underperforms task-specific architectures, even though it has been pre-trained on vastly more data. This finding suggests that there is significant room for improvement from teaching VLMs like CLIP about composing concepts.

To improve VLMs for compositional zero-shot learning, we introduce compositional soft prompting (ℂ𝕊ℙ), a parameter-efficient learning technique that tunes tokens of vocabulary to represent primitive concepts in a composable way. Fine-tuning large pre-trained models such as CLIP requires huge amounts of compute and may lead to overfitting (Sung et al., 2021; Mitchell et al., 2022) (see also Section 3.5). This challenge has motivated several soft prompting techniques in both language and vision (Lester et al., 2021; Qin and Eisner, 2021; Vu et al., 2021; Zhou et al., 2021). These works tune a single prompt on a downstream supervised task, often in a few-shot setting. For instance, they typically use prompts such as `A photo of [class]` and tune the prefix `A photo of` on the entire dataset. In contrast, ℂ𝕊ℙ is a novel way of soft prompting. We treat the attributes and objects that are composed to define classes as learnable tokens of vocabulary in a prompt as `A photo of [attribute] [object]`. We tune on multiple `[attribute]` and `[object]` prompt compositions, and then we recompose them into new prompts for zero-shot inference (Figure 3.1).

Our results show that ℂ𝕊ℙ improves over the zero-shot performance of CLIP. ℂ𝕊ℙ significantly improves over CLIP across three benchmark datasets by an average accuracy of 13.7 percentage points in the closed-world setting and 8.0 percentage points in the open-world setting (using the AUC metric). ℂ𝕊ℙ also outperforms CoOp, a soft prompting method that tunes the prefix context, by an average of 7.3 percentage points in the closed-world setting and 4.3 percentage points in the open-world setting on the AUC metric.

In addition to improved benchmark accuracy, ℂ𝕊ℙ has several other advantages when tested on other kinds of zero-shot inferences without any changes to training. We show that the learned attribute vocabulary can be decomposed to better classify attributes in isolation, using prompts of the form `A photo of [attribute] object`. We also show that training ℂ𝕊ℙ with attribute-object compositions improves CLIP's performance on attribute-attribute-object compositions. Finally, we show that ℂ𝕊ℙ improves generalization to compositions of *unseen* attributes and seen objects. Prior work on compositional zero-shot learning typically only evaluates unseen compositions of seen attributes and seen objects.

In summary, our main contributions are:

- We introduce compositional soft prompting (ℂ𝕊ℙ), a parameter-efficient learning technique to improve the compositionality of large-scale vision-language models (VLMs). The attributes and objects that are composed to define classes are treated as learnable tokens of vocabulary. Unlike existing work on soft prompting, our learned prompts are tuned on multiple compositions and then recomposed in new combinations for inference.
- ℂ𝕊ℙ improves the AUC accuracy of CLIP by an average of 10.9 percentage points across three benchmark datasets (Isola et al., 2015; Yu and Grauman, 2014;

Mancini et al., 2021a). It also improves over CoOp, a soft-prompting method that tunes the prefix context, by an average of 5.8 percentage points on AUC.

- We conduct additional experiments to analyze $\mathbb{CSP}$. We show that training on attribute-object compositions improves CLIP's accuracy on attribute classification alone, attribute-attribute-object compositions, and compositions of pretrained and fine-tuned vocabulary.

3.2 Related Work

We describe the related work in prompting, parameter-efficient learning, and compositional zero-shot learning.

Prompting Prompting is a recent focus in the language and vision communities that has shown benefits in zero-shot and few-shot learning on a wide range of tasks (Bach et al., 2022a; Bommasani et al., 2021a; Brown et al., 2020; Lester et al., 2021; Radford et al., 2021b; Qin and Eisner, 2021; Sanh et al., 2022; Vu et al., 2021; Zhou et al., 2021, 2022). Discrete prompts are typically hand-written text inputs that provide guidelines to large pre-trained models such as CLIP, GPT-3 (**?**), etc. for inference without updating the model parameters. While manually engineering prompts can help achieve better accuracy, it is often time-consuming and impractical to find the best prompt.

Soft prompting is an alternative to discrete prompts, where a part of the prompt is learned by backpropagating without fine-tuning the entire model. Several works using soft prompts show improved accuracy compared to hand-crafted prompts (Lester et al., 2021; Li and Liang, 2021; Qin and Eisner, 2021; Shin et al., 2020; Vu et al., 2021; Zhou et al., 2021). In all these works, soft prompts are a single input concatenated to all inputs for the entire task. In contrast, we learn tokens for each primitive concept from multiple compositions and recompose them in new ways to represent unseen classes for

zero-shot inference. We show in Section 3.5 that traditional soft prompting can also improve CLIP on compositional zero-shot learning, but generally not as much as $\mathbb{CSP}$.

Recent works have used soft prompting for large-scale vision-language models. Zhou et al. (2021) propose CoOp (contextual optimization), a soft prompting method for few-shot object classification. Other applications include visual question answering (Jin et al., 2022) and video understanding (Ju et al., 2021). Again, these works tune a single soft prompt on the entire dataset. One exception is Ge et al. (2022), which learns multiple soft prompts for cross-domain adaption. While our work shares similarities with their work, there are important differences. Our work decomposes the class labels into multiple parts rather than splitting the prompt into domain-related granularities such as domain-agnostic context, domain-specific context, and class label. Furthermore, we focus on compositional zero-shot learning where we do not have access to labeled examples from the unseen classes in the test set whereas they assume access to all the test classes during training.

Zhou et al. (2022) extend CoOp to CoCoOp (conditional contextual optimization) to reduce overfitting in prompt tuning for few-shot object classification. They learn a lightweight network that takes the image representation and produces a conditional vector that is added to the soft tokens in the prompt.

Parameter-Efficient Learning Soft prompting is closely related to the growing body of work in parameter-efficient learning (Houlsby et al., 2019; Guo et al., 2021; Mahabadi et al., 2021; Sung et al., 2021; Ben-Zaken et al., 2022; Liu et al., 2022). They add small feedforward networks between the layers in the pretrained model, or use sophisticated techniques to select a sparse set of model parameters and update them by fine-tuning on a labeled training set. However, unlike $\mathbb{CSP}$, the methods do not assign semantic meaning to the parameters in order to enable composition.

Compositional Zero-Shot Learning The growing interest in compositional zero-shot learning has contributed to several architectural innovations (Li et al., 2020b; Mancini et al., 2021a,b; Misra et al., 2017; Naeem et al., 2021; Nagarajan and Grauman, 2018; Purushwalkam et al., 2019; Radenović et al., 2021). Early works compose attributes and objects with a transformation function (Misra et al., 2017; Nagarajan and Grauman, 2018). Recent work uses separate encoding layers for attributes and objects, and then combines them with late fusion using a linear layer or a multilayer perceptron (Purushwalkam et al., 2019). The most successful methods represent the attribute and object relationship in a graph and learn their compositions via graph convolutional networks (Mancini et al., 2021b; Naeem et al., 2021; Ruis et al., 2021). Yun et al. (2022) study the emergence of compositionality in CLIP pre-training by learning linear classifiers rather than soft prompts for primitive concepts. Scialom et al. (2022) show that continually fine-tuning large language models can learn composable instructions.

Evaluating CLIP in a fair setting compared to the existing task-specific architectures for compositional zero-shot learning is challenging. On the one hand, CLIP is trained on a web-scale dataset to which other methods do not have access, and may even contain some of the classes from the unseen split. On the other hand, existing task-specific architectures fine-tune orders of magnitude more parameters than soft prompting, while using specialized architectures that do not adapt as easily as VLMs to new tasks. For these reasons, our work focuses on improving CLIP-based baselines, and we include a summary of the results comparing task-specific architectures in Section 3.5.

Compositional zero-shot learning is also closely related to the broader goal of compositionality in artificial intelligence (Chomsky, 1956; Fodor and Pylyshyn, 1988). Existing works have studied compositionality for image generation (Herzig et al., 2020), video synthesis (Ye et al., 2019; Bar et al., 2021; Nawhal et al., 2020), visual reasoning (Johnson et al., 2017), semantic parsing (Drozdov et al., 2022), language grounding (Jin

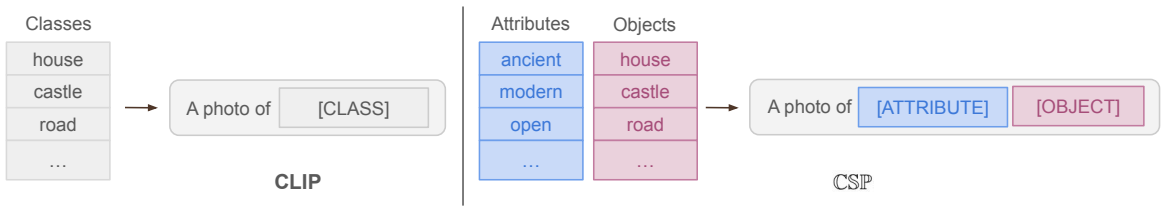

Figure 3.2: Comparison of CLIP in a zero-shot and $\mathbb{CSP}$ in a compositional zero-shot setting.

et al., 2022), and question answering (Yuan et al., 2019). For more in-depth discussion on compositionality, we refer the readers to Hupkes et al. (2020).

3.3 Preliminaries

In this section, we formally define the task of compositional zero-shot learning and describe how to use CLIP for compositional zero-shot inference. Let $\mathbb{A} = \{a_0, a_1, ..., a_n\}$ be the set of possible attributes and $\mathbb{O} = \{o_0, o_1, ..., o_m\}$ be the set of possible object categories. Let the label space $\mathbb{Y}$ be the Cartesian product of the attribute set and the object category set, $\mathbb{Y} = \mathbb{A} \times \mathbb{O}$. We are given two disjoint label subsets such that $\mathbb{Y}_{seen} \subset \mathbb{Y}$, $\mathbb{Y}_{unseen} \subset \mathbb{Y}$, and $\mathbb{Y}_{seen} \cap \mathbb{Y}_{unseen} = \varnothing$ where $\mathbb{Y}_{seen}$ and $\mathbb{Y}_{unseen}$ are the set of the seen and unseen classes. At training time, we are given examples $\mathbb{S}_{seen} = \{(x_1, y_1), ..., (x_n, y_n)\}$ to learn some discriminative model $f : \mathbb{X} \rightarrow \mathbb{Y}_{seen}$. During inference, we want the model to predict both seen and unseen classes in the test set, i.e., $f : \mathbb{X} \rightarrow \mathbb{Y}_{test}$. In the closed-world evaluation, the test set is defined as $\mathbb{Y}_{test} = \mathbb{Y}_{seen} \cup \mathbb{Y}_{unseen}$. In the open-world evaluation, the model has to consider all possible permutations of the attribute-object compositions, i.e., $\mathbb{Y}_{test} = \mathbb{Y}$ and $\mathbb{Y}_{unseen} = \mathbb{Y} - \mathbb{Y}_{seen}$.

We can easily adapt CLIP to compositional zero-shot inference by defining prompts appropriately. The prompts used in CLIP for zero-shot inference differ from the prompts used in works like GPT-3 (**?**) and T0 (Sanh et al., 2022). Instead of defining one prompt for an N-way classification task, the N classes are transformed into natural

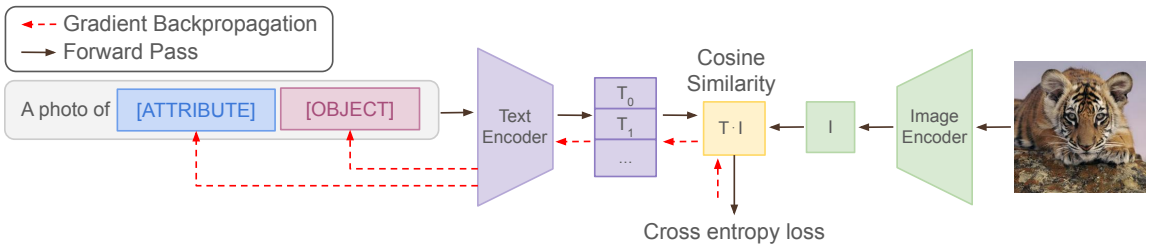

Figure 3.3: Training setup for ℂ𝕊ℙ. The prompt with the attribute and object vocabulary is passed through the text encoder to get the text representation. The example is passed through the image encoder for the image representation. Next, we take the cosine similarity for all the prompts with the image and compute the cross entropy loss. Finally, we backpropagate the loss through the text encoder and update the attribute and object vocabulary weights.

language prompts such as `A photo of [class]` (Figure 3.2, left). This results in N prompt vectors, which are used to compute the cosine similarity with the image representation for prediction. For compositional zero-shot inference, we replace the `[class]` placeholder with the attribute and object composition as follows: `a photo of [attribute] [object]`. The change in prompt format allows us to represent pairs of attribute and objects such as `a photo of young cat` in the text encoder without changes.

3.4 Compositional Soft Prompting

In this section, we introduce compositional soft prompting (ℂ𝕊ℙ), a parameter-efficient learning technique for fine-tuning large pretrained models for better compositionality.

Motivation The goal of our work is to improve VLMs such as CLIP on compositional generalization where they seem to underperform the current state-of-the-art methods This is perhaps because CLIP's pretraining on data crawled from the web does not provide sufficient supervision about attributes and how they combine with different objects. Therefore, we aim to teach VLMs such as CLIP how to better compose

primitive concepts. We approach this is a vocabulary-learning problem because it is parameter efficient and gives us a natural way to compose new classes.

Prompt Construction $\mathbb{CSP}$ treats the attributes and objects that are composed to define classes as learnable tokens of vocabulary and tunes them on multiple prompt compositions. We represent each primitive concept, either attribute or object, as a new, auxiliary token in the VLM's vocabulary. To represent a class, we combine a fixed prefix and the corresponding primitive concepts, for example `A photo of young tiger`, where `young` maps to the auxiliary weight vector for the corresponding attribute and `tiger` maps to the auxiliary weight vector for the corresponding object. Then, we use these prompts for compositional zero-shot learning in the same way as we would for CLIP, as described in Section 3.3. Training and inference with $\mathbb{CSP}$ is very simple as we only need to swap the vocabulary of the attributes and objects for any desired composition in the prompt (Figure 3.2, right). As a result, our method tunes only $(|\mathbb{A}| + |\mathbb{O}|) \times d$ parameters where d is the dimension of the vocabulary embedding. This is a novel form of soft prompting, because prior work (Lester et al., 2021; Qin and Eisner, 2021; Zhou et al., 2021) tune prompts for seen classes and usually one prompt for the whole task, whereas we compose learned prompt tokens to represent unseen classes at test time.

Training We want to learn vector embeddings for the new, auxiliary vocabulary: $\boldsymbol{\theta} = [\boldsymbol{\theta}_{\mathbb{A}}; \boldsymbol{\theta}_{\mathbb{O}}]$ where $\boldsymbol{\theta} \in \mathbb{R}^{(|\mathbb{A}|+|\mathbb{O}|)\times d}$. We learn them by composing prompts with them and fine-tuning them on the seen classes. Figure 3.3 shows the overall learning process for $\mathbb{CSP}$.

First, we initialize the learnable vocabulary with pretrained embeddings from CLIP using the concept names, i.e. attributes and objects. If a concept, such as `Faux Fur`, has multiple tokens, we average their pre-trained representations to initialize the learnable vocabulary. Next, for each class, we construct a prompt with the pretrained vocabulary

	Composition			*Train*		*Validation*			*Test*		
Dataset	$\lvert\mathbb{A}\rvert$	$\lvert\mathbb{O}\rvert$	$\lvert\mathbb{A}\times\mathbb{O}\rvert$	$\lvert\mathbb{Y}_{seen}\rvert$	$\lvert\mathbb{X}\rvert$	$\lvert\mathbb{Y}_{seen}\rvert$	$\lvert\mathbb{Y}_{unseen}\rvert$	$\lvert\mathbb{X}\rvert$	$\lvert\mathbb{Y}_{seen}\rvert$	$\lvert\mathbb{Y}_{unseen}\rvert$	$\lvert\mathbb{X}\rvert$
MIT-States Isola et al. (2015)	115	245	28175	1262	30338	300	300	10420	400	400	12995
UT-Zappos Yu and Grauman (2014)	16	12	192	83	22998	15	15	3214	18	18	2914
C-GQA Mancini et al. (2021a)	413	674	278362	5592	26920	1252	1040	7280	888	923	5098

Table 3.1: Summary statistics of the datasets used in our experiments.

for the prefix context and learnable vocabulary for the primitive concepts:

$$t_{a,o} = (\boldsymbol{w}_0, \ldots, \boldsymbol{w}_p, \boldsymbol{\theta}_a, \boldsymbol{\theta}_o)$$

where $\boldsymbol{w}_i \in \mathbb{R}^d$ are the tokens of the prefix context, and $\boldsymbol{\theta}_a$ and $\boldsymbol{\theta}_o$ are the learnable parameters for the attribute and the object in the prompt.

Next, we pass the prompt with learnable parameters through the text encoder to get the text representation:

$$\boldsymbol{t}_{a,o} = \frac{VLM_{\mathbf{T}}\ (t_{a,o})}{||\, VLM_{\mathbf{T}}\ (t_{a,o})||}$$

Then, for some image x, we get the image representation from the image encoder:

$$\boldsymbol{x} = \frac{VLM_{\mathbf{V}}\ (x)}{||\, VLM_{\mathbf{V}}\ (x)||}$$

Finally, we compute the class probability:

$$p_{\boldsymbol{\theta}}(y = (a,o) \mid x) = \frac{\exp\left(\boldsymbol{x} \cdot \boldsymbol{t}_{a,o}/\tau\right)}{\sum_{(\hat{a},\hat{o})\in\mathbb{Y}_{seen}} \exp\left(\boldsymbol{x} \cdot \boldsymbol{t}_{\hat{a},\hat{o}}/\tau\right)}$$

where $\tau \in \mathbb{R}$ is the temperature parameter from CLIP. We learn the parameters $\boldsymbol{\theta}$ by minimizing the cross entropy loss on the training dataset:

$$-\frac{1}{|\mathbb{S}_{seen}|} \sum_{(x,y)\in\mathbb{S}_{seen}} \log p_{\boldsymbol{\theta}}(y \mid x) + \lambda||\boldsymbol{\theta}||^2$$

where $\lambda \in \mathbb{R}$ is the weight decay.

```
def inference(
    batch_images: nn.Tensor,
    test_pairs: List[List, List],
    model: nn.Module
):
    """
    Function to run inference with the fine-tuned embeddings.
    Args:
        batch_images (torch.Tensor): minibatch of images [n, h, w, c]
        test_pairs (tuple): attribute-object pairs in the test
            split [m, 2]
        model (nn.Module): model with the fine-tuned embeddings
    Returns:
        torch.Tensor: cosine similarties of the minibatch images
            and attribute-object pairs [n, m]
    """
    prompt_template = "a photo of x x"
    tokenized_prompt = tokenize(prompt_template)
    tokenized_prompt = tokenized_prompt.repeat(len(test_pairs))
    token_tensor = model.token_embedding(tokenized_prompt)

    # fine-tuned embeddings
    attr, obj = zip(*test_pairs)
    attr_emb = model.soft_embedding(attr)
    obj_emb = model.soft_embedding(obj)

    # replace the "x x" in prompt template with fine-tuned embeddings
    token_tensor = replace_emb(token_tensor, attr_emb, obj_emb)

    # l2-normalized
    text_rep = model.text_encoder(token_tensor)
    image_rep = model.image_encoder(batch_images)

    logits = (image_rep @ text_rep) * model.logit_scale.exp()

    return logits
```

Figure 3.4: Torch-like pseudocode for inference with $\mathbb{CSP}$.

Inference During inference, we recompose the fine-tuned attribute and object vocabulary in the prompt. We compose the candidate prompts with the tuned $\boldsymbol{\theta}$ with the (attribute, object) pairs in the same way during training. In both closed-world and open-world settings, we only replace attribute and objects with the fine-tuned parameters in the prompt. Finally, we calculate the most likely attribute and object pair as follows:

$$\hat{y} = \underset{y \in \mathbb{Y}_{test}}{\operatorname{argmax}} \; p_{\boldsymbol{\theta}}(y \mid x)$$

3.4.1 Pseudocode

Figure 3.7 shows the Torch-like pseudocode for inference with $\mathbb{CSP}$. The function accepts the minibatch of images, test pairs, and the clip model with the fine-tuned embeddings and returns the cosine similarities between the image representation and

the text representation scaled by a constant scalar.

3.5 Experimental Evaluation

In this section, we describe our experiments with $\mathbb{CSP}$. We compare $\mathbb{CSP}$ to CLIP-based baselines in the closed-world and open-world settings of compositional zero-shot learning. We also compare the performance of fine-tuned CLIP with $\mathbb{CSP}$on the benchmark datasets. Finally, we demonstrate that $\mathbb{CSP}$ can generalize beyond these benchmarks to three modified settings: attribute-only classification, attribute-attribute-object composition, and inference with unseen attributes.

Dataset We experiment with three attribute-object composition benchmarks: MIT-states (Isola et al., 2015), UT-Zappos (Yu and Grauman, 2014), and C-GQA (Naeem et al., 2021). Table 3.1 summarizes the statistics of the datasets. MIT-states contains images of naturally occurring objects where each object is described by an adjective. UT-Zappos contains images of shoes paired with fine-grained states. For this dataset, we use the split suggested by Purushwalkam et al. (2019). C-GQA, a newly introduced dataset derived from the Stanford GQA dataset (Hudson and Manning, 2019), contains images of objects paired with states.

Benchmark Evaluation We follow the standard closed-world and open-world evaluation protocols. In the closed-world setting, the unseen classes are a subset of all possible attribute-object combinations and are defined in the dataset. In the open-world setting, the model considers all possible attribute-object combinations.

Following prior work (Mancini et al., 2021a), we report the performance in the generalized zero-shot learning for both the closed-world and the open-world settings. In generalized zero-shot learning, we include both the seen and unseen classes in the test set. Several works have noted that zero-shot models are biased towards the seen

classes, so we follow the standard of adding a scalar bias to the unseen classes (Chao et al., 2016; Ruis et al., 2021) . Following prior work, we vary the bias from $-\infty$ to $+\infty$ to get a curve indicating the seen accuracy on the x-axis and unseen accuracy on the y-axis. We report the area under the curve (AUC) and select the operating point with the best harmonic mean (H) between the seen and unseen accuracy. We also report the best seen accuracy (S) when bias is $-\infty$ and the best unseen accuracy (U) when the bias is $+\infty$. We average over five random seeds, selecting the best-performing model after each epoch on the validation data.

		MIT-States			
	Method	S	U	H	AUC
Closed	CLIP	30.2	46.0	26.1	11.0
	CoOp	34.4 $_{0.1}$	47.6 $_{0.1}$	29.8 $_{0.1}$	13.5 $_{0.0}$
	$\mathbb{CSP}$	**46.6** $_{0.1}$	**49.9** $_{0.1}$	**36.3** $_{0.1}$	**19.4** $_{0.1}$
Open	CLIP	30.1	14.3	12.8	3.0
	CoOp	34.6 $_{0.1}$	9.3 $_{0.0}$	12.3 $_{0.1}$	2.8 $_{0.0}$
	$\mathbb{CSP}$	**46.3** $_{0.3}$	**15.7** $_{0.1}$	**17.4** $_{0.1}$	**5.7** $_{0.0}$

Table 3.2: Closed-world (Closed) and open-world (Open) results on MIT-States. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error.

		UT-Zappos			
	Method	S	U	H	AUC
Closed	CLIP	15.8	49.1	15.6	5.0
	CoOp	52.1 $_{0.5}$	49.3 $_{1.8}$	34.6 $_{1.7}$	18.8 $_{1.4}$
	$\mathbb{CSP}$	**64.2** $_{0.7}$	**66.2** $_{1.2}$	**46.6** $_{1.2}$	**33.0** $_{1.3}$
Open	CLIP	15.7	20.6	11.2	2.2
	CoOp	52.1 $_{0.5}$	31.5 $_{2.9}$	28.9 $_{2.3}$	13.2 $_{1.6}$
	$\mathbb{CSP}$	**64.1** $_{0.7}$	**44.1** $_{0.3}$	**38.9** $_{0.5}$	**22.7** $_{0.4}$

Table 3.3: Closed-world (Closed) and open-world (Open) results on UT-Zappos. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error.

		C-GQA			
	Method	S	U	H	AUC
Closed	CLIP	7.5	25.0	8.6	1.4
	CoOp	20.5 $_{0.2}$	**26.8** $_{0.3}$	17.1 $_{0.2}$	4.4 $_{0.1}$
	$\mathbb{CSP}$	**28.8** $_{0.1}$	**26.8** $_{0.1}$	**20.5** $_{0.1}$	**6.2** $_{0.0}$
Open	CLIP	7.5	4.6	4.0	0.27
	CoOp	21.0 $_{0.2}$	4.6 $_{0.1}$	5.5 $_{0.1}$	0.70 $_{0.0}$
	$\mathbb{CSP}$	**28.7** $_{0.2}$	**5.2** $_{0.1}$	**6.9** $_{0.1}$	**1.20** $_{0.0}$

Table 3.4: Closed-world (Closed) and open-world (Open) results on C-GQA. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error.

Following prior work, we apply feasibility calibration to all methods for prediction in the open-world setting. The open-world setting is particularly challenging as the label space contains all possible combinations of attributes and objects in the dataset. For instance, the label space contains feasible compositions such as `young cat` and `eroded cliff` and infeasible compositions such as `eroded cat`. Existing work shows a significant drop in model performance from the closed-world setting to the open-world setting (Mancini et al., 2021a,b). As suggested in Mancini et al. (2021a), we apply a simple feasibility calibration based on GloVe embeddings (Pennington et al., 2014) to filter out infeasible compositions.

Baselines In our experiments, we primarily compare with CLIP-based baselines. We compare $\mathbb{CSP}$ to pretrained CLIP (Radford et al., 2021b) and CoOp (Zhou et al., 2021). CoOp is a soft-prompting method that learns the prefix context with limited labeled examples in a few-shot setting. CoOp resembles prompt tuning (Lester et al., 2021) applied to VLMs.

Training Details We implement $\mathbb{CSP}$ and the baselines with a pretrained CLIP model in PyTorch (Paszke et al., 2019). We use the CLIP model ViT-L/14 which is the largest available model in our experiments. Nonetheless, our method is agnostic to the choice of CLIP architecture. The pretrained CLIP ViT-L/14 model has a vision

Method	*MIT-States*	*UT-Zappos*	*C-GQA*
CLIP	11.0	5.0	1.4
CLIP (FT)	**22.2** $_{0.1}$	24.5 $_{1.6}$	**10.5** $_{0.2}$
$\mathbb{CSP}$	19.4 $_{0.1}$	**33.0** $_{1.3}$	6.2 $_{0.0}$

Table 3.5: Closed-world results comparing $\mathbb{CSP}$ and fine-tuned CLIP (FT). For $\mathbb{CSP}$ and CLIP (FT), we report the average AUC on 5 random seeds with standard error.

transformer (ViT) as the image encoder and a transformer as the text encoder.

We train $\mathbb{CSP}$ and CoOp by minimizing the cross entropy loss with the Adam optimizer over the seen split in the dataset for 20 epochs. We use a single NVIDIA RTX 3090 or V100 GPU depending on their availability to train all our models. For each dataset, we choose the best hyperparameters based on the performance on the validation split.

Benchmark Results Our results in Table **??** show that $\mathbb{CSP}$ significantly improves over CLIP on all the benchmark datasets in the closed-world and open-world settings. In the closed-world setting, we outperform CLIP in the AUC metric by 8.4 points on MIT-states, 28.0 point on UT-Zappos, and 4.8 points on C-GQA. Additionally, we show that $\mathbb{CSP}$ beats CoOp, a soft-prompting method, in the AUC metric by 5.9 points on MIT-States, 14.2 points on UT-Zappos, and 1.8 points on C-GQA. In results in the open-world setting show that $\mathbb{CSP}$ improves over CLIP by 2.7 points on MIT-States, 20.5 points on UT-Zappos, and 0.93 points on C-GQA in the AUC metric. We outperform CoOp on MIT-States by 3.1 points, UT-Zappos by 9.5 points, and C-GQA by 0.50 points in the AUC metric.

Comparison with Full Fine-Tuning We aim to understand the potential benefits of fine-tuning all the parameters of CLIP instead of a small number. To that end, we fine-tune all the parameters in CLIP on the seen classes in our benchmark datasets.

Table 3.5 shows that fine-tuning CLIP can improve generalization to unseen classes

Method	*MIT-States*	*UT-Zappos*	*C-GQA*
ProtoProp	-	**34.7**	-
CGE	6.5	33.5	4.2
Co-CGE	6.6	33.9	4.1
CLIP	11.0	5.0	1.4
$\mathbb{CSP}$	**19.4** $_{0.1}$	33.0 $_{1.3}$	**6.2** $_{0.0}$

Table 3.6: Closed-world results comparing $\mathbb{CSP}$ with task-specific architectures. For $\mathbb{CSP}$, we report the average AUC on 5 random seeds with standard error.

but still achieves lower AUC compared to $\mathbb{CSP}$ on UT-Zappos. In addition, fine-tuning CLIP requires GPUs with more memory and often meticulous hyperparameter selection (Dong et al., 2022).

Comparison with Task-Specific Architectures We compare $\mathbb{CSP}$ to existing task-specific compositional zero-shot learning architectures. We consider the following task-specific architectures: CGE (Naeem et al., 2021), Co-CGE (Mancini et al., 2021b) and ProtoProp (Ruis et al., 2021). These methods are the most recent best-performing methods in the closed-world setting.

Our results in Table 3.6 show that $\mathbb{CSP}$ outperform existing task-specific architectures on MIT-States and C-GQA while being competitive on UT-Zappos in AUC. We see that $\mathbb{CSP}$ improves over existing task-specific architectures by 12.8 points on MIT-states and 2.0 points on C-GQA in AUC.

Method	Accuracy
CLIP	62.7
CoOp	65.2 $_{0.3}$
$\mathbb{CSP}$	**72.6** $_{0.4}$

Table 3.7: Unseen accuracy on 5 random seeds with std. error.

Generalization to Higher-Order Compositions To test the additional flexibility afforded by VLMs, we test if training $\mathbb{CSP}$ with attribute-object compositions can generalize to higher-order compositions such as attribute-attribute-object compositions.

We annotate a novel challenge dataset: AAO-MIT-States, a subset derived from the MIT-States dataset. In this dataset, we annotate the images in the test split of the MIT-States dataset with an additional attribute, to get an attribute-attribute-object pair as the class label.

We compare CLIP, CoOp, and $\mathbb{CSP}$ to classify images with attribute-attribute-object classes. Since these class compositions are not present during training, we treat them as unseen classes and calculate the unseen accuracy. We take the best performing models for MIT-States and run inference on the challenge dataset.

Table 3.7 shows that $\mathbb{CSP}$ improves over CLIP by an average 9.9 percentage points on unseen accuracy and generalizes to attribute-attribute-object compositions without any modifications or training. The results demonstrate that $\mathbb{CSP}$ improves the compositionality of CLIP's vocabulary, even in ways that were not explicitly supervised.

Generalization to Mixed Vocabulary To further test the additional flexibility afforded by VLMs, we also evaluate $\mathbb{CSP}$ on compositional zero-shot learning with a mixture of pretrained and fine-tuned vocabulary. This evaluation stems from the practical need to combine new unseen attributes with fine-tuned vocabulary. Evaluating in this setting will allow us to assess whether the benefits of $\mathbb{CSP}$ extend to classes including vocabulary not seen during fine-tuning. This setup goes beyond the above benchmarks, which include unseen *combinations* of attributes and objects, but all attributes and objects are seen during training. Now, we include completely unseen attributes.

We apply $\mathbb{CSP}$ on UT-Zappos with different fractions of attributes as seen attributes. We randomly select 25%, 50%, 75%, and 100% of the attributes and all the objects from the training set. Then, we remove from the seen classes the attribute-object pairs that include an unseen attribute. Finally, we train the on the remaining seen attribute-object pairs with five random seed values.

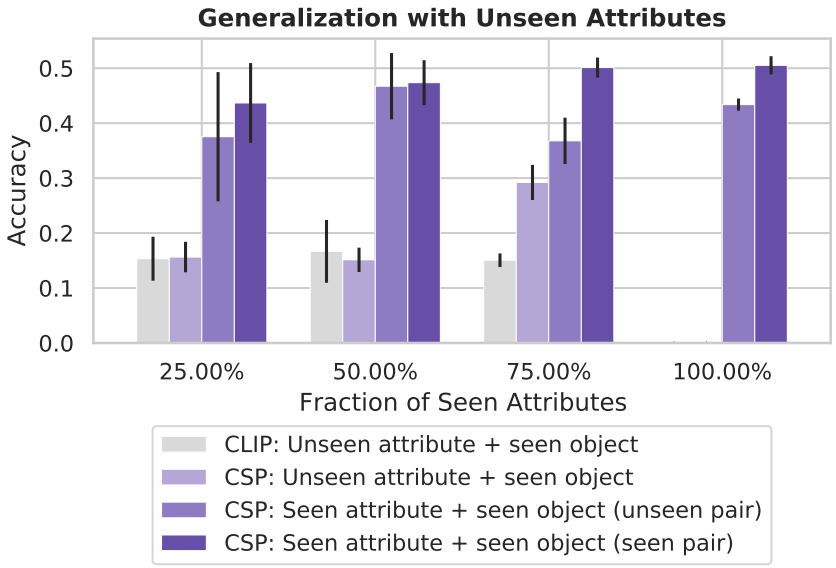

Figure 3.5: Results of $\mathbb{CSP}$ and CLIP with different fractions of pretrained and fine-tuned vocabulary. In each fraction, we report the average performance of CLIP and $\mathbb{CSP}$ on 5 random attribute splits.

For each split of the seen and unseen attributes, we evaluate CSP by dividing the classes into three buckets: (1) unseen attribute + seen object pairs, (2) seen (i.e., fine-tuned) attribute + seen object pairs in unseen combinations, and (3) seen attribute + seen object pairs in seen combinations. In this evaluation, we refer to the classes in the first and second buckets as the unseen classes and those in the third bucket as the seen classes. This evaluation is more general than typical compositional zero-shot learning, which only evaluates on classes in the second and third buckets. Similar to our evaluation in Section 3.5, we add a scalar bias to the unseen classes and select the bias that maximizes the harmonic mean between accuracy on the unseen and seen classes in the validation set of UT-Zappos. We then report accuracy on unseen examples in each of the three buckets. To contextualize the performance of $\mathbb{CSP}$, we report the accuracy of CLIP on the unseen attribute + seen object pairs.

Figure 3.5 shows that the performance on unseen attribute + seen object pairs improves with $\mathbb{CSP}$ and sufficient training pairs. Initially, the performance of CLIP and $\mathbb{CSP}$ are comparable but by providing more combinations of supervision for the

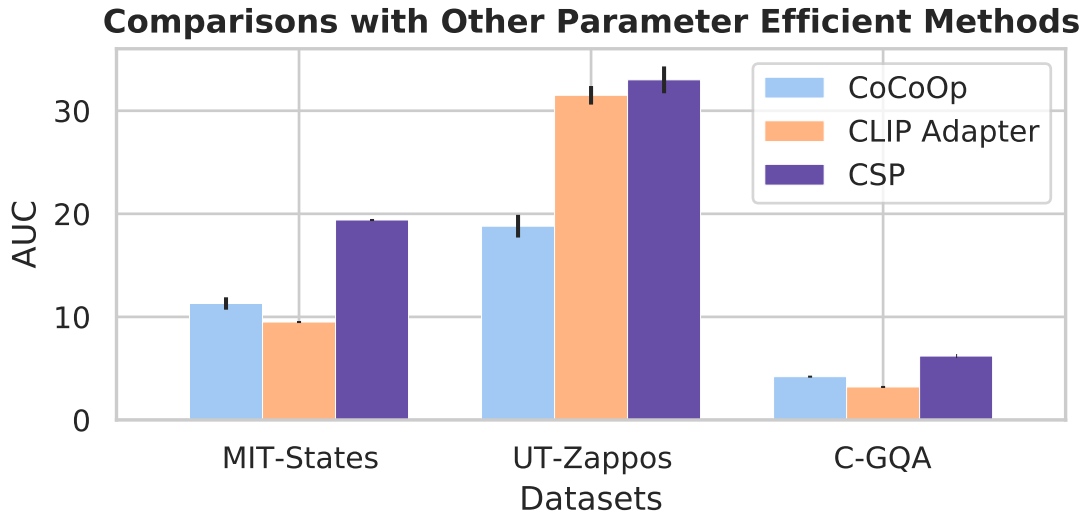

Figure 3.6: Closed-world results on MIT-States, UT-Zappos, and C-GQA comparing CoCoOp (Appendix 3.8), CLIP adapters (Appendix 3.9), and $\mathbb{CSP}$. We report the average AUC on 5 random seeds.

objects $\mathbb{CSP}$ significantly outperforms CLIP on the unseen attribute + seen object evaluation bucket. These results demonstrate that the fine-tuned vocabulary of $\mathbb{CSP}$ can improve the compositional zero-shot performance of pretrained vocabulary.

Additional Experiments We summarize the additional experiments included in the Appendices 3.8, 3.9, 3.10, and 3.11.

We compare $\mathbb{CSP}$ with other parameter efficient methods (see Figure 3.6). In Appendix 3.8, we experiment with CoCoOp, the conditional variant CoOp that incorporates visual information, on compositional zero-shot learning (Zhou et al., 2022). Our results show that $\mathbb{CSP}$ outperforms CoCoOp on all three datasets. In Appendix 3.9, we compare CLIP adapter (Gao et al., 2021) and $\mathbb{CSP}$ on compositional zero-shot learning. We see that $\mathbb{CSP}$ still achieves the highest AUC across the datasets compared to CLIP adapters.

In Appendix 3.10, to further test their flexibility, we see whether the learned vocabulary generalizes to classifying either attributes or objects alone. Our results show that $\mathbb{CSP}$ consistently improves attribute classification performance but can often

reduce object classification accuracy.

Finally, in Appendix 3.11, we compare different ResNet and ViT backbones of CLIP. Our results show that $\mathbb{CSP}$ performance improves with larger backbones. In particular, we observe the highest gains with ViT backbones.

3.6 Conclusion

In this chapter, I addressed the limitation of existing models' inability to handle compositional knowledge—specifically, their struggle to recognize novel combinations of known concepts without requiring separate annotations for each. I introduced $\mathbb{CSP}$, a compositional soft prompting technique designed to overcome the limited capability of existing models to handle compositional knowledge. By treating attributes and objects as learnable vocabulary tokens within vision-language models like CLIP, our method enables the recognition of novel attribute-object combinations without the need for separate annotations for each possible pair. This approach directly addresses the scalability issue, where the exponential growth of potential combinations makes comprehensive labeling impractical. Our experiments demonstrate that $\mathbb{CSP}$ significantly enhances compositional zero-shot performance while maintaining computational efficiency with only a small number of additional parameters. The learned vocabulary not only generalizes to unseen combinations but also adapts to more complex, higher-order compositions. By empowering models to compose new concepts from existing ones, $\mathbb{CSP}$ reduces the reliance on extensive labeled datasets and facilitates more efficient knowledge transfer from domain experts.

3.7 Comparison of Existing Task-Specific Architectures

In this section, we compare $\mathbb{CSP}$ to existing task-specific compositional zero-shot learning methods.

Baselines We consider the following compositional zero-shot learning methods in closed-world and open-world setting: AoP (Nagarajan and Grauman, 2018), LE+ (Misra et al., 2017), TMN (Purushwalkam et al., 2019), SymNet (Li et al., 2020b), CompCos (Mancini et al., 2021a), CGE (Naeem et al., 2021), and Co-CGE (Mancini et al., 2021b). We also consider ProtoProp (Ruis et al., 2021) in the closed-world setting.

	Method	S	U	H	AUC
Closed	AoP(Nagarajan and Grauman, 2018)	14.3	17.4	9.9	1.6
	LE+ (Misra et al., 2017)	15.0	20.1	10.7	2.0
	TMN(Purushwalkam et al., 2019)	20.2	20.1	13.0	2.9
	SymNet(Li et al., 2020b)	24.2	25.2	16.1	3.0
	CompCos (Mancini et al., 2021a)	25.3	24.6	16.4	4.5
	ProtoProp (Ruis et al., 2021)	-	-	-	-
	CGE (Naeem et al., 2021)	32.8	28.0	21.4	6.5
	Co-CGE (Mancini et al., 2021b)	32.1	28.3	20.0	6.6
	CLIP (Radford et al., 2021b)	30.2	46.0	26.1	11.0
	CoOp (Zhou et al., 2021)	34.4 $_{0.1}$	47.6 $_{0.1}$	29.8 $_{0.1}$	13.5 $_{0.0}$
	$\mathbb{CSP}$ (Ours)	**46.6** $_{0.1}$	**49.9** $_{0.1}$	**36.3** $_{0.1}$	**19.4** $_{0.1}$
Open	AoP(Nagarajan and Grauman, 2018)	16.6	5.7	4.7	0.7
	LE+ (Misra et al., 2017)	14.2	2.5	2.7	0.3
	TMN(Purushwalkam et al., 2019)	12.6	0.9	1.2	0.1
	SymNet(Li et al., 2020b)	21.4	7.0	5.8	0.8
	CompCos (Mancini et al., 2021a)	21.4	7.0	5.8	0.8
	CGE (Naeem et al., 2021)	32.4	5.1	6.0	1.0
	Co-CGE^{CW} (Mancini et al., 2021b)	31.1	5.8	6.4	1.1
	Co-CGE^{open} (Mancini et al., 2021b)	30.3	11.2	10.7	2.3
	CLIP (Radford et al., 2021b)	30.1	14.3	12.8	3.0
	CoOp (Zhou et al., 2021)	34.6 $_{0.1}$	9.3 $_{0.0}$	12.3 $_{0.1}$	2.8 $_{0.0}$
	$\mathbb{CSP}$ (Ours)	**46.3** $_{0.3}$	**15.7** $_{0.1}$	**17.4** $_{0.1}$	**5.7** $_{0.0}$

Table 3.8: Closed-world (Closed) and Open-world (Open) results on *MIT-States*. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error. The results for AoP, LE+, TMN, SymNet, CompCos, CGE, and Co-CGE are obtained from Mancini et al. (2021b).

	Method	S	U	H	AUC
Closed	AoP(Nagarajan and Grauman, 2018)	59.8	54.2	40.8	25.9
	LE+ (Misra et al., 2017)	53.0	61.9	41.0	25.7
	TMN(Purushwalkam et al., 2019)	58.7	60.0	45.0	29.3
	SymNet(Li et al., 2020b)	49.8	57.4	40.4	23.4
	CompCos (Mancini et al., 2021a)	59.8	62.5	43.1	28.1
	ProtoProp (Ruis et al., 2021)	62.1	65.5	50.2	**34.7**
	CGE (Naeem et al., 2021)	**64.5**	**71.5**	**60.5**	33.5
	Co-CGE (Mancini et al., 2021b)	62.3	66.3	48.1	33.9
	CLIP (Radford et al., 2021b)	15.8	49.1	15.6	5.0
	CoOp (Zhou et al., 2021)	52.1 $_{0.5}$	49.3 $_{1.8}$	34.6 $_{1.7}$	18.8 $_{1.4}$
	$\mathbb{CSP}$ (Ours)	64.2 $_{0.7}$	66.2 $_{1.2}$	46.6 $_{1.2}$	33.0$_{1.3}$
Open	AoP(Nagarajan and Grauman, 2018)	50.9	34.2	29.4	13.7
	LE+ (Misra et al., 2017)	60.4	36.5	30.5	16.3
	TMN(Purushwalkam et al., 2019)	55.9	18.1	21.7	8.4
	SymNet(Li et al., 2020b)	53.3	44.6	34.5	18.5
	CompCos (Mancini et al., 2021a)	53.3	44.6	34.5	18.5
	CGE (Naeem et al., 2021)	61.7	**47.7**	39.0	23.1
	Co-CGE^{CW} (Mancini et al., 2021b)	62.0	44.3	40.3	23.1
	Co-CGE^{open} (Mancini et al., 2021b)	61.2	45.8	**40.8**	**23.3**
	CLIP (Radford et al., 2021b)	15.7	20.6	11.2	2.2
	CoOp (Zhou et al., 2021)	52.1 $_{0.5}$	31.5 $_{2.9}$	28.9 $_{2.3}$	13.2 $_{1.6}$
	$\mathbb{CSP}$ (Ours)	**64.1** $_{0.7}$	44.1 $_{0.3}$	38.9 $_{0.5}$	22.7 $_{0.4}$

Table 3.9: Closed-world (Closed) and Open-world (Open) results on *UT-Zappos*. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error. The results for AoP, LE+, TMN, SymNet, CompCos, CGE, and Co-CGE are obtained from Mancini et al. (2021b) and ProtoProp from Ruis et al. (2021).

	Method	S	U	H	AUC
Closed	AoP(Nagarajan and Grauman, 2018)	17.0	5.6	5.9	0.7
	LE+ (Misra et al., 2017)	18.1	5.6	6.1	0.8
	TMN(Purushwalkam et al., 2019)	23.1	6.5	7.5	1.1
	SymNet(Li et al., 2020b)	26.8	10.3	11.0	2.1
	CompCos (Mancini et al., 2021a)	28.1	11.2	12.4	2.6
	ProtoProp (Ruis et al., 2021)	-	-	-	-
	CGE (Naeem et al., 2021)	**33.5**	15.5	16.0	4.2
	Co-CGE (Mancini et al., 2021b)	33.3	14.9	15.5	4.1
	CLIP (Radford et al., 2021b)	7.5	25.0	8.6	1.4
	CoOp (Zhou et al., 2021)	20.5 $_{0.2}$	**26.8** $_{0.3}$	17.1 $_{0.2}$	4.4 $_{0.1}$
	$\mathbb{CSP}$ (Ours)	28.8 $_{0.1}$	**26.8** $_{0.1}$	**20.5** $_{0.1}$	**6.2** $_{0.0}$
Open	AoP(Nagarajan and Grauman, 2018)	-	-	-	-
	LE+ (Misra et al., 2017)	19.2	0.7	1.0	0.08
	TMN(Purushwalkam et al., 2019)	-	-	-	-
	SymNet(Li et al., 2020b)	26.7	2.2	3.3	0.43
	CompCos (Mancini et al., 2021a)	26.7	2.2	3.3	0.43
	CGE (Naeem et al., 2021)	**32.7**	1.8	2.9	0.47
	Co-CGE^{CW} (Mancini et al., 2021b)	32.1	2.0	3.4	0.53
	Co-CGE^{open} (Mancini et al., 2021b)	32.1	3.0	4.8	0.78
	CLIP (Radford et al., 2021b)	7.5	4.6	4.0	0.27
	CoOp (Zhou et al., 2021)	21.0 $_{0.2}$	4.6 $_{0.1}$	5.5 $_{0.1}$	0.70 $_{0.0}$
	$\mathbb{CSP}$ (Ours)	28.7 $_{0.2}$	**5.2** $_{0.1}$	**6.9** $_{0.1}$	**1.20** $_{0.0}$

Table 3.10: Closed-world (Closed) and Open-world (Open) results on *C-GQA*. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error. The results for AoP, LE+, TMN, SymNet, CompCos, CGE, and Co-CGE are obtained from Mancini et al. (2021b).

Results Our results in Table 3.8, Table 3.9 and Table 3.10 show that $\mathbb{CSP}$ outperform existing compositional zero-shot learning method on MIT-States and C-GQA while being competitive on UT-Zappos in AUC. In the closed-world setting, $\mathbb{CSP}$ improves over existing compositional zero-shot learning methods by 12.8 points on MIT-states and 2.0 points on C-GQA in AUC. In the open-world setting, we improve over the existing compositional zero-shot learning methods on MIT-States by 3.4 points and C-GQA by 0.42 points in AUC.

3.8 Conditional Variant of CoOp and CSP

We experiment with CoCoOp, the conditional variant of CoOp, on compositional zero-shot learning. CoCoOp showed improved performance over CoOp in few-shot object classification by using visual information to condition the prompts. We investigate if additional visual information in prompts can help in compositional zero-shot learning.

CoCoOp uses a lightweight network to generate an image-specific bias vector and adds them to the learnable vocabulary in the prompt. For fair comparison, we also extend $\mathbb{CSP}$ to CoCSP to incorporate visual information into the prompts.

Setup We train CoCoOp and CoCSP with the same hyperparmeters in Appendix 3.14. Additionally, the lightweight network in our experiments is a two-layer multilayer perceptron with a ReLU activation between the two layers. The input and the output dimensions of the network are d and the hidden dimension is $d/16$ where d is the dimension of the vocabulary. The generated image-specific bias vector is added to the context vocabulary in CoCoOp and attribute-object vocabulary in CoCSP.

Method	*MIT-States*	*UT-Zappos*	*C-GQA*
CLIP	11.0	5.0	1.4
CoOp	13.5 $_{0.0}$	18.8 $_{1.4}$	4.4 $_{0.1}$
$\mathbb{CSP}$	**19.4** $_{0.1}$	**33.0** $_{1.3}$	**6.2** $_{0.0}$
CoCoOp	11.3 $_{0.6}$	18.8 $_{1.1}$	4.2 $_{0.1}$
CoCSP	17.6 $_{0.1}$	32.7 $_{0.3}$	5.7 $_{0.0}$

Table 3.11: Closed-world results on MIT-States, UT-Zappos, and C-GQA. For $\mathbb{CSP}$, CoOp, CoCoOp, and CoCSP, we report the average AUC on 5 random seeds with standard error. * denotes a bug in the soft prompt for CoCoOp which we will fix in the final version. In CoCoOp for C-GQA, we average the pre-trained vocabulary for attributes and objects with multiple tokens instead of using them as is in the prompt.

Results Table 3.11 shows CoCSP improves upon CoCoOp across the three datasets. We also observe no benefits over non-conditional methods such as $\mathbb{CSP}$ and CoOp in AUC. We suspect that additional parameters for compositional generalization from

seen pairs of concepts to unseen pairs.

3.9 CLIP Adapters with Compositional Soft Prompts

We extend CLIP adapter (Gao et al., 2021) to compositional zero-shot learning. Adapters are an alternate method of parameter-efficient learning for large-scale models that has shown improved performance on several downstream tasks (Houlsby et al., 2019).

CLIP adapters are a variant of the adapters for few-shot object classification (Houlsby et al., 2019). Instead of adding small feedforward networks to all the layers, they learn a multilayer perceptron in the image encoder and the text encoder. They transform the image representation from the encoder with the multilayer perceptron and include a residual connection from the image encoder and the same for the text representation. While the CLIP adapters can be added to the text encoder, the authors show that visual adapters perform better than textual adapters. For this reason, we compare with visual adapters in the experiment.

Setup We add a CLIP adapter, a two-layer multilayer perceptron with ReLU activation, to the image encoder. We use the same prompt prompt as $\mathbb{CSP}$ and train only the CLIP adapter. For fair comparison, we also train a model with $\mathbb{CSP}$ and CLIP adapters where we jointly train the prompts and adapter on the task. We train with the same hyperparamters settings for CLIP adapters and CLIP adapters + $\mathbb{CSP}$ in Appendix 3.14.

Method	*MIT-States*	*UT-Zappos*	*C-GQA*
CLIP	11.0	5.0	1.4
CLIP adapter	9.5 $_{0.1}$	31.5 $_{0.9}$	3.2 $_{0.1}$
CLIP adapter + $\mathbb{CSP}$	8.3 $_{0.2}$	32.5 $_{1.0}$	2.7 $_{0.0}$
$\mathbb{CSP}$	**19.4** $_{0.1}$	**33.0** $_{1.3}$	**6.2** $_{0.0}$

Table 3.12: Closed-world results on MIT-States, UT-Zappos, and C-GQA. For CLIP adapter, CLIP adapter + $\mathbb{CSP}$, and $\mathbb{CSP}$, we report the average AUC on 5 random seeds with standard error.

Results Table 3.12 shows that CLIP adapters can improve performance over CLIP but $\mathbb{CSP}$ performs better on its own. We also observe that combining CLIP adapters with $\mathbb{CSP}$ can often hurt performance in AUC.

3.10 Decomposition of Attributes and Objects

To further test the flexibility of $\mathbb{CSP}$, we investigate how the learned vocabulary performs on attribute classification and object classification separately. We create prompts of the form `a photo of [attribute] object` and `a photo of [object]` classify images according to either attribute or object. We measure accuracy on two sets of evaluation data. The first is the seen classes of each dataset, meaning that the classification is performed on new examples of attributes and objects that previously have been seen in the same combination. The second is the unseen classes of the original datasets, so this can be thought of as a kind of domain shift problem, evaluating how well the learned primitive concepts can be reused in isolation on novel attribute-object combinations.

Table 3.13 shows that $\mathbb{CSP}$ consistently improves attribute classification performance, while often reducing object classification accuracy. Sometimes CoOp improves over CLIP's object classification accuracy as well. This results indicates that $\mathbb{CSP}$ learns useful standalone attribute representations that are also composable with objects, but that the resulting object representations might not be as good on their own.

		MIT-States		*UT-Zappos*		*C-GQA*	
	Method	A	O	A	O	A	O
S	CLIP	16.6	44.3	8.8	58.5	3.6	25.4
	CoOp	18.7 $_{0.2}$	**47.6** $_{0.1}$	25.8 $_{3.4}$	57.8 $_{3.1}$	7.0 $_{0.6}$	**26.3** $_{0.4}$
	$\mathbb{CSP}$	**24.5** $_{0.2}$	40.2 $_{0.2}$	**66.4** $_{0.5}$	**63.8** $_{1.2}$	**12.5** $_{0.1}$	24.2 $_{0.1}$
U	CLIP	19.4	46.8	11.1	**65.6**	4.1	25.4
	CoOp	22.0 $_{0.2}$	**49.6** $_{0.1}$	21.5 $_{6.5}$	37.1 $_{2.9}$	5.9 $_{0.5}$	**25.9** $_{0.4}$
	$\mathbb{CSP}$	**23.3** $_{0.1}$	36.3 $_{0.1}$	**49.2** $_{1.8}$	46.6 $_{3.6}$	**6.0** $_{0.1}$	23.8 $_{0.1}$

Table 3.13: Results for decomposition of learned attribute and object vocabulary. We report average top-1 accuracy for attribute (A) and object (O) classification on 5 random seeds with standard error. Evaluation is done on seen (S) classes and unseen (U) classes of each dataset.

3.11 Model Ablation

Table 3.14 shows that $\mathbb{CSP}$ with a performance improves with larger backbones. We note that $\mathbb{CSP}$ generally improves performance over CLIP. In particular, we see the gains are highest with ViT backbones.

		MIT-States				*UT-Zappos*				*C-GQA*			
Method	Backbone	S	U	H	AUC	S	U	H	AUC	S	U	H	AUC
CLIP	ResNet-50	21.1	34.4	18.4	5.6	6.4	43.6	6.4	1.4	6.1	17.1	6.1	0.7
CLIP	ResNet-101	25.2	37.4	21.7	7.5	11.2	35.2	11.9	2.8	7.3	19.7	7.6	1.1
CLIP	ViT B/32	25.1	39.1	21.4	7.5	9.6	42.4	10.0	2.4	7.3	22.1	7.4	1.2
CLIP	ViT L/14	30.2	46.0	26.1	11.0	15.8	49.1	15.6	5.0	7.5	25.0	8.6	1.4
$\mathbb{CSP}$	ResNet-50	35.0 $_{0.1}$	30.3 $_{0.1}$	23.0 $_{0.1}$	8.3 $_{0.1}$	21.8 $_{1.6}$	11.3 $_{1.7}$	10.2 $_{1.2}$	1.8 $_{0.3}$	17.9 $_{0.3}$	14.7 $_{0.2}$	10.2 $_{0.2}$	1.8 $_{0.0}$
$\mathbb{CSP}$	ResNet-101	38.9 $_{0.1}$	32.1 $_{0.2}$	25.2 $_{0.1}$	9.9 $_{0.1}$	42.2 $_{1.2}$	9.1 $_{1.3}$	10.0 $_{1.1}$	2.6 $_{0.5}$	17.7 $_{0.1}$	17.0 $_{0.2}$	12.0 $_{0.1}$	2.3 $_{0.0}$
$\mathbb{CSP}$	ViT B/32	36.4 $_{0.4}$	42.5 $_{0.2}$	28.6 $_{0.1}$	12.4 $_{0.1}$	57.1 $_{0.4}$	57.3 $_{0.6}$	39.3 $_{0.6}$	24.2 $_{0.4}$	**30.1** $_{0.1}$	23.4 $_{0.2}$	19.4 $_{0.3}$	5.7 $_{0.1}$
$\mathbb{CSP}$	ViT L/14	**46.6** $_{0.1}$	**49.9** $_{0.1}$	**36.3** $_{0.1}$	**19.4** $_{0.1}$	**64.2** $_{0.7}$	**66.2** $_{1.2}$	**46.6** $_{1.2}$	**33.0** $_{1.3}$	28.8 $_{0.1}$	**26.8** $_{0.1}$	**20.5** $_{0.1}$	**6.2** $_{0.0}$

Table 3.14: Closed-world ablation results with respect to different backbone architectures of CLIP. We report the average performance of the model on 5 random seeds with standard error.

3.12 Pseudocode

Figure 3.7 shows the Torch-like pseudocode for inference with $\mathbb{CSP}$. The function accepts the minibatch of images, test pairs, and the clip model with the fine-tuned embeddings and returns the cosine similarities between the image representation and the text representation scaled by a constant scalar.

```
def inference(
    batch_images: nn.Tensor,
    test_pairs: List[List, List],
    model: nn.Module
):
    """
    Function to run inference with the fine-tuned embeddings.
    Args:
        batch_images (torch.Tensor): minibatch of images [n, h, w, c]
        test_pairs (tuple): attribute-object pairs in the test
            split [m, 2]
        model (nn.Module): model with the fine-tuned embeddings
    Returns:
        torch.Tensor: cosine similarties of the minibatch images
            and attribute-object pairs [n, m]
    """
    prompt_template = "a photo of x x"
    tokenized_prompt = tokenize(prompt_template)
    tokenized_prompt = tokenized_prompt.repeat(len(test_pairs))
    token_tensor = model.token_embedding(tokenized_prompt)

    # fine-tuned embeddings
    attr, obj = zip(*test_pairs)
    attr_emb = model.soft_embedding(attr)
    obj_emb = model.soft_embedding(obj)

    # replace the "x x" in prompt template with fine-tuned embeddings
    token_tensor = replace_emb(token_tensor, attr_emb, obj_emb)

    # l2-normalized
    text_rep = model.text_encoder(token_tensor)
    image_rep = model.image_encoder(batch_images)

    logits = (image_rep @ text_rep) * model.logit_scale.exp()

    return logits
```

Figure 3.7: Torch-like pseudocode for inference with $\mathbb{CSP}$.

3.13 Feasibility Calibration for Open-World Setting

Feasibility calibration aims to filter out infeasible compositions that might be present in the open-world setting. To filter out infeasible compositions, we follow the post-training calibration from Mancini et al. (2021b). They conjecture that similar objects share similar attributes while dissimilar objects are unlikely to share attributes. For example, `cat` and `dog` can share the attribute `old` but `cat` and `cliff` do not share the attribute `eroded`.

We calculate the feasibility compositions for the composition (a, o) by computing the relationships between the objects and the attributes. First, we find the similarities between the objects:

$$\rho_o(a, o) = \max_{\hat{o} \in \mathbb{O}_{seen}} \frac{\phi(o) \cdot \phi(\hat{o})}{||\phi(o)||\ ||\phi(\hat{o})||}$$

Hyperparameter	*MIT-States*	*UT-Zappos*	*C-GQA*
Learning rate	$5e-05$	$5e-04$	$5e-05$
Batch size	128	128	128
Attribute dropout	0.3	0.2	0.3
Weight decay	$1e-05$	$1e-05$	$5e-05$

Table 3.15: Hyperparameters for MIT-States, UT-Zappos, and C-GQA.

where $\rho_o(.)$ is the similarity between the object o with other objects $\hat{o}$ and $\phi(.)$ is an embedding function that maps attributes to pretrained embedding. We compute similarities between the attributes in the same way.

Next, we combine the two similarities with a pooling function. In our case, we use mean pooling μ:

$$\rho(a, o) = \mu(\rho_o(a, o), \rho_a(a, o))$$

where $\rho(a, o)$ is the feasbility score for the composition (a, o). Finally, we filter out infeasible compositions by considering compositions above a threshold T calibrated on the validation set to get our final prediction:

$$\hat{y} = \underset{y \in \mathbb{Y}_{test},\, \rho(a,o) > T}{\operatorname{argmax}} p_{\boldsymbol{\theta}}(y \mid x)$$

Following prior work, we compute feasibility calibration using GloVe embeddings (Pennington et al., 2014), and filter out the infeasible attribute-object compositions based on the performance on the validation split.

3.14 Hyperparameters

In our work, we find the best hyperparameters for training $\mathbb{CSP}$ via a grid search. We train the ViT-B/32 model for 50 epochs and use the same best performing hyperparameters to train all our models including ViT L/14. We run a grid search with the following hyperparameters: (1) learning rate: $\{5e-03, 5e-04, 5e-05\}$, (2) batch

size: $\{128, 256\}$, (3) attribute dropout: {0.0, 0.1, 0.2, 0.3}, and (4) weight decay: $\{1e-05, 5e-05\}$. We choose the hyperparameters for a dataset based best unseen accuracy on the validation split. We reduce the number of epochs to 20 with ViT L/14 as we found our models tend to converge earlier.

Table 3.15 shows the hyperparameters used to train $\mathbb{CSP}$ on all the datasets. The experiments with CoOp and CLIP adapters do not use dropout as the original architecture did not use dropout in the text or the image encoder. For the rest of the architecture, we use the same hyperparameters for CoOp and CLIP adapters as $\mathbb{CSP}$.

3.15 Additional Qualitative Examples

In this section, we include additional examples from the compositional zero-shot image to text retrieval tasks. Selected samples from CGQA in Figure **??** show that fine-tuning the vocabulary enables $\mathbb{CSP}$ to better identify composed concepts compared to CLIP. In particular, we observe that $\mathbb{CSP}$ ranks relevant attributes in the attribute-object composition better than CLIP.

3.16 Dataset Creation

We create AAO-MIT-States from the MIT-States dataset (Isola et al., 2015). Below we include details on the annotation interface, annotators, and aggregation process for the annotations.

We annotate an additional attribute for the images paired with unseen classes in the test split of MIT-States. The interface has three main components: (1) general instructions, (2) image with caption, and (3) list of populated attributes. The general instructions provide the annotators with a detailed description of the annotation task. To the right of the instructions is a randomly sampled image from the test split of MIT-States. Additionally, we include a caption describing the image. For example,

suppose we select an image of a wet cat, we ask the users: “which attribute best describes the cat in the image presented?”. Since we have a large number of attributes in the MIT-States dataset, we need a way to reduce the list of attributes the user observes while annotating a single image. We use CLIP to predict the attributes except for the original attribute in the image and choose the top-5 attributes as annotation candidates. We also include an option to select none of the above.

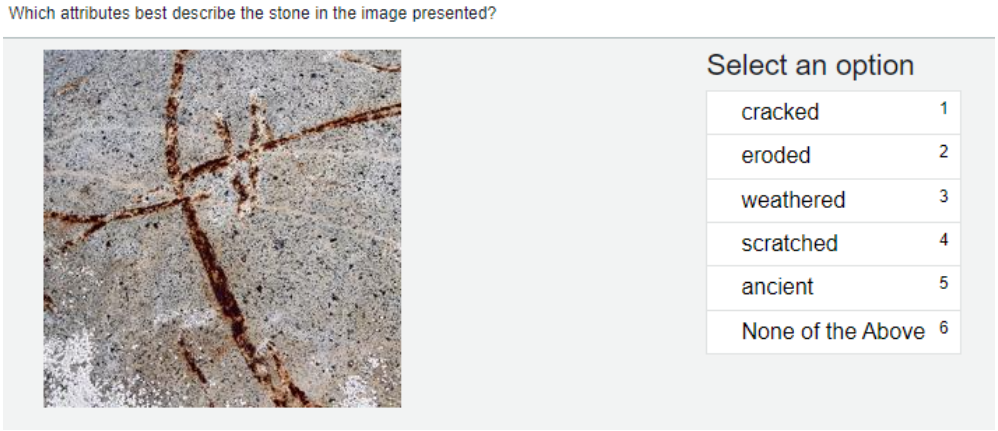

Figure 3.9: Example annotation interface.

The dataset was annotated by two of the authors and two undergraduate research assistants. We randomly sampled a total of 1200 images from test-split and asked the annotators to annotate from the interface. We received annotations for 1089 instances where each image received exactly three annotations. We aggregate examples for our dataset where all the three annotators agreed on the same attribute other than “None of the above". The total number of examples in the final annotated dataset is 193. We have open-sourced the dataset in our code.

CHAPTER 4

Alfred: A System for Prompted Weak Supervision

[1]

In this chapter, I introduce Alfred, the first prototype system designed to facilitate prompted weak supervision through natural language interfaces. Alfred enables domain experts to create labeling functions using prompt templates instead of writing code, thereby lowering the technical barriers associated with traditional weak supervision workflows. By supporting knowledge expression in natural language, Alfred broadens participation in machine learning development and provides a more intuitive, accessible approach for incorporating domain expertise into labeling pipelines.

4.1 Introduction

Acquiring labeled data is a significant challenge for machine learning for its time-consuming and expensive nature. Programmatic weak supervision (PWS) provides a more efficient method of data annotation by using noisy heuristics to label data. In a

[1] https://github.com/BatsResearch/alfred

Figure 4.1: Examples of a labeling function versus a prompted labeling function. For the first example, each expresses supervision relating mentions of President Biden to the category of politics. Instead of specifying an intricate regular expression, a prompted labeling function uses the prompt “Text: [[instance]] Does this text mention President Biden?” where [[instance]] is replaced by the news article to be labeled. The response is mapped to a vote on the true label. The second example demonstrates how heuristics about positive sentiment that were previously hard to define can be flexibly expressed as a natural language question. Instead of defining a set of keywords for fuzzy sentiment matching, we can simply ask large pretrained models for answers about the sentiment. For the third example, we consider an animal labeling task where we use the visual attributes “stripes” to vote for the classes TIGER and ZEBRA. Previously, we would need to first collect supervised training data for attributes like stripes and then train classifiers to make the decisions. With modern vision-language models (e.g. CLIP), we can simply express the attribute detection task as a set of candidate prompts.

typical PWS setup, domain experts design labeling functions (LFs) as programs that vote for a label or abstain. (Ratner et al., 2016b, 2017) Recently, there has been a growing interest in creating LFs from large, pre-trained models through prompting (Smith et al., 2022; Arora et al., 2022; Zhang et al., 2022b). In the shift to this new setting, executing LFs goes from the least to the most computationally expensive part of the process, highlighting the importance of providing a software infrastructure that facilitates efficient development. However, existing toolkits for large language models mainly prioritize prompt templating and tuning, leaving an unmet need for a system that that connects prompting with the creation of training data.

Prompted models offer a unique opportunity to enhance existing PWS systems. Traditional PWS systems require programming LFs with code that specifies heuristic domain knowledge. With large pre-trained models, natural language-based prompts can be used as LFs, also known as prompted LFs (Smith et al., 2022). This approach allows the easy expression of complex rules that were previously difficult to specify using code, as the example in Figure 4.1 shows. The ability to use prompts to define labeling functions simplifies and streamlines the weak supervision process, as well as potentially elevating the quality of the annotations. This benefit is particularly helpful for tasks involving computer vision, where previously PWS has been limited to tasks for which models can identify key features (such as objects) over which to program rules. Whether the domain is language or multi-modal, prompts let users experiment with different heuristics (and phrasings of those heuristics). Therefore, enabling an iterative development experience is essential for the success of a prompt-based PWS system.

The switch to prompted models for weak supervision also presents significant challenges. It first requires rethinking the abstractions and workflow of first-generation PWS systems. Instead of editing code and managing libraries of functions, users must manage libraries of prompts, track their outputs on multiple datasets, and develop strategies for mapping those outputs to labels. This change is complicated by large models' demand for computational resources. The throughput of the models is a new development bottleneck. The extra overhead of remotely hosted models is a further hindrance to the iterative workflow of weak supervision.

Existing open-source software for prompting concentrates on prompt engineering (Orr, 2022; Bach et al., 2022b), prompt chains (Chase, 2022) or continuous (i.e., soft) prompt tuning (Ding et al., 2022); placing less emphasis on throughput for a large-model-in-the-loop workflow. Additionally, many existing open-source systems have not developed support for vision-language models, despite their benefits for data labeling.

Incorporating large pre-trained models into a PWS system remains an open problem in the open-source software space, requiring innovative solutions to address these challenges and complement existing software focused on other aspects of prompting.

We present Alfred, a versatile PWS system that leverages large pre-trained models for image and text annotations. Alfred aims to provide an environment for the rapid development of prompt-based supervision, while maintaining a consistent development experience similar to established PWS systems. We designed Alfred with usability and efficiency in mind, aiming to provide a rapid and smooth experience for developing prompt-based supervision. Alfred supports popular large language models from Hugging Face's transformer package (Wolf et al., 2020), including the GPT family (Radford et al., 2019; Brown et al., 2020), the T5 family (Raffel et al., 2020), etc., and vision-language models like CLIP (Radford et al., 2021a), etc. Alfred also supports local ONNX models, or API-based models from OpenAI, AI21, and Cohere. Moreover, Alfred provides easy templating tools to help users quickly create, evaluate, and refine prompted LFs. Alfred offers easy ways to deploy inference servers remotely, in addition to local model hosting. Alfred also optimizes model inference throughput with improved batching techniques and provides utilities for efficient LLM deployment and interaction. Finally, Alfred contains a library of label models to distill the outputs of prompted labeling functions into the final training datasets for downstream end models.

Alfred is a prototype for a second generation of PWS systems with prompting at their core. To this end, we highlight three key feature of Alfred:

- **Prompt-based weak supervision for images and text.** Alfred provides the necessary tools for users to create, evaluate, and refine prompt templates for both image and text weak supervision tasks. The inclusion of a query caching system that automatically stores and updates model responses facilitates development.
- **Optimized inference throughput.** Alfred implements a dynamic batching mechanism that optimizes large sets of prompts. This feature allows models hosted by

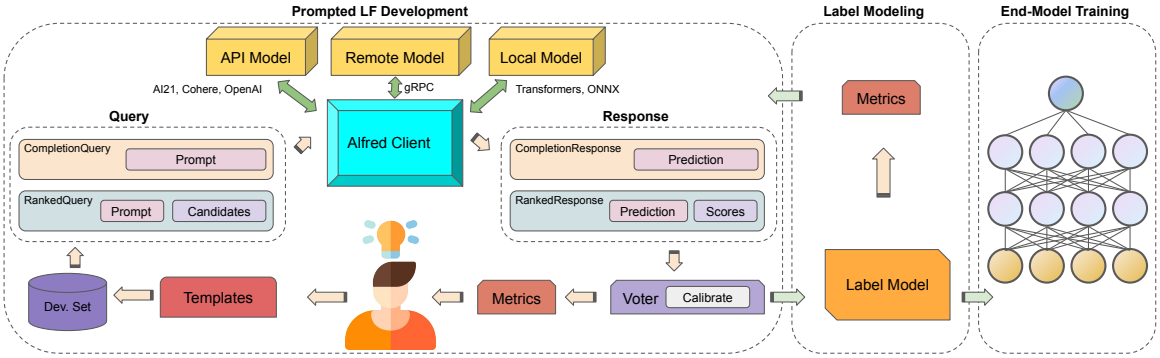

Figure 4.2: A typical workflow for programmatic weak supervision with Alfred. First, developers use Alfred to iteratively design, evaluate, and refine their prompted labeling functions (LFs). They use prompt Templates to generate Queries to Models based on data. The responses of Models are mapped to votes on the true labels for examples by Voters, which can be calibrated. Then the included Label Models combine the noisy votes to produce probabilistic training labels for an end model.

Alfred to achieve 2-3× greater throughput than naive implementations.

- **Seamless local development experience.** Alfred can host models remotely and make them accessible to developers via gRPC, a high-throughput protocol for remote procedure calls.[2] It enables sending datasets to servers in large chunks to maintain higher throughput. Alfred also implements a SSH-based port-forwarding utility for the gRPC connection, easing deployment on shared clusters such as those at universities.

4.2 Related Work and Background

Alfred sits at the intersection of programmatic weak supervision and large pretrained models. In this section, we overview the most related work.

Programmatic Weak Supervision. Traditionally, supervised learning relied on manually labeled data, and data labeling has been seen as a key bottleneck for many applications. Recently, a family of programmatic weak supervision (PWS) methods have offered an alternative to costly manual annotations by incorporating multiple

[2] grpc.io

sources of noisy labels to create training datasets(Zhang et al., 2022a). Typically, a PWS system such as Snorkel (Ratner et al., 2017) has a three-stage setup: First, developers will create heuristics called labeling functions (LFs) that vote on the label for an example (or abstain). Second, a label model will reconcile the noisy votes and provide probabilistic labels for the data. Finally, freshly annotated data is used to train an end model with a noise-aware loss objective (e.g. in a classification setting, this can be a soft cross entropy (Ratner et al., 2016b)). Alfred focuses on the first two stages and aim to efficiently incorporate modern large pretrained models into the development workflow.

Prompting for Pre-Trained Models. With the emergence of large pre-trained langugae and vision-language models, prompting has become a popular approach to many few-shot and zero-shot tasks (Brown et al., 2020; Schick and Schütze, 2021a; Radford et al., 2021a). Prompting can create training examples, generate data, modify datasets, and improve model reasoning (Schick and Schütze, 2021b; Ye et al., 2022a; Chia et al., 2022a; Wu et al., 2022a; Wang et al., 2022; Wei et al., 2022; Zelikman et al., 2022). This presents a unique opportunity for combining prompting for large pretrained models and weak supervision. Recent studies have investigated strategies to combine large language models into weak supervision frameworks (Smith et al., 2022; Chen et al., 2022; Arora et al., 2022; Zhang et al., 2022b). Alfred aims to provide a platform for the rapid development of weak supervision applications that rely on large pre-trained language and vision-language models, as well as enable experimentation with new ways of using those models.

Systems for Prompt Development. Prompting has led to the development of software toolkits that aid in prompting and research across various tasks. Many tools have been developed for various use cases with large language models. PromptSource (Bach et al., 2022b) is a development environment for creating and archiving sets of prompts. OpenPrompt (Ding et al., 2022) is a library focused on tuning prompts and prompted

models with training data. Manifest (Orr, 2022) provides a unified front end for prompting large language models via different APIs. LangChain (Chase, 2022) provides convenient utilities for building applications that chain together multiple prompts and outputs. To complement the existing tools in this growing space, Alfred is designed to be a PWS system based on prompting both large language and vision-language models.

4.3 Prompted LF Development

Alfred is designed to enable the development and application of prompted labeling functions (LFs) Compared to the typical workflow of PWS systems, where developing LFs is not computationally demanding, developing prompted LFs has model inference as a bottleneck. These large models are often hosted remotely on virtual instances or computing clusters, which can add to the challenge of iterative prompt development. In an iterative development environment, creating prompted labeling functions requires a platform that provides optimal throughput and low latency for a rapid local development experience. To illustrate Alfred's key focuses, we illustrate a typical workflow (Figure 4.2) for using Alfred to create a training dataset:

Step 1: Task Setup For a large model to be used in the development loop, developers can elect to use either self-hosted models or API-based models. For self-hosted models, Alfred provides an `AlfredServer` to host the model on cloud or cluster nodes. As the main development interface, the user can simply start a `Client` by specifying the type of model. Before creating prompted LFs, users need to familiarize themselves with the task by exploring the raw, unlabeled dataset. If there is no development subset available, the developer can annotate a small portion of the data as a held-out evaluation benchmark. Alfred implements a `Dataset` class based on Apache Arrow for fast data access. User may load a `Dataset` from CSV, JSON or Dataframe objects. It also offers direct support for datastes from the Hugging Face 'Dataset' package (Lhoest

et al., 2021). During the exploration process, developers may gain insights into the data and the label space, and identify potentially useful heuristics. Moreover, users can freely experiment with prompts with a few unlabeled instances by directly interacting with the `Client`.

Step 2: Iterative Prompt Development: When the user is ready for prompt development, they can use a `Template` to define a prompted LF for either text completion or scoring schemes. A `Template`, given a `Dataset`, will produce the corresponding `Query` objects. `Client` will return the appropriate `Response` object for each `Query`. To map the model responses to votes, users define the corresponding `Voter` and identify the label maps and matching functions to be used for each prompted LFs. Label maps define how potential model responses are associated with the label space. Matching functions specify how the `Voter` determines a match. By default, Alfred employs an exact match mechanism, but this can be substituted with user-defined matching functions for uncased matching or embedding similarity matching, etc. A `Voter` can be optionally calibrated to reduce model-specific biases (Zhao et al., 2021). With the model responses and the Voter, users can obtain the label votes for each of their prompted LFs and examples in a matrix format. Finally, users can evaluate the quality of their prompted LFs using a set-aside development `Dataset` with desired metrics. Here, users can continue to refine their prompted LFs and iterate as necessary. Once users are satisfied with the performance of their development benchmark, they may proceed.

Step 3: Aggregate Prompt Responses Finally, Alfred can aggregate the votes from each `Voter` with a `LabelModel` to produce probabilistic estimates of the true labels for the examples. Alfred also supports *partial labels*, i.e., labels that narrow down the possible set of classes but are not specific enough to vote on a single class (Yu et al., 2022). The probabilistic labels can then be used to train a wide range of end models.

```
from alfred import Client
from alfred.fm.query import RankedQuery,
                            CompletionQuery
LMClient = Client(...)

headline = "Liverpool wins 7-0!"

LMClient(
    CompletionQuery(headline
    + " What is the topic of this headline?")
)
# Example Response:
# >> CompletionResponse(prediction="Football")

LMClient(
    RankedQuery(headline
    + " What is the topic of this headline?",
      candidate=["Sports", "Politics",
                 "Tech", "Business"])
)
# Example Response:
# >> RankedResponse(prediction="Sports",
#      scores={"Sports":0.76, "Politics":0.10,
#              "Tech":0.07, "Business":0.07 })
```

Figure 4.3: Typed `Query` and `Response` in Alfred

4.4 System Design

In this section, we describe and highlight the key design decisions for Alfred.

4.4.1 Query and Response Types

We identify two main patterns using prompts for PWS: text completion and scoring. Text completion is when a language model generates responses using a heuristic decoding strategy over the whole model vocabulary, while scoring is when a language ranks candidate completions or a vision-language model ranks candidate prompts, i.e., captions. Alfred implements typed `Query` and `Response` classes for these two patterns (Figure 4.3). Upon applying the `Template` operation on a dataset instance, it produces either a `CompletionQuery` or a `RankedQuery` for each instance based

on the `Template` definition. The resulting query can be directly fed into the `Client`. The `Client` then returns a corresponding `CompletionResponse` or `RankedResponse` with the prediction as the main payload, along with any other requested or useful information, such as the logits for each candidate.

4.4.2 Templates for Prompted LFs

Prompt templates are at the core of systems for prompting. In Alfred, prompt templates are expressed as `Template` objects. For natural language tasks, users use the `StringTemplate` class. To produce `Query` objects, users can call 'Template.apply(instance)' or 'Template.apply_to_dataset(dataset).' A `StringTemplate` is defined with a template string with keywords enclosed by double square brackets, e.g. "[[text]] Does the previous context express spouse relation between [[entity_a]] and [[entity_b]]?". An optional field for `Template` is 'answer_choices,' where one may specify the candidate completions. By specifying the 'answer_choices,' the `StringTemplate` would yield a `RankedQuery`. An example code snippet showing the creation of a `RankedQuery` is in Figure 4.4. For image annotation tasks, users may define an `ImageTemplate` by specifying the candidate prompts. Upon applying to images, `ImageTemplate` will produce `RankedQuery` objects with the images and candidate prompts.

4.4.3 Throughput Optimization

Alfred is designed to handle large numbers of queries. Self-hosted models from the Transformers package (Wolf et al., 2020) are set up to use model parallelization enabled by Accelerate (Sylvain Gugger, 2022), with user-customizable device maps for parallelizing the model across multiple GPUs. Alfred adopts a dynamic batching strategy that groups instances with similar lengths together and adjusts the input batch size dynamically to maximize model inference throughput. The core idea of the

```python
from alfred.template import StringTemplate, ImageTemplate

string_template = StringTemplate(
    "Context: [[text]]\n\nIs the above text about weather?",
    answer_choices = ["Yes", "No"]
)
example = {'text': "A pleasant day with a sunny sky."}
prompt = string_template.apply(example)
# >> RankedQuery("""Context: A pleasant day with a sunny sky.
#                   Is the above text about weather?""",
#                 candidates=["Yes", "No"])

image_template = ImageTemplate(
    {"label": ["cat", "dog"]},
    template = "A photo of [[label]]."
)
example = cat_image
prompt = image_template.apply(example)
# >> RankedQuery(example, candidates=["A photo of cat.", "A photo of dog."])
```

Figure 4.4: Example code snippet for creating a `RankedQuery` from a `StringTemplate` or a `ImageTemplate`.

dynamic batching strategy is to group input instances with similar token lengths to minimize padding and maximize memory utilization.

With the dynamic batching strategy, on a node with 8 NVIDIA Tesla V100s, Alfred achieves a speedup of up to 2.5× and a token throughput increase of 2.9× for approximately 500 queries (∼21,000 tokens) compared to an unoptimized strategy with T0++ (**?**), an 11-billion parameter T5-based (Raffel et al., 2020) language model in FP32. Additionally, Alfred includes a client-side query caching system that automatically stores and updates model responses to facilitate prompt development and avoid redundant queries during development. Alfred also implements a server-side caching system for large multi-modal pretrained models such as CLIP. At inference time, Alfred will cache the input data with its corresponding encoded latent representations from different encoder head for each modalities. This server-side caching system effectively avoids redundant encoding computation on the server end.

4.4.4 Remote Self-Hosting of Models

The computational demands of large pre-trained models can pose a challenge when using them for weak supervision development. To address this challenge, Alfred provides utilities for deploying and interacting with models on remote virtual instances or computing clusters. Additionally, Alfred implements a SSH-based tunneling service that ensures a secure local connection while preserving all Alfred functionality. The tunneling utility also simplifies deployment of the server on shared computing clusters, with the login node serving as a jump host for the computing node. This is particularly useful for using Alfred on centrally-managed shared computing clusters such as those at universities. Alfred's built-in SSH tunneling is also capable of handling 2-factor authentication, which is common for shared clusters. To enable efficient communication between the client and server, Alfred uses gRPC, a high-performance, open-source remote procedure call framework. This enables Alfred to provide a seamless development experience for weak supervision development without the need for expensive local resources.

4.4.5 Mapping Responses to Votes

Another core piece of the Alfred system is the `Voter` class. Each `Voter` defines how to map model responses to votes for the true label of an exmaple. The votes can be class labels or partial labels (e.g., attributes) specified by the users. The voting mechanism also relies on a matching function, which by default only casts a vote for an exact match. Users may provide their intended matching mechanisms such as uncased matching or embedding similarity matching for more flexibility for each `Voter`. Furthermore, `Voter` can be contextually calibrated for the specific `Template` class to reduce model bias towards predicting certain answers. Recent studies show calibration can be helpful for many prompt-based tasks (Zhao et al., 2021; Smith et al., 2022). By calling 'Client.calibrate(Template, Voter),' Alfred will calibrate the voting weights

according to the strategy proposed by Zhao et al. (2021) and automatically apply the calibration during voting.

4.4.6 Label Models for Aggregating Votes

Alfred currently includes four label models for combining the votes from prompted labeling functions. The four label models are available to meet different use cases. The `MajorityVote` model is a baseline option suitable for fast development iteration, while the `NaiveBayes` model is recommended as the standard label model. Alfred also includes `NPLM` (Yu et al., 2022) (noisy partial label model) to support weak supervision from partial labels, which are labels that narrow down the possible set of classes but are not specific enough to vote on a single class. `FlyingSquid` (Fu et al., 2020) is the fourth model option and is recommended when `MajorityVote` is not accurate enough but more speed than `NaiveBayes` is needed. These label model classes have a unified interface, providing a consistent experience for users. After processing votes, the label model module generates probabilistic labels, represented as a distribution over the label space, for the given unlabeled dataset. Finally users can use the estimated probabilistic labels to train an end model for the downstream task.

4.5 Example Use Cases

In this section, we present two example use cases for how Alfred can be used to create training data for specific machine learning tasks using natural language prompts in both text and image domains. We measure the labeling quality by taking the top-1 accuracy of the estimated probabilistic labels given by the label model. The notebooks to reproduce these examples are in the Alfred repository.

4.5.1 Youtube Comment Spam Detection

Zero Shot	Prompted LFs	Prompted LFs+C
46.8	57.8	65.3

Table 4.1: Top-1 accuracy on YouTube spam detection. Zero Shot refers to prompting T0++ directly. +C means applying contextual calibration on the `Voter` objects.

In this experiment, we use Alfred to annotate the training split of YouTube spam detection dataset. (Alberto et al., 2015) We replicate the setup used by smith2022language The prompts are translated from the code-based labeling functions provided by the WRENCH benchmark (Zhang et al., 2021b), a comprehensive weak supervision benchmark. Alfred also includes a `WrenchBenchmarkDataset` abstraction for easily running this benchmark. In total, we define 10 prompted labeling functions with `StringTemplate` objects. Responses are mapped to votes using `Voter` objects. For this experiment, we use T0++ (Sanh et al., 2022) as the backbone model for Alfred. Following smith2022language, we also calibrate the responses from T0++ when voting using the contextual calibration strategy proposed by zhao-2022-auto. Finally we aggregate the votes using the `NaiveBayes` label model to produce the probablistic labels. Table 4.1 shows that Alfred makes reproducing the results of smith2022language easy, demonstrating that the combination of weak supervision and calibration yield a dramatic improvement over zero-shot prompting alone.

4.5.2 Pet Breed Classification

Zero Shot	Prompted LFs
86.0	92.4

Table 4.2: Top-1 accuracy on Oxford-IIIT Pet breed classification. Zero Shot refers to prompting CLIP directly.

Traditionally, programmatic weak supervision for vision has been limited by the ability to express supervision in code, relying on models such as object or attribute detectors to extract features and classify. However, these object detectors often depend

heavily on supervised training data, becoming a bottleneck for applying programmatic weak supervision in various vision tasks. Fortunately, with large-pretrained vision-language model like CLIP (Radford et al., 2021a), we are now able to express supervision with natural language. For this task, we develop prompts to classify 37 different breeds of pets from the Oxford-IIIT Pet dataset (Parkhi et al., 2012). We use CLIP-ViT/L-14 as the backbone model and developed three simple prompted LFs. The first two prompted LFs use templates "a photo of [[label]]" and "a photo of [[label]] [cat/dog]" where "[cat/dog]" is selected based on the breed. The third prompted LF produces a partial label with the template "a photo of [cat/dog]", encouraging fine-grained labels to match with the coarse-grained type detected by CLIP. We combine the votes using the `NPLM` label (Yu et al., 2022) to support weak supervision at various levels of granularity in the label space. Table 4.2 shows that this multi-granular weak supervision provides a nice boost over zero-shot prompting alone.

4.6 Discussion and Future Work

This paper introduces Alfred, a prototype for the next generation of programmatic weak supervision systems that leverage the potential of large pre-trained models. Alfred complements the existing ecosystem of large-pretrained-model toolkits, offering optimized inference throughput, a smooth local development experience, and compatibility with vision-language models to support image annotation tasks. Alfred represents a notable advancement in the domain of programmatic weak supervision, as it enables users to express their domain-specific knowledge and heuristics with flexible natural language prompts for language and vision-language models. This approach can be more user-friendly than conventional PWS systems, which requires expert programming of weak supervision sources. Our objective is for Alfred to serve as the infrastructure and experimentation platform for many future weak supervision research projects and applications. Furthermore, we plan to extend Alfred's capabilities to accommodate a

wider range of multimodal large pre-trained models, such as Whisper (Radford et al., 2022) and LayoutLMs (Xu et al., 2020a,b; Huang et al., 2022).

4.7 Limitations

Alfred is a prototype for the second generation of PWS systems, which incorporate large pre-trained models. However, there are some potential limitations to consider. As with all PWS approaches, application quality is limited by the quality of the weak supervision sources used to vote on the labels. In this case of prompted labeling functions, this depends on how well suited the prompts and model are to the task and domain. If they are not well suited, then additional fine-tuning of the prompted models will be necessary. Compared with traditional labeling functions written in code, understanding when and why labeling functions fail on certain examples can be particularly challenging. Methods for explanations such as minimal contrastive edits (**?**) can potentially help address this limitation. We plan to explore incorporating such methods into Alfred.

4.8 Ethics Statement

One major concern for Alfred is the potential for biased or unfair labeling. Large pre-trained models are trained on massive datasets, which can reflect societal biases and inequalities. Consequently, supervision generated by these models can perpetuate and amplify these biases, leading to discrimination or unfair treatment in downstream applications. Therefore, it is essential to carefully consider the quality and representativeness of the backbone model for Alfred, as well as the prompts used for labeling data. To address potential labeling biases, human oversight and auditing are needed during the development loop to spot and correct any issues. While Alfred has the potential to enhance the efficiency of programmatic data labeling, it is crucial to carefully consider

and address potential ethical challenges.

4.9 Conclusion

In this chapter, I addressed the limitation of technical barriers to knowledge transfer, where current systems often require domain experts to have programming skills to encode their knowledge into rigid, code-based labeling functions. I introduced *Alfred*, a prototype system designed to enhance programmatic weak supervision by leveraging large pre-trained models, including large language models and vision-language models. Alfred enables domain experts to express their domain-specific knowledge and heuristics through flexible natural language prompts, eliminating the need for programming expertise and making the labeling process more accessible.

By reducing the manual workload and lowering technical barriers, Alfred empowers domain experts to effectively transfer their knowledge into data labeling tasks within weak supervision frameworks to facilitate efficient knowledge transfer from experts to models. Alfred's optimized inference throughput, smooth local development experience, and compatibility with vision-language models for image annotation tasks further enhance the practical application of weak supervision.

CHAPTER 5

Leveraging Large Language Models for Structure Learning in Prompted Weak Supervision

This chapter explores how large language models can be efficiently used to model dependencies between labeling functions in prompted weak supervision. By leveraging embedding similarities among prompts, the Structure Refining Module introduced in this chapter improves label aggregation performance, especially in settings with redundant or correlated prompted labeling functions.

5.1 Introduction

Scarcity of labeled data is a crucial bottleneck for many supervised machine learning applications. Recently, programmatic weak supervision (PWS) (Ratner et al., 2016b, 2017; Zhang et al., 2022a) emerges as a promising approach for efficiently creating large amounts of labeled data in alternative to costly manual labeling. PWS relies on multiple expert-designed labeling functions (LFs) from rules or heuristics to act as

weak supervision sources and vote for a label. In practice, given unlabeled data, a PWS system infers probabilistic labels by resolving the agreements and disagreements among the noisy LFs through label modeling. Recently, a new paradigm called prompted weak supervision (PromptedWS) (Smith et al., 2022; Yu and Bach, 2023) has been proposed to integrate pre-trained large language models (LLMs) with PWS. In PromptedWS, program-based LFs written in code are replaced with prompted LFs: natural language prompts in which their respective labeling decisions are made through LLM inferences. However, the predictions from LLMs can be strongly correlated due to model biases, leading to inferior labeling performance. Accurately estimating and managing the dependency structure among prompted LFs presents a challenging problem. Pro-

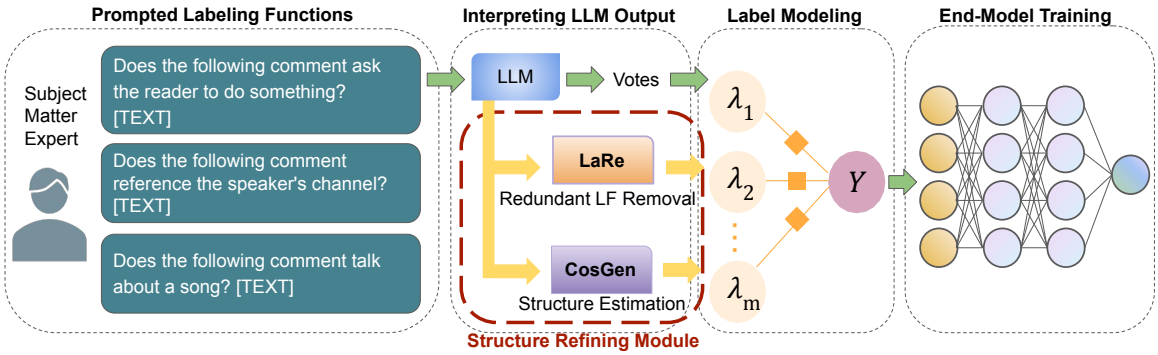

Figure 5.1: Prompted weak supervision workflow showing the Structure Refining Module as a plugin.

grammatic weak supervision has been successful for various applications in information extraction, medical imaging and sequence tagging (Bach et al., 2019a; Suri et al., 2020; Ré et al., 2019; Kuang et al., 2022; ?). Recently, there have been a growing interest in integrating powerful pretrained LLMs into weak supervision workflows (Smith et al., 2022; Arora et al., 2022; Zhang et al., 2022b). prompted weak supervision (PromptedWS) (Smith et al., 2022) presents a unique opportunity to replace rigid code-based LFs with more flexible natural language prompts. While this approach offers a much more flexible paradigm for weak supervision sources, prompted LFs may be susceptible to model bias and strong correlations due to their reliance on the outputs of LLMs. Therefore, it is crucial to identify and manage the dependency structures among the

prompted LFs to ensure accurate label estimation.

While structure learning in the supervised learning settings has been extensively studied (Meinshausen and Bühlmann, 2006; Zhu et al., 2010), learning the structure in programmatic weak supervision presents significant challenges due to the absence of the true labels (Bach et al., 2017b). The difficulties are further exacerbated in a prompted weak supervision setup, where individual supervision sources are expressed as natural language prompts for LLMs, the sole shared knowledge source. To handle the correlations of labeling functions (LFs), early work relies on user-specified dependency structures (Ratner et al., 2016b). Previous studies have also explored how to automatically discover the structure with only unlabeled data (Bach et al., 2017b; Varma et al., 2017). However, existing methods are not designed to handle correlations in Prompted weak supervision setup. In prompted weak supervision, weak labels are solely based on querying prompted LFs via the Large Language Models and the model responses alone may not fully represent the correlations among the prompted LFs.

In this paper, we propose a Structure Refining Module, a simple yet effective structure learning method that plugs into existing PromptedWS workflows (Figure 5.1). The key idea is to guide and accelerate the structure learning process by using the LLM's internal representation of prompted LFs. We find that the similarity of the representations of the prompted LFs is predictive of harmful correlations. Further, they can be computed much more quickly than executing the prompted LFs on many examples. Our approach contains two core components: ***La****beling function* ***Re****moval* (LaRe) and ***Co****rrelation* ***S****tructure* ***Gen****eration* (CosGen). LaRe is used for automatically detecting redundant prompted LFs to reduce both LLM query cost and avoid severe biases. CosGen is specifically designed to efficiently learn the dependency structures among prompted LFs, utilizing only the vector embeddings of the prompted LFs themselves, without relying on any labels or unlabeled data. In practice, LaRe will first be applied on the given set of prompted LFs to detect and remove any unnecessary prompted LFs. Then

CosGen will produce the dependency structure for label modeling. Our approach allows the end users to quickly and accurately discover the correlations while significantly reducing the computational cost for both structure learning and label modeling.

To summarize, our main contributions are three-fold:

1. We propose a novel method for efficient structure learning in prompted weak supervision by leveraging similarity in the embedding space of prompted labeling functions.
2. We demonstrate the effectiveness of our proposed method by showing that it outperforms the prompted weak supervision baseline by an average of 5.9 points while saving significant computation.
3. We conduct comprehensive ablation and analysis experiments. We validate the core assumptions and analyze the contribution of each component of our method.

5.2 Background

5.2.1 Problem Setup

Prompted Weak Supervision: In a prompted weak supervision (PromptedWS) setup, given a dataset $D = \{x_1, x_2, \cdots, x_n\}$ of classification task with n unlabeled data points and each $x_i (i \in [n])$ with an unobserved true label $y_i \in \mathcal{Y}$, the goal is to infer the true label using weak supervision. We are also given a small set of heuristics created by subject matter experts (SMEs) manually for labeling the unlabeled data. For example, in the case of labeling spam comments, the SME may find that many spam examples contains call to action, such as asking readers to subscribe to a channel or asking the readers to buy something. Then she may create a heuristic such as "Does the following comment ask the reader to do something?". In PromptedWS, these heuristics are formatted as prompts, which are then fed into an instruction-tuned LLM

such as T0++ (Sanh et al., 2021) to get the answer. After that, we map the answer to a label or abstain according to the SMEs' heuristics. Formally, by observing the features in the development set, the SME could design a set of heuristics $\mathcal{H} = \{h_1, \cdots, h_m\}$ as well as a label map $\mathcal{M} : \mathcal{S} \to \mathcal{Y} \cup \{\emptyset\}$. The heuristics are encapsulated by a prompt template. For example, in the case of website comment, the prompt template would be

```
Does the following comment ask the reader to do something?  [TEXT]
```

where `[TEXT]` is a placeholder for the text of each comment. By feeding this template to the pre-trained LLM $\mathcal{A}$, it returns a text string: $s = \mathcal{A}(h, x) \in \mathcal{S}$, where $\mathcal{S}$ is the set of all possible strings that could be returned from the LLM. Then the label map $\mathcal{M}$ maps the returned answer s to one of the class labels within the label space $\lambda_i \in \mathcal{Y}$ to vote for a label or maps s to a special symbol $\emptyset$, indicating abstaining and not voting for any label. In the case of the above example prompt, the label map would map positive responses such as `YES` and `True` to the SPAM label and all the other responses to abstentions.

Combining the above prompt template and label map together, we get a set of *Prompted labeling functions*: $S_{\text{LF}} = \{\mathcal{M}(\mathcal{A}(h_1, \cdot)), \cdots, \mathcal{M}(\mathcal{A}(h_m, \cdot))\}$. Applying the above m LFs to the unlabeled dataset $D = \{x_1, \cdots, x_n\}$ would create a $n \times m$ label matrix L. The rest of the workflow remains the same as a standard weak supervision framework: a *label model* is used to aggregates the votes in L to produce probabilistic estimates of the true label. While many label modeling methods assume conditional independence among the LFs given the latent label (Ratner et al., 2016b, 2017), recent label models may consider dependency structures provided by the user (Fu et al., 2020). Finally an appropriate *end model*, such as a deep neural network, is trained by minimizing the expected empirical risk with respect to the probabilistic estimates of the true labels.

5.2.2 Related Work

Programmatic Weak Supervision: Manually labeling training data is prohibitively expensive and time-consuming. A common alternative is to use weak supervision sources. Estimating the accuracy of multiple sources is well studied in many areas such as crowd sourcing(Dalvi et al., 2013; Joglekar et al., 2015), boosting (Schapire and Freund, 2013; Balsubramani and Freund, 2015b; Zhang et al., 2016), and co-training (Blum and Mitchell, 1998). In this paper, we focus on programmatic weak supervision (Ratner et al., 2016b, 2017; Zhang et al., 2022a), which estimates the accuracy of weak sources without *any* labeled data. In programmatic weak supervision, users encode various weak supervision sources such as user-written heuristics (Ratner et al., 2017; Meng et al., 2018; Awasthi et al., 2020b), knowledge bases (Liang et al., 2020; Hoffmann et al., 2011), pre-trained models (Bach et al., 2019a; Zhang et al., 2021a; Yu et al., 2022; Smith et al., 2022), and third-party tools (Lison et al., 2020) into labeling functions (LF), which can then give votes on the true labels of the unlabeled data.

Integrating LLMs into Weak Supervision: Prompting is a powerful technique for many few-shot and zero-shot applications with large language models (Liu et al., 2021). Prompting has been used to create and modify datasets in many ways (Schick and Schütze, 2021c; Ye et al., 2022b; Chia et al., 2022b; Wu et al., 2022b; Bonifacio et al., 2022; Lang et al., 2022; Elazar et al., 2021; Zhang et al., 2022c). Recent studies explored integrating prompting and weak supervision (Smith et al., 2022). Prompting also offers a unique opportunity to relax rigid code-based programmatic weak supervision sources. In this work, we aim to make the integration of LLMs into weak supervision more effective by mitigating the downside of hazardous correlations among the prompted LFs.

Structure Learning in Weak Supervision: It can be highly beneficial to capture the statistical dependencies among LFs during label modeling since model misspecification can lead to incorrect estimates of the true labels (Cachay et al., 2021). Multiple

methods for learning such dependencies have been proposed (Bach et al., 2017b; Varma et al., 2017, 2019b). Bach et al. (Bach et al., 2017b) and Varma et al. (Varma et al., 2019b) learn dependency structures from the votes of the LFs. Varma et al. (Varma et al., 2017) infer the structure through static analysis of the code specifying the LFs, with additional assumptions that the LFs operate over domain-specific primitives. With the dependency structure in hand, many methods can then incorporate this information into the label modelling process for improved label quality, either through factor functions (Ratner et al., 2016b; Shin et al., 2021) or graph structures (Ratner et al., 2019a; Fu et al., 2020; Varma et al., 2019c). The structure obtained by our Structure Refining Module can be incorporated into any such method.

5.3 Method

In this section, we describe Structure Refining Module for estimating the dependency structure among prompted LFs in the PromptedWS setting. We explore the potential of Large Language Models beyond its current usage in PromptedWS by leveraging its powerful representation capabilities. The core idea is to extract embeddings in addition to the inference output. Our method assumes that LFs with high similarity in the latent representation space are more likely to be correlated. Specifically, for each prompted LFs, we first extract vector embeddings from the last hidden layer of the LLMs. Then we can compute a pairwise similarity matrix M based on the extracted embeddings for all prompted LFs. Specifically, we encapsulated our method into one plug-in for PromptedWS. Our method contains two sequential components: ***La****beling function* ***Re****moval* (LaRe) and ***Co****rrelation* ***S****tructure* ***Gen****eration* (CosGen). LaRe is designed to remove heavily correlated LFs while balancing performance and efficiency. It can be especially helpful when the total number of LFs is very large. In addition to LaRe, we introduce CosGen to estimate the dependency structures for the prompted LFs. We include the pseudocode in 1. Next, we describe the LaRe and CosGen in

detail.

Algorithm 1 Pseudocode for Structure Refining Module

Input: Similarity matrix M, original Prompted LF set S_{LF}, number of LF to be removed: m_r, number of edges in the graph: m_e.
$S_r = \emptyset$, $\mathcal{E} = \emptyset$
// Labeling function removal (**LaRe**)
for t $\in [m_r]$ **do**
    $(i, j) = \arg\max M_{i,j}$
    $k = \max\{i, j\}$
    $S_r \leftarrow S_r \cup \{k\}$
    $M_{k,:} \leftarrow 0, M_{:,k} \leftarrow 0$
end for
$S'_{\text{LF}} \leftarrow S_{\text{LF}}/S_r$
// Correlation structure generation (**CosGen**)
Let M' be the similarity matrix of LFs in S'_{LF}
$(i, j) = \arg\min M'_{i,j}$
$M'_{i,:} \leftarrow 0,\ M'_{:,i} \leftarrow 0,\ M'_{j,:} \leftarrow 0,\ M'_{:,j} \leftarrow 0$
for $t \in [m_e]$ **do**
    $(i, j) = \arg\max M'_{i,j}$
    $\mathcal{E} = \mathcal{E} \cup \{(i, j)\}$
    $M'_{i,j} \leftarrow 0, M'_{j,i} \leftarrow 0$
end for
return Refined labeling function set S'_{LF}, edges in the graph $\mathcal{E}$

Name	Task	Class	P(positive)	#LFs	#Train	#Validation	#Test
YouTube	Spam Detection	HAM,SPAM	0.488(0.02)	10	1586	120	250
SMS	Spam Detection	HAM,SPAM	0.132(< 0.01)	73	4571	500	500
Spouse	Relation extraction	NOT SPOUSE, SPOUSE	0.074(< 0.01)	11	22254	2811	2701

Table 5.1: Summary statistics for text classification dataset used in our experiments. P(positive) is the class balance of the positive label calculated by the relative frequency of all gold labeled splits with standard error in the parentheses. Note that since we focus on PromptedWS, all the labeling functions we refer to here are prompted LFs as defined in Smith et al. (2022).

LaRe: The goal of LaRe is to remove redundant prompted LFs in order to reduce computation need while maintaining labeling accuracy. To begin, the user may decide how many LFs should be removed by examining the similarity matrix among the prompted LFs set S_{LF}. Higher concentrations in the similarity matrix indicate higher correlations or possibility of redundant LFs to be removed. In addition, more LFs could lead to more redundant LFs to be removed when the labeling function set S_{LF} is large.

	YouTube(Acc)	SMS(F1)	Spouse(F1)
WRENCH (Zhang et al., 2021b)	94.6 ± 0.5	93.5 ± 0.7	25.5 ± 1.8
PromptedWS (Smith et al., 2022)	94.8 ± 1.2	90.0 ± 4.1	52.1 ± 1.3
PromptedWS + WSSL (Varma et al., 2019b)	93.0 ± 1.3	**94.5**$\pm$1.7	63.9 ± 0.9
PromptedWS + Structure Refining Module	**95.6**$\pm$0.4	94.2 ± 1.4	**64.8**$\pm$0.2

Table 5.2: Main results from WRENCH benchmark, PromptedWS, PromptedWS with data based weak supervision structure learning method (WSSL) and PromptedWS with Structure Refining Module (ours). For all the experiments, we run 5 random seeds and reports the mean and standard error.

After deciding how many labeling functions to be removed, potentially redundant LFs are then identified, removed based on their similarity matrix M. First we will rank the pairwise similarities for each LF pair. Then at each step, we find the LF pair that has the largest similarity and remove the LF with a larger index to keep our algorithm deterministic. Removing the redundant LF results in a new set of LFs S'_{LF}, where $|S'_{\text{LF}}| = m - m_r$ and m_r is the total number of LFs we removed.

CosGen: CosGen aim to estimate the dependency among LFs based on the correlations indicated in the similarity matrix M', which contains only the correlation information of the refined LF set S'_{LF}. One key intuition is to construct dependency for highly correlated LFs indicated by M'. However, the structure directly estimated from this method may be non-identifiable and can not be used for label modeling. In order to guarantee the structure to be identifiable, we first find two LFs that have the smallest correlation and assume these two to be mutually independent while at the same time, also to be independent with all the other LFs. Then we find the rest dependencies by assigning edges to LFs with high similarity. The main hyperparameter for CosGen is the number of edges (m_e) we want to have in our graph and it decides how sparse the graph is. In practice, we can also replace this parameter to a percentage threshold on the total number of possible edges. Note that $m_e \leq \frac{(m-2)\cdot(m-3)}{2}$. The dependency structure inferred from our similarity matrix can serve as the input to any probabilistic label model that supports source dependency modeling.

5.4 Experimental Results

Dataset In our experiments, we evaluated on three datasets: YouTube (Alberto et al., 2015), SMS (Gómez Hidalgo et al., 2006), and Spouse (Corney et al., 2016). (See the summary of the datasets in Table 5.1.) We follow the prompted labeling functions from (Smith et al., 2022), which are translated from labeling functions from WRENCH (Zhang et al., 2021b), a standard weak supervision benchmark. We include all natural language prompts for our experiments in 5.8

Baselines We empirically evaluate the our method against three baselines: (1) the standard weak supervision with labeling functions in WRENCH benchmark (Zhang et al., 2021b). (2) PromptedWS (Smith et al., 2022) (without our refining module) (3) PromptedWS with weak supervision structure learning (WSSL) (Varma et al., 2019b) by passing the structure learned from WSSL to the label model in the PromptedWS) We report default metrics specified in the WRENCH benchmark (Zhang et al., 2021b) for direct comparison: for YouTube dataset, we report accuracy and for SMS and spouse dataset, we report F1. Our performance metrics are reported as the mean and standard error of 5 independent runs using different random seeds.

Setup We follow the conventional workflow of programmatic weak supervision and compare 4 approaches relevant to programmatic weak supervision and structure learning. For each dataset in our analysis, we assume the training splits are unlabeled, the LFs (either prompted LFs in PromptedWS or code-based LFs from WRENCH benchmark) are applied to the unlabeled training splits to generate weak labels. Throughout our experiments, we use T0++ (Sanh et al., 2021), a 11 billion parameter instruction-tuned large language model based on T5 (Raffel et al., 2020) architecture. T0++ is trained using large dataset of supervised tasks transformed into instruction prompted training data and can achieve competitive zero shot classification performance. T0++ model requires 42 GB of GPU to efficiently run locally without parameter offloading. We use 2 NVIDIA A100 GPUs for inference. In all of our experiments, unless otherwise

specified, we use Flyingsquid (Fu et al., 2020) as our label model to combine and denoise the labelers' votes into probabilistic labels and incorporate infered LFs dependency structures. The resulting probabilistic labels are then used to train an end classification model based on RoBERTa (Liu et al., 2019b).

Results Table 5.2 outlines the performance of the 3 baselines (WRENCH, PromptedWS, PromptedWS+WSSL) and as well as our method (PromptedWS+Structure Refining Module). The hyperparameters for LaRe, CosGen and the end model are chosen by performance on the validation dataset. For Spouse dataset, our method improves 39.3 F1 points compared to WRENCH benchmark, and improved 12.7 F1 score compared to the PromptedWS. For Youtube, Structure Refining Module increase 1.0 and 0.8 Acc points in comparison to WRENCH and PromptedWS respectively while surpassing PromptedWS+WSSL by 2.6 Acc points. For SMS dataset, Structure Refining Module also provides performance advantage ovber WRENCH and PromptedWS by 0.7 and 4.2 F1 points. On average, our method improves 13.7 points compared to WRENCH benchmark, 5.9 points compared to PromptedWS and 1.1 points compared to PromptedWS+WSSL over the three datasets. Besides potential performance advantage compared to WSSL, Structure Refining Module can be more efficient in both time and computation. We provide further analysis on performance efficiency of Structure Refining Module in Section 5.5.5.

Overall, our experimental results demonstrate that Structure Refining Module can significantly improve the performance over the PromptedWS which doesn't take into account the correlations among prompted LFs. In addition, the PromptedWS in complementary with our method can achieve superior performance, outperforming the WRENCH benchmark in all the three datasets used in our experiment. Most notably in Spouse dataset, we improved 39.3 F1 points over WRENCH benchmark. Our method outperforms or achieves comparable result to the state-of-the-art structure learning method WSSL (Varma et al., 2019b), indicating that the correlations found using our

refining model are more effective or comparable to learning the structure from data.

5.5 Ablation and Analysis

In this section, we provide further analysis on our core assumption that high similarities in LFs embedding similarities suggest correlation. We also exam the contribution of LFs removal and correlation structure generation from similarities from both performance and efficiency perspectives. We conduct experiments to evaluate the efficiency trade-offs for label modeling in our settings and lastly we provide intuitions on how to set the hyperparameters for Structure Refining Module.

5.5.1 Similarities reveal correlations

Table 5.3: Toy experiment of removing one of the most correlated LFs. The first row is the result of PromptedWS without moving any LFs, and the second and third rows are the result of PromptedWS after removing one LF from the most correlated LFs pairs.

	YouTube	**SMS**	**Spouse**
PromptedWS	94.8(1.2)	90.0(4.1)	52.1(1.3)
Removing most correlated LF	94.7±0.7	93.0±0.8↑	53.0±1.2↑
	95.3(0.3)↑	91.4(2.2)↑	52.3(1.8)↑

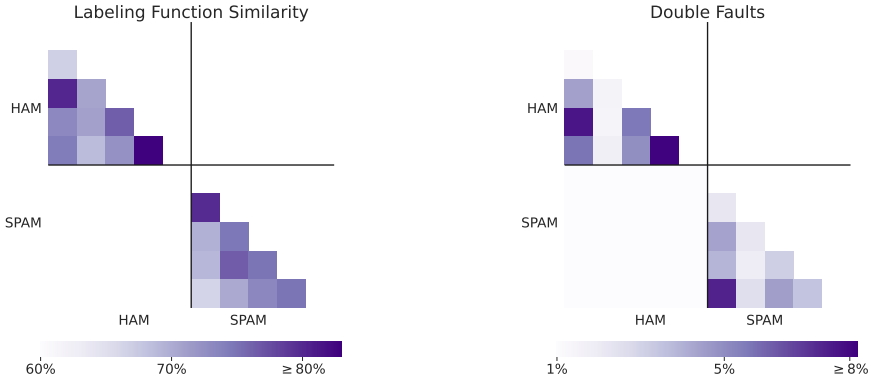

Figure 5.2: Visualization of the similarity matrix for labeling functions in the YouTube dataset compared with double faults, i.e., examples on which both labeling functions make a mistake.

In this subsection, we aim to show that LFs similarities in the embedding spaces are valid indicators for their correlations. Previous work (Bach et al., 2017b; Varma

et al., 2019b) characterizes LFs correlation structures based on their output, which can be sensitive to the data and can result in spurious correlations. We aim to learn more expressive correlations that are invariant to data. To achieve this, we focus on finding correlations intrinsic to the LFs. Here we consider each LF as a task and capture their correlations through their respective task embeddings. We use T0++ (Sanh et al., 2021) to embed each LFs with last layer embeddings to compute cosine similarities matrix. The heatmap visualization of the similarity matrix for the YouTube dataset is shown in Figure 5.2. As demonstrated by the figure, the similarity information has significant overlaps with the correlated errors among LFs, which can be used to remove the most correlated LFs and still achieve good performance. In addition, we construct a toy experiment to further exam our assumption. This experiment is based on the intuition that if two labeling functions (LFs) are correlated, removing one should not significantly harm the model's performance. Removing the most correlated LF leads to more accurate weights assigned to each LF, resulting in a more accurate end model. The results from our toy experiment align with the assumption. As shown in Table 5.3, after removing one of the most correlated LFs, the result either improved or slightly dropped below the baseline. This investigation further suggests that the similarity information in the latent embedding space can reveal information about the underlying correlations of LFs.

5.5.2 Role of Labeling Function Removal

In this subsection, we exam the contribution of Labeling function Removal (LaRe) in our Structure Refining Module. The core idea for LaRe is that when LFs are highly correlated and redundant, removing some can improve performance and efficiency. To extensively understand effect of LFs removal, we can set $m_e = 0$ for our Structure Refining Module to only use LaRe. We examined the impact of removing LFs based on correlation by conducting experiments where we removed 10%, 30%, 50%, and 70% of

Table 5.4: We documented the number of labeling functions removed as well as accuracy or F1 metric for each experiment. We report the mean and standard error with 5 random runs and the best results are indicated in bold. We highlight results that outperform PromptedWS with arrows. We also estimate the total number of token saved when running with Train/Test/Validation splits.

					Estimated # saved tokens		
		Performance(Acc/F1)	**#(LF removed)**	**# (prompts saved)**	**Train**	**Test**	**Validation**
Youtube	PromptedWS	94.8 ± 1.2	0	0	0	0	0
	removing 10%	**95.3 ± 0.3** ↑	1	1586	70900	11369	5186
	removing 30%	90.5±1.7	3	4758	212700	34107	15558
	removing 50%	87.5±1.9	5	7930	354500	56845	25930
	removing 70%	83.1±0.5	7	11102	496300	79583	36302
SMS	PromptedWS	90.0 ± 4.1	0	0	0	0	0
	removing 10%	**94.2± 1.4** ↑	7	31997	1343069	144984	147903
	removing 30%	93.5± 0.8 ↑	22	100562	4221074	455664	464838
	removing 50%	88.6±3.4	36	164556	6907212	745632	760644
	removing 70%	88.7±3.1	51	233121	9785217	1056312	1077579
Spouse	PromptedWS	52.1 ± 1.3	0	0	0	0	0
	removing 10%	53.0± 1.2 ↑	1	22254	1980962	234659	249159
	removing 30%	**58.5± 1.3** ↑	3	66762	5942886	703977	747477
	removing 50%	55.4±1.1 ↑	6	133524	11885772	1407954	1494954
	removing 70%	0±0	8	178032	-	-	-

labeling functions.

As shown in 5.3, using LaRe alone could improve the performance over vanilla PropmtedWS. Even though the original labeling functions used in PromptedWS are carefully selected to avoid correlation and facilitate weak supervision, remove 10% of the labeling functions (on YouTube and SMS) and remove 30% of the labeling functions (on Spouse) can still improve the performance, further suggesting that there still could be some redundancies with manually selected LFs. Note that our experiments are relatively coarse by setting the removal rate to be 10%, 30%, 50%, 70% for all the dataset because the total number of labeling functions varies from 10 to 146 for the six group experiments, so the effects of "removing 70%" could be different on different dataset based on the total number of LFs as well as the redundancies of the total LFs set. In practice, the number or portion of removal should be more fine-grained and tailored to the total number of labeling functions to achieve a better performance. We expect to remove redundant labeling functions without tuning hyperparameters on label model and end model, so we apply the same hyperparameters set used for PromptedWS

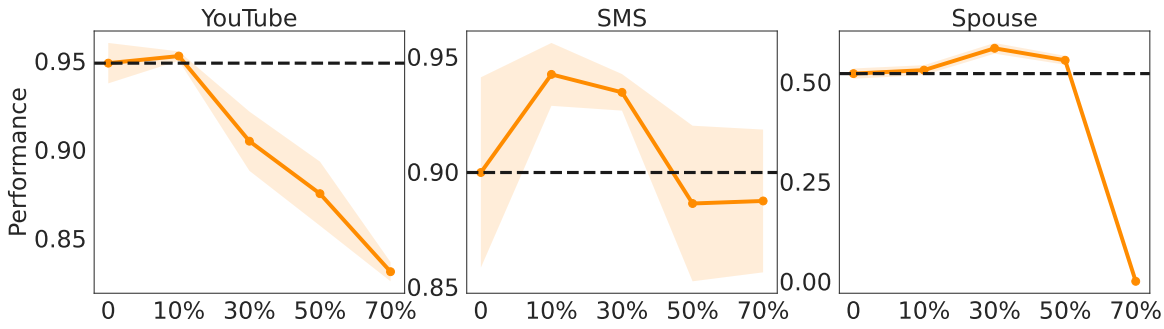

Figure 5.3: Performance on different removal rate. The dashed lines are the prompted weak supervisions without any removal. We plot the results of removing 10%, 30%, 50%, 70% of the labeling functions.

when doing our experiments on removal. In practice, as in our experiments, we use the validation split from WRENCH to evaluate, compare and choose the appropriate removal threshold. Overall, the experiments show that the LaRe in our Structure Refining Module can help improving the performance. These experiments demonstrate that the *removing* part in our refining module could help to improve the performance even in the worst case. Note that whether the performance improve or drop depends on the trade-off between the performance gain by handling the dependencies and the performance drop due to the reduced information from removing labeling functions, so the performance plot over removal isn't perfectly concave. Also, we should not only care about the removal threshold that achieves the best performance, but all the removal thresholds where the performance are better or equivalent to the PromptedWS (namely, the all the points above or near the dashed line in Figure 5.3), since these are points indicating an improved efficiency without negatively affect performance.

Performance and efficiency: trade-off or win-win? Inference with Large Language Models (LLMs) can be computationally expensive and time-consuming. One benefit of removing redundant LFs is efficiency: when the total number of labeling functions is reduced, we are feeding less tokens to the LLMs and therefore save more computations and time. When weighing performance and efficiency, it's important to consider the feasibility of removing LFs in terms of the effect on performance. However, as our

experiments show, it may not always be a trade-off; Sometimes it could be a win-win scenario with Structure Refining Module. As shown in Figure 5.3, for YouTube dataset, removing 10% of the labeling functions improves the performance while saving more computations. For SMS, removing 10% or 30% both improve performance. In the most extreme cases of removing 70% of the labeling functions, though the performance decreases 1.3 points, it reduces 51 labeling functions, and saved in total of 233121 prompts. For the Spouse dataset, removing 30% of LFs presents a win-win scenario, both improving performance and saving computation. We include the detailed statistics in table 5.4.

5.5.3 Effect of CosGen

Table 5.5: Comparison of CosGen, weak supervision structure learning(WSSL) and without structures.

	YouTube	**SMS**	**Spouse**
PromptedWS	**94.8**±1.2	90.0±4.1	52.1±1.3
PromptedWS + WSSL	93.0±1.3	**94.5**±1.7	63.9±0.9
PromptedWS + CosGen	94.2±0.5	93.4±1.2	**64.6**±0.5

Another core component of Structure Refining Module is Correlation Structure Generation (CosGen). The main output of CosGen is the estimated structure $\mathcal{E}$, which contains all the labeling function pairs that are considered as correlated and potentially dependent on each other. In this subsection, we compare the CosGen module of our Structure Refining Module with weak supervision structure learning (WSSL) (Varma et al., 2019b), a state-of-the-art structure learning for programmatic weak supervision that learns a sparse structure from data. We compare, evaluate and analyze both methods in terms of performance and efficiency. The performance comparisons are given in Table 5.5.

Table 5.5 shows that compared to PromptedWS without any consideration of correlations, passing the structure is extremely effective on Spouse dataset, improved 12.5 F1 score with CosGen and improved 11.8 F1 score using the WSSL. For SMS

dataset, passing the structure also improves the results, boosting 3.4 Acc score and 4.5 Acc score using CosGen and the structure from WSSL respectively. Although both the structure from our refining module and the structure learned from data help to deal with dependencies and can effectively improve the performance, CosGen is much more efficient than WSSL. The time and computational cost for CosGen is negligible while the time of learning structures using WSSL depends on the data and the number of LFs.

	YouTube	**SMS**	**Spouse**
time(s)	0.11	24.76	0.15

Table 5.6: WSSL runtime for inferring LFs structures.

As shown in Table 5.6, structure learning with WSSL takes around 0.1s for YouTube and Spouse dataset and around 24s to learn a structure of SMS for WSSL. Note that running a label model without any structures for SMS dataset only takes less than 1s. Compared to the label model run time, 24s for computing the structure alone should be considered as expensive and should not be ignored. The same holds for both YouTube and Spouse dataset: when using WSSL to learn a structure, the time spend on obtaining the structure could be longer than label model runtime. In contrast, CosGen produces a structure that can be as effective as WSSL, but with negligible computation time.

5.5.4 Handling high redundancy

Table 5.7: Comparisons with PromptedWS in augmented settings with redundent prompted LFs.

	YouTube(Acc)	**SMS(F1)**	**Spouse(F1)**
PromptedWS (Smith et al., 2022)	93.8±1.2	89.0±3.6	31.0±4.2
w/ Structure Refining Module	**94.9**±1.1	**95.0**±1.0	**58.9**±1.9

To empirically evaluate the performance of Structure Refining Module in high redundancy scenarios, we conducted additional experiments using an original aug-

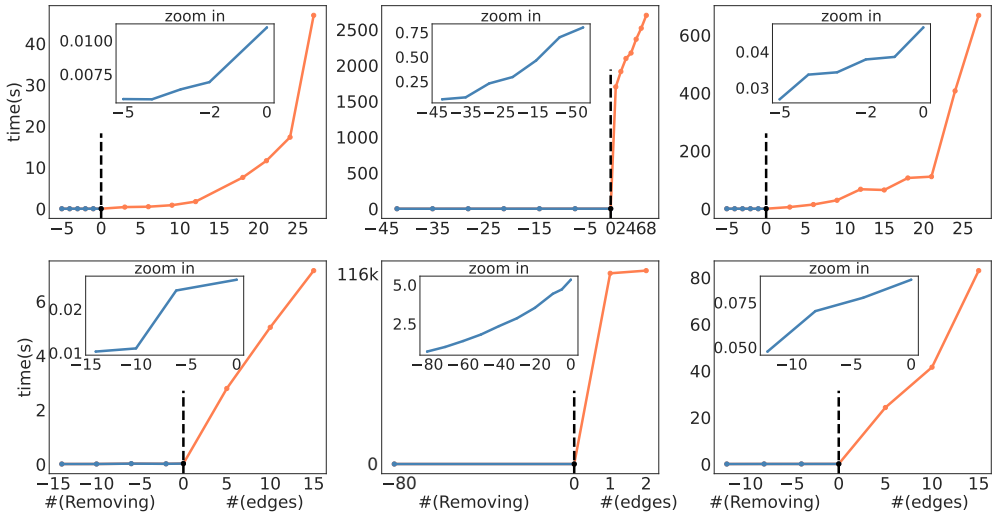

Figure 5.4: Label model running time for LaRe (left side of the dashed vertical line; plotted using blue) and CosGen (right side of dashed vertical line; plotted with orange). We zoom in the LaRe (blue plots) to better show the tendency in the subplots. From left to right columns are plots for Youtube, SMS and Spouse respectively. The first row describes experiments with original set of prompted LFs and the second row describes experiments with augmented LFs.

mentation strategy for prompted LFs. Using the same datasets, we translated the natural language prompts for each prompted LFs into a different language and then back into English. This process resulted in a new set of prompted LFs that are highly redundant with the original set. We then merged these augmented prompted LFs with the original set, resulting in a larger and intentionally highly redundant set of prompted LFs. The results of our experiments are shown in Table 5.7, which demonstrates that Structure Refining Module achieved a significant improvement of 11.7 points over the PromptedWS baselines. This result indicates the effectiveness of Structure Refining Module in high redundancy scenarios.

5.5.5 Effect on label model efficiency

In this subsection, we specifically exam how the computational time of the label model can change with Structure Refining Module. In Figure 5.4, we plot the label model run time for both removing the LFs and feeding dependency structures into label model. A zero value on the horizontal axis represents PromptedWS without LaRe

and CosGen, with negative values indicating the number of LFs removed and positive values indicating the number of edges. The vertical axis represents the run time of the label model. We use blue color to represent number of LFs removed while and use orange color to represent number of edges in the structures. We observe that pass in the structure generally harms the efficiency of the label model. Even passing a simple structure with just one edge can increase the computational time of the label model greatly. For example, for the SMS dataset, passing a structure with only one edge leads to a surge in the running time from less than 1 second to around 1700 seconds, while for the SMS dataset with augmented LFs, it surges from 5 seconds to around 116×10^3 seconds. Similarly, as the number of edges increases, the computational time of the label model also increases, as seen in the YouTube and Spouse datasets. This tendency has been shown in both the YouTube and Spouse dataset, where when the number of edges increases, the running time of the label model tends to increase. As for removing LFs, the running time saved on label model is not so remarkable unless the number of LFs are large. Since for dataset with a few LFs, the label model is already fast enough even without removing any LFs. (Less than 1s for all the three dataset even with augmented LFs except for SMS dataset). In Figure 5.4, we zoom in on the running time for LF removal to see how running time changes when removing different number of LFs. Overall, our analysis suggests that both removing LFs and increasing sparsity in structures can improve the efficiency of the label model. The choice should depend on the specific characteristics of the dataset and the trade-off between accuracy and efficiency.

5.5.6 Selecting hyperparameters

In this subsection, we provide some intuition on how to select m_r and m_e for Structure Refining Module. When selecting the hyperparameters for Structure Refining Module, it is crucial to consider the balance between performance and computational

costs. In scenarios with only a few LFs, occurrences of heavy redundancies among LFs also became rarer. So removing too many labeling functions could harm performance. In this case, the number to be removed m_r should be relatively small. Since there are not many LFs to remove, there is not much to gain in computational time and resources. Here, performance should be the primary consideration. When the total number of LFs is small, a relatively sparser dependency structure among LFs may boost performance without significantly increasing run time. Thus, in this case, m_e can be set to a non-zero value.

In contrast, if the number of LFs are large, there might be more substantial redundancies among LFs and thus removing some of them can improve the performance. In this case, We set m_r to be relatively large. Moreover, since LFs removal saves computational time on the label model as well as computational cost for LLM inferences (such as costs on prompting), we can trade some performance for efficiency by selecting a large m_r. We also prefer small m_e, the parameter controlling the number of edges of structure to relieve the computation cost.

5.6 Conclusion and Discussion

In this chapter, I introduce Structure Refining Module, a novel approach for efficient structure discovery and management in Prompted Weak Supervision. Our approach asks large language models for information beyond the inference output, leveraging similarities in the embedding space to detect redundant LFs and learn the dependency structures among prompted LFs. We demonstrated the effectiveness of our approach through extensive experiments and provide comprehensive ablation analysis for Structure Refining Module. We believe that the Structure Refining Module is a valuable tool for weak supervision practitioners who wish to optimize their workflow for both efficiency and performance. In future work, we plan to further investigate scalable and robust integration of LLMs into programmatic weak supervision workflows.

Ethical Considerations One major concern for our method is unfair labeling due to spurious and sensitive feedback from LLMs. As a result, supervised model using the data labeled by the proposed system may be subject to biased treatment in downstream applications. Therefore it is essential to consider the potential biased output from LLMs and subject matter experts should also conduct careful moderation with the development dataset to better identify and contain the biases from LLMs. Our proposed method aims to improve both the accuracy of probabilistic labeling and efficiency in structure estimation. However, our methods may be unfairly used in applications that are subject to data privacy violations and the consequent labels or supervised-trained models. It is crucial to apply careful scrutiny towards the detailed applications of our method. While our method has the potential to decrease the bias from LLMs, it is important to consider and carefully address these ethical challenges to ensure fair and responsible use.

5.7 Appendix: Prompted Labeling Functions

Dataset	Template	Label
YouTube	Does the following comment talk about a song?\n\n[TEXT]	HAM
	Is the following comment fewer than 5 words?\n\n [TEXT]	HAM
	Does the following comment mention a person's name?\n\n [TEXT]	HAM
	Does the following comment express a very strong sentiment?\n\n [TEXT]	HAM
	Does the following comment express a subjective opinion?\n\n [TEXT]	HAM
	Does the following comment reference the speaker's channel or video? \n\n [TEXT]	SPAM
	Does the following comment ask you to subscribe to a channel?\n\n [TEXT]	SPAM
	Does the following comment have a URL?\n \n[TEXT]	SPAM
	Does the following comment ask the reader to do something?\n\n [TEXT]	SPAM
	Does the following comment contain the words "check out"? \n\n [TEXT]	SPAM
Spouse	Context: [TEXT]\n \n Are [PERSON1] and [PERSON2] family members?	NOT SPOUSE
	Context: [TEXT]\n \n Is [PERSON1] said to be a family member?	NOT SPOUSE
	Context: [TEXT]\n \n Is [PERSON2] said to be a family member?	NOT SPOUSE
	Context: [TEXT]\n \n Are [PERSON1] and [PERSON2] dating?	NOT SPOUSE
	Context: [TEXT]\n \n Are [PERSON1] and [PERSON2] co-workers?	NOT SPOUSE
	Context: [TEXT]\n \n Is there any mention of "spouse" between the entities [PERSON1] and [PERSON2]?	SPOUSE
	Context: [TEXT]\n \n Is there any mention of "spouse" before the entity [PERSON1]?	SPOUSE
	Context: [TEXT]\n \n Is there any mention of "spouse" before the entity [PERSON2]?	SPOUSE
	Context: [TEXT]\n \n Do [PERSON1] and [PERSON2] have the same last name?	SPOUSE
	Context: [TEXT]\n \n Did [PERSON1] and [PERSON2] get married?	SPOUSE
	Context: [TEXT]\n \n Are [PERSON1] and [PERSON2] married?	SPOUSE

Template for SMS	Does the following text message contain the words "[KEYWORDS]"?\n \n [TEXT]	Label
[KEYWORDS]	??1.50, ??500, ??5000, call for offer, cash prize, chat date, chat to, childporn, credits, dating call, direct, expires now, fantasies call, free phones, free price, free ringtones, free sex, free tone, guaranteed free, guaranteed gift, hard live girl, important lucky, inviting friends, latest, latest offer, message call, new mobiles, no extra, password, please call, sms reply, unlimited calls, urgent award guaranteed, urgent prize, voucher claim, welcome reply, win shopping, winner reward, won call, won cash, won cash prize, won claim	SPAM
	I, I can did, I it, I miss, I used to, adventuring, amrita, can't talk, did u got, do you, fb, goodo, hee hee, i'll, jus, link, maggi, mine, my kids, noisy, praying, shit, should I, thanks, that's fine, thats nice, u how 2, we will, where are, wtf, your I	HAM

Table 5.8: Prompted Labeling Functions for YouTube and Spouse datasets. The YouTube dataset has class labels HAM and SPAM, and the dataset has class labels NOT SPOUSE and SPOUSE. The label map transforms the answer "yes" to the value denoted by the label column(either HAM or SPAM for YouTube dataset and either NOT SPOUSE or SPOUSE for spouse dataset) and we consider other answers as abstention. for the SMS dataset, The template asks whether there are certain keywords in the text and the label map transforms the answer "yes" to the value denoted by the label column (either HAM or SPAM) and we consider other answers as abstention. prompted Labeling Functions are from Smith et al. (2022).

CHAPTER 6

Alfred Apps and Agent Alfred: Towards Standardized Evaluation and Automated Application of Prompted Weak Supervision

This chapter addresses challenges in the application and evaluation of Prompted Weak Supervision (PWS). We first introduce **Alfred-Apps**, a standardized benchmark suite developed to facilitate the evaluation and comparison of PWS methods (Section 6.3). Subsequently, we detail **AgentAlfred**, a prototype system investigating the use of large language models (LLMs) to automate components of the PWS workflow based on natural language interaction (Section 6.4). We present experimental results evaluating PWS pipelines within this framework (Section 6.5), demonstrating how knowledge from large foundation models can be effectively transferred to significantly smaller end models, and discuss the implications (Section 6.6).

6.1 Introduction

The acquisition of large labeled datasets often presents a significant bottleneck in machine learning model development. Weak supervision (WS) provides an alternative by enabling model training using accessible, though potentially noisy, supervision sources, such as user-defined heuristics (Ratner et al., 2017; Zhang et al., 2022a). Recently, the capabilities of large language models (LLMs), often possessing billions of parameters, have spurred the development of Prompted Weak Supervision (PWS). In this paradigm, natural language prompts query LLMs for labels or generate labeling functions (LFs) (Smith et al., 2022; Yu and Bach, 2023). PWS utilizes the knowledge encoded in these large foundation models, allowing practitioners to translate domain expertise into supervision signals. This process can be viewed as a form of knowledge distillation, enabling the transfer of capabilities from billion-parameter scale models to train significantly smaller, more deployment-friendly end models (e.g., models with tens or hundreds of millions of parameters, such as the 86M parameter DeBERTa-V3-Base used in our experiments), potentially reducing the effort compared to implementing traditional code-based LFs.

However, the practical application and rigorous study of PWS face notable challenges. First, the rapid development of PWS techniques lacks a standardized evaluation framework. The diversity in prompting strategies, LLM backends, and aggregation methods makes rigorous comparison between studies difficult, hindering reproducible research and objective assessment of progress. Evaluating only raw zero-shot LLM performance is often insufficient, as effective PWS typically involves integrating multiple LFs and employing sophisticated aggregation techniques. Second, applying PWS effectively still requires considerable user expertise and effort. While constructing basic **zero-shot prompts** (requesting a label from the full class set $\mathcal{Y}$) is relatively simple, achieving high performance often necessitates designing more precise **heuristic prompts**. These encode specific rules or patterns, aiming for high precision on

relevant data subsets while potentially abstaining otherwise (output mapped to $\mathcal{Y} \cup \{\text{ABSTAIN}\}$). Developing, managing, and iterating upon a collection of diverse prompts requires familiarity with WS principles and workflows.

Existing WS frameworks, primarily developed for code-based LFs (Ratner et al., 2017), often lack direct support for prompt-based workflows. While recent PWS studies propose valuable methods (**??**), they frequently employ custom experimental setups, limiting direct comparability. Furthermore, although LLM agents demonstrate potential for automating complex tasks (**??**), their application to automating the end-to-end PWS workflow—from prompt generation guided by natural language intent to execution and management—has not been systematically investigated.

This work aims to address these limitations through two primary contributions. First, we introduce **Alfred-Apps**, a benchmark suite and software package designed to support standardized development and evaluation of PWS methods. It offers diverse tasks, consistent data formats, baseline prompt implementations, and evaluation metrics relevant to PWS. Second, we present **AgentAlfred**, a prototype LLM agent system built upon the Alfred-Apps framework. AgentAlfred interprets natural language commands to automate specific PWS workflow steps, such as generating candidate prompts and coordinating their execution, thereby exploring a means to reduce the manual effort involved. Our experiments utilize Alfred-Apps to evaluate PWS pipelines and investigate the effectiveness of LFs generated via AgentAlfred, comparing their performance to that of predefined LFs. Collectively, Alfred-Apps and AgentAlfred represent steps towards making PWS a more rigorous, reproducible, and systematically applicable technique for leveraging expert knowledge through foundation models, particularly for training smaller, efficient end models.

Summary of Contributions.

- We identify and address limitations concerning standardization and automation

in the PWS field.

- We introduce **Alfred-Apps** (Section 6.3), a standardized benchmark suite and software package providing tasks, metrics, and baselines for evaluating and developing PWS methods using LLMs.
- We present **AgentAlfred** (Section 6.4), a prototype LLM agent system investigating the automation of PWS workflow steps (e.g., prompt generation, execution) via natural language interaction, integrated with the Alfred-Apps framework.
- We provide experimental validation using Alfred-Apps (Section 6.5), evaluating the performance of structured PWS pipelines and assessing the quality of labeling functions generated automatically by AgentAlfred, demonstrating effective knowledge transfer to smaller end models.
- We discuss the implications of these components for knowledge transfer, particularly from large foundation models to smaller task-specific models, and outline potential directions for future research (Section 6.6).

6.2 Related Work

Our research builds upon and relates to several areas of prior work:

Weak Supervision (WS). Foundational WS methods focused on aggregating multiple noisy LFs, typically implemented as code, using label models to estimate true labels without extensive ground truth data (Dawid and Skene, 1979a; Ratner et al., 2016b, 2017; **?**). These established the viability of training models using programmatic heuristics. Our work leverages these principles but focuses on LFs derived from natural language prompts interacting with LLMs, addressing the unique aspects of this interaction.

Prompted Weak Supervision (PWS). Utilizing LLMs for supervision has seen increasing interest, encompassing direct zero-shot/few-shot labeling (**?**) and integration within structured WS frameworks (Smith et al., 2022; Yu and Bach, 2023). Research explores prompt engineering, explanation-based supervision, calibration, and active learning within the PWS context. When used to train a distinct end model, PWS can be seen as a sophisticated form of knowledge distillation. While these studies contribute specific PWS methods, our work provides supporting infrastructure (Alfred-Apps for standardization, AgentAlfred for investigating automation) intended to facilitate rigorous comparison and practical application of such methods.

Benchmarks for Data-Centric AI and WS. Benchmarks like DataPerf (**?**) and WRENCH (**?**) offer valuable resources for data-centric evaluation. However, they may not fully address the specific needs of LLM-centric PWS, such as evaluating prompt variations, managing different LLM backends, or analyzing prompt-based LF characteristics. Alfred-Apps is designed specifically to address these requirements, providing a platform tailored for PWS research.

LLM Agents for Automation. The development of LLM-powered agents capable of planning, tool use, and task execution is an active research area (**?????**), with applications emerging in software engineering (**?**) and scientific discovery (**?**). AgentAlfred applies the agent concept to the specific workflow of prompted weak supervision, investigating the automation of prompt generation, execution, and management within the structured Alfred-Apps environment.

6.3 Alfred-Apps: A Standardized Benchmark for PWS

Reproducible research in PWS necessitates standardized tasks, data formats, and evaluation protocols. Existing WS benchmarks may not fully cater to the specific requirements of prompt-based LFs and LLM integration. We developed **Alfred-Apps**

to address this limitation. It functions as both a benchmark suite for comparing PWS methods and a software package for developing PWS applications, extending the Alfred system (Yu and Bach, 2023).

Design Principles. Alfred-Apps was designed to support PWS research based on the following principles: Standardization, Task Diversity, Modularity, Comprehensive Evaluation, and Baselines (as detailed in the previous draft).

Benchmark Task Suite. Alfred-Apps v1.0 comprises the six tasks listed previously (MedNLI, ContractNLI, etc.). Detailed information on data sources, preprocessing steps, and data splits is provided with the software package.

Supported Supervision Types. The framework provides built-in support for common prompt-based LF types:

- **Zero-shot Prompt LFs:** Employ prompts that ask an LLM to directly label an input instance using the full label set (output in $\mathcal{Y}$). Default templates are included.

  Example (Toxicity Detection Task, Labels: toxic, non-toxic):

  ```
  Input Text: [[text]]
  ---
  Question: Is the above text toxic?
  Instruction: Classify the text as either toxic or non-toxic.
  Choose from: toxic, non-toxic
  Answer:
  ```

- **Heuristic Prompt LFs:** Utilize prompts that encode specific rules or patterns, typically designed for high precision, outputting a target label or abstaining (output mapped to $\mathcal{Y} \cup \{\text{ABSTAIN}\}$). User-defined heuristics can be integrated

via configuration.

Example (Toxicity Detection Task, Heuristic: presence of insults):

```
Input Text: [[text]]

---

Definition: The text contains direct insults or derogatory terms
            targeting a person or group
            (e.g., 'idiot', 'stupid', 'loser').
Instruction: Carefully review the input based on the definition above.
                Does the input clearly meet the definition?
                Answer ONLY with "Yes", "No", or "Unsure".
Answer (Yes/No/Unsure):
```

(Here, 'Yes' would typically map to the 'toxic' label, while 'No' and 'Unsure' map to ABSTAIN).

The architecture allows for extension to other LF types.

Use Cases. Alfred-Apps serves primarily as: (1) An evaluation platform enabling comparison of PWS techniques based on performance across standardized tasks and metrics. (2) A foundational software layer providing programmatic access to tasks, data handling, and LF execution, suitable for building higher-level Prompted Weak Supervision tools like AgentAlfred.

6.4 AgentAlfred: Investigating PWS Workflow Automation

The typical PWS workflow involves iterative steps: defining LFs (prompts), generating labels, analyzing LF properties (e.g., agreement, coverage), and refining LFs. This

process can require significant manual effort and expertise. **AgentAlfred** explores the potential of using an LLM agent to automate parts of this workflow based on natural language directives.

Goal. The objective is to investigate whether an LLM agent can assist users, particularly domain experts potentially less familiar with WS intricacies, in navigating the PWS process. Users interact via natural language requests (e.g., "Generate 5 heuristic prompts for Toxic-Chat focusing on insults"), which AgentAlfred translates into operations within the Alfred-Apps environment.

Architecture Overview. AgentAlfred utilizes an LLM to parse user requests and orchestrate interactions with the Alfred-Apps backend. Its components include: A Natural Language Interface, Planner, LLM Interaction Module, Alfred-Apps Executor, and State Manager (Registry/Project Context). The system functions as an interface layer, attempting to automate execution details based on higher-level user intent.

6.5 Experiments

We conduct experiments to evaluate PWS pipelines implemented using Alfred-Apps and to assess the performance of LFs generated by AgentAlfred.

Setup.

- **Tasks:** The six text classification tasks included in Alfred-Apps v1.0.
- **Agent LLM Backend:** Qwen2.5-32B-Instruct (Qwen Team, 2024) serves as the LLM for AgentAlfred's internal processing.
- **LF LLM Backends:** Various instruction-tuned models (often billions of parameters) are used for executing the generated LFs, including Mistral-7B, Llama-3.1-8B, Llama-3.2-1B, Gemma-2-9B, Phi-3-Mini, Qwen2.5-0.5B, Phi-4, and Qwen2.5-

7B (Jiang et al., 2023; Meta, 2024a,b; Gemma Team, 2024; Microsoft, 2024; Qwen Team, 2024). Models accessed via transformers Wolf et al. (2020) package provided by Huggingface. Inference conducted using vLLM. Kwon et al. (2023)

- **End Model:** DeBERTa-V3-Base (He et al., 2020), a transformer encoder model with approximately 86 million parameters, trained on probabilistic labels produced by the label model aggregating LF outputs. This highlights the significant parameter reduction from the LF backends (Often Billions) to the final task-specific model (Often Millions).
- **Compared Settings:** We evaluate average Accuracy and Macro F1 across the six tasks. The settings are:
    1. **Z-S Test Labeler:** Direct zero-shot prediction by the LLM on the test set $\mathcal{D}_{test}$.
    2. **Z-S Distilled:** End model trained on zero-shot LLM predictions generated on the unlabeled set $\mathcal{D}_U$.
    3. **PWS-Full (Default LFs):** End model trained using labels aggregated by a Naive Bayes label model (Ratner et al., 2016b) from multiple predefined (zero-shot + heuristic) LFs provided within Alfred-Apps, applied to $\mathcal{D}_U$.
    4. **PWS-Full (AgentAlfred LFs):** The same PWS pipeline as above, but utilizing LFs generated by AgentAlfred in response to basic instructions (e.g., requesting 3 zero-shot and 5 heuristic LFs for a given task).

Results: PWS Pipeline Effectiveness. Table 6.1 shows the performance benefits of the PWS pipeline compared to simpler baselines. The PWS-Full setting, which aggregates multiple default zero-shot and heuristic LFs using a label model, generally yields the highest performance, demonstrating the value of structured aggregation and denoising for knowledge transfer.

Table 6.1: Effectiveness of PWS Pipeline Components. Average performance (Accuracy % and Macro F1) across six Alfred-Apps tasks, comparing direct Zero-Shot inference (Z-S Test Labeler), Zero-Shot Distillation (Z-S Distilled), and the full PWS pipeline using default LFs (PWS-Full). Best performance per model is typically in the PWS-Full columns.

LF Backend Model	Z-S Test Labeler		Z-S Distilled		PWS-Full (Default LFs)	
	ACC	F1	ACC	F1	ACC	F1
Phi-3.5-mini-instruct	65.81	44.55	65.41	43.26	**70.77**	**47.24**
Phi4	66.18	42.70	0.00*	0.00*	**72.56**	**48.12**
Qwen2.5-0.5B-Instruct	41.95	25.45	41.96	24.52	**48.66**	**30.25**
gemma-2-9b-it	61.49	40.94	61.76	40.60	**67.14**	**47.14**
llama-3.1-8B-it	63.41	41.54	54.14	33.46	**67.14**	**43.25**
llama-3.2-1B-it	54.28	27.98	54.97	**30.38**	**57.19**	30.03
mistral-inst-v0.2-7b	62.70	46.08	62.05	46.03	**64.24**	**46.88**

End model for Distilled and PWS-Full is DeBERTa-V3-Base (86M parameters).
*Z-S Distilled results for Phi4 are pending

Results: AgentAlfred LF Generation. Table 6.2 compares the PWS-Full pipeline performance using the default, predefined LFs from Alfred-Apps versus using LFs automatically generated by AgentAlfred. This comparison assesses the quality of the agent-generated supervision.

Analysis.

- **Effectiveness of PWS-Full Pipeline and Knowledge Transfer:** As shown in Table 6.1, aggregating multiple LFs using a label model (PWS-Full setting) generally leads to improved end-model performance compared to direct zero-shot prediction or simple distillation. This supports the value of integrating diverse knowledge sources and applying denoising. Notably, this process effectively transfers knowledge from billion-parameter LF backend LLMs to a much smaller 86M parameter DeBERTa-V3-Base end model. The resulting performance often exceeds that of the direct zero-shot application of the larger LLMs, highlighting PWS as an effective distillation technique that incorporates structured supervision.
- **Limitations of Simpler Baselines:** Table 6.1 also shows that direct zero-shot

Table 6.2: Comparison of PWS-Full Performance using Default LFs vs. AgentAlfred-Generated LFs. Average performance (Accuracy % and Macro F1) across six Alfred-Apps tasks. Bold indicates the better result for each model/metric pair between Default and Agent.

LF Backend Model	PWS-Full (Default LFs)		PWS-Full (AgentAlfred LFs)	
	ACC	F1	ACC	F1
Phi-3.5-mini-instruct	70.77	47.24	**71.01**	**47.90**
Phi4	**72.56**	**48.12**	69.90	47.78
Qwen2.5-0.5B-Instruct	48.66	30.25	**50.57**	**30.68**
gemma-2-9b-it	**67.14**	**47.14**	66.12	45.55
llama-3.1-8B-it	67.14	43.25	**68.36**	**44.97**
llama-3.2-1B-it	57.19	**30.03**	**57.71**	29.31
mistral-inst-v0.2-7b	64.24	46.88	**64.49**	**47.76**
Qwen2.5-7B-Instruct	—	—	**71.16**	**48.07**

End model for all settings is DeBERTa-V3-Base (86M parameters).
— Results for Qwen2.5-7B-Instruct pending

performance exhibits considerable variance, and simple distillation (Z-S Distilled) does not consistently improve results, likely due to noise propagation. This underscores the potential benefit of the more structured aggregation in PWS-Full.

- **Performance of Agent-Generated LFs:** Comparing the PWS-Full columns in Table 6.2, the end-model performance achieved using LFs generated by AgentAlfred is comparable to, and in several cases slightly exceeds, the performance using the predefined Default LFs. This suggests that the investigated automated LF generation approach can produce supervision sources of competitive quality with reduced manual specification, demonstrating promise for automating PWS workflows. Performance variations (e.g., Phi4, Gemma-2) might indicate the presence of particularly effective or complex heuristics in the default set that the current agent generation strategy does not consistently replicate, suggesting avenues for future improvement in automated prompt generation techniques.

6.6 Discussion

This work contributes towards establishing more systematic and accessible approaches for PWS. Alfred-Apps offers a standardized platform for evaluation, while AgentAlfred represents an exploration into automating aspects of the PWS workflow using LLM agents.

Alfred-Apps: Facilitating Rigorous PWS Evaluation. By providing standardized tasks, metrics, data splits, and baseline implementations, Alfred-Apps aims to enable researchers to more reliably measure and compare the effectiveness of different PWS techniques. Such standardization is valuable for fostering reproducible research and tracking progress in the field.

AgentAlfred: Exploring Automation in PWS. AgentAlfred demonstrates the potential for LLM agents to simplify parts of the PWS process. By translating natural language requests into actions like prompt generation and LF execution, it could lower the technical threshold for domain experts. Our experimental results indicate that its automated LF generation capability can produce effective supervision sources.

PWS as Structured Knowledge Distillation. The experimental success of the PWS-Full setting, where large LLMs provide supervision to train a much smaller end model (DeBERTa-V3-Base 86M parameters), emphasizes the role of PWS as a powerful knowledge distillation technique. It allows leveraging the capabilities of billion-parameter foundation models to create efficient, high-performing task-specific models. Importantly, the structured combination of diverse LFs (zero-shot + heuristic) and label model aggregation often allows the distilled model to achieve better performance than could be obtained by simply fine-tuning on the raw zero-shot outputs of the teacher LLM (as seen by comparing PWS-Full to Z-S Distilled).

Limitations and Future Directions. AgentAlfred currently functions as a prototype primarily focused on executing predefined automation steps. Future research could explore more adaptive and collaborative agent behaviors: Automated LF Analysis, Intelligent Suggestion Generation, Interactive Refinement Loop, and Structure-Aware Prompt Management. These advancements could lead to agents that more actively assist users in developing and optimizing PWS strategies.

6.7 Conclusion

In this chapter, I introduced Alfred-Apps, a benchmark suite intended to standardize the evaluation and development of prompted weak supervision methods, and AgentAlfred, a prototype system investigating the use of LLM agents to automate PWS workflow components. Alfred-Apps provides the necessary tasks, baselines, and structure to facilitate reproducible research in PWS. AgentAlfred demonstrates the potential for lowering the barrier to PWS application by translating natural language commands into workflow actions, including the automated generation of effective labeling functions. Our experiments highlight the effectiveness of PWS pipelines, which combine diverse prompt-based LFs and label model aggregation, as a method for knowledge distillation from large foundation models (billions of parameters) to smaller, efficient end models (86M parameters). Furthermore, our results indicate that agent-based automation can yield LFs of competitive quality. These contributions aim to advance PWS as a more practical and rigorous technique for knowledge transfer in machine learning using foundation models.

6.8 Appendix I: Example AgentAlfred Interaction Workflow

This section illustrates a typical workflow interacting with AgentAlfred using natural language commands for a hypothetical sentiment analysis task. The explanatory text in parentheses describes the internal actions AgentAlfred takes.

6.8.1 Project Setup

User:

Let's create a project for sentiment analysis. The task is 'Classify customer review sentiment' with labels positive, negative, and neutral, using the 'text' field for the review. Call it 'sentiment_reviews'.

System Response:

(AgentAlfred identifies the 'create_project' action and extracts parameters: id='sentiment_reviews', task='Classify customer review sentiment', labels=['positive', 'negative', 'neutral'], fields='text': 'Input text')

```
[Success] Project 'sentiment_reviews' created and activated.
```

6.8.2 Heuristic Discovery

User:

Could you discover 3 heuristics that might indicate sentiment?

System Response:

(AgentAlfred identifies the 'discover_heuristics' action with num=3. It calls HeuristicService, which uses a prompt template with the current project context. The service parses the LLM's response, creates Heuristic objects, and saves them to the registry.)

`[Success] Discovered and saved 3 heuristics.`

`(System might optionally display a summary table of the discovered heuristics).`

6.8.3 Zero-Shot Labeler Generation

User:

Now, create 2 general zero-shot labelers for classifying sentiment.

System Response:

(AgentAlfred identifies the 'create_zsl' action with num=2. It calls LabelerService, which uses a ZSL Generation Prompt. The service parses the response, validates output labels, creates Labeler objects, and saves them.)

`[Success] Generated and saved 2 zero-shot labelers.`

`(System might display a summary table of the new labelers).`

6.8.4 Heuristic-Based Labeler Generation

User:

Okay, generate heuristic labelers based on the active heuristics.

System Response:

(AgentAlfred identifies the 'create_hl' action. It retrieves active heuristics, then for each, calls LabelerService which uses the HL Generation Prompt. The service parses responses, creates Labeler objects linked to heuristics, and saves them.)

`[Success] Generated and saved 3 heuristic labelers based on 3 active heuristics.`

`(System might display a summary table of the new heuristic labelers).`

6.8.5 Listing Artifacts

User:

Show me the list of active labelers for this project.

System Response:

(AgentAlfred identifies the 'list_labelers' action. It queries the registry for active labelers matching the current project.)

`[Success] Found 5 labelers for project 'sentiment_reviews'.`
`(System displays a formatted table showing the 2 ZSL and 3 HL labelers).`

6.8.6 Labeler Refinement

User:

Refine labeler 'l_zsl_sentiment_reviews_167...' with feedback: 'Make the prompt explicitly ask to consider sarcasm'.

System Response:

(AgentAlfred identifies the 'refine_labeler' action, extracting ID and feedback. It calls LabelerService.refine_labeler. The service uses the Template Refinement Prompt. The service updates the labeler object in the registry.)

`[Success] Labeler 'l_zsl_sentiment_reviews_167...' refined and saved.`
`(System might display the updated labeler details or the LLM's explanation).`

6.8.7 Exporting

User:

Export the active labelers to a CSV file.

System Response:

(AgentAlfred identifies the 'export' action. It retrieves active labelers, formats them using exporter.export_labelers_to_csv, and saves the file.)

```
[Success] Exported 5 labelers to
/workspace/exports/sentiment_reviews_classify_customer_review_sentiment_labelers
Data: 'path': '/path/to/workspace/exports/...'
```

6.9 Appendix II: AgentAlfred Prompt Templates

This appendix details the core prompt templates utilized by AgentAlfred's service modules (`NLCommandParser`, `HeuristicService`, `LabelerService`) to interact with foundation models (LLMs) for interpreting commands, generating supervision sources, and refining prompts.

6.9.1 Natural Language Command Parsing Prompt

This prompt guides the LLM integrated within the `NLCommandParser` to interpret user input in natural language and map it to a structured agent action with corresponding parameters.

Prompt template for parsing natural language commands

System: You are an intelligent command parser for the AgentAlfred system. Your task is to interpret the user's natural language command and map it to one of the known structured agent actions, extracting all relevant parameters.

User Command: "user_command"

Known Agent Actions: known_actions

Parameter Reference: - create_project: requires 'id' (string), 'task' (string), 'labels' (list of strings), optional 'fields' (dict as JSON string). - load_project: requires 'id' (string). - list_projects: no parameters. - project_info: no parameters. - update_project: optional 'task' (string), 'labels' (list of strings), 'fields' (dict as JSON string). - update_project_context_nl: requires 'addition' (string - the text to add to the task description). - discover_heuristics: optional 'num' (int, default 5). - list_heuristics: optional 'all' (bool, default false). - activate_heuristic: requires 'id' (string). - deactivate_heuristic: requires 'id' (string). - create_zsl: optional 'num' (int, default 3). - create_hl: no parameters. - list_labelers: optional 'type' (string: 'zero_shot' or 'heuristic'), optional 'all' (bool, default false). - refine_labeler: requires 'id' (string), 'feedback' (string). - activate_labeler: requires 'id' (string). - deactivate_labeler: requires 'id' (string). - export: optional 'format' (string: 'csv' or 'python', default 'csv'), optional 'path' (string), optional 'all' (bool, default false). - show_template: requires 'id' (string) OR 'ids' (comma-separated string list). Extract *all* mentioned template IDs.

Instructions: 1. Analyze the user command to determine the most likely intended **Agent Action** from the known list. 2. Extract all parameters mentioned or implied in the command that are relevant to the identified action. 3. Format the output ONLY as XML like the example below.

Continue: Prompt template for parsing natural language commands

4. If the command is unclear or doesn't map well to a known action, use the action "unknown". 5. For boolean parameters like 'all', output 'true' or 'false'. 6. For list parameters like 'labels' or 'ids', output comma-separated strings within the value tag. 7. For dictionary parameters like 'fields', output a valid JSON string within the value tag. 8. For 'show_template', if only one ID is mentioned, use '<param name="id">'. If multiple are mentioned (e.g., "show templates X and Y"), use '<param name="ids">' with a comma-separated list.

Output Format (XML): <parsed_command> <action>identified_action_name</action> <parameters> <param name="param_name_1"><value>extracted_value_1</value></param> <param name="param_name_2"><value>extracted_value_2</value></param> <!– Add more <param> blocks as needed –> </parameters> </parsed_command>

6.9.2 Zero-Shot Labeler (ZSL) Generation Prompt

This prompt is used by the `LabelerService` to request the LLM to generate multiple diverse zero-shot prompt templates for a given classification task, based on its description, label space, and input fields.

Prompt template for generating zero-shot labeler prompts

System: You are an expert prompt designer for LLM-based text classification, aiming for compatibility with weak supervision systems. Instruction: Design num_templates diverse zero-shot classification prompts for the task below. For each prompt: 1. Provide a descriptive snake_case name (e.g., ‘zsl_entailment_check‘). 2. Structure the ‘prompt_text‘ using ONLY the placeholders provided in the ‘Input Fields‘ section below (e.g., [[text]], [[question]]). Use a clear Question/Instruction format suitable for the task. End the prompt_text with "Answer: ". 3. Define the ‘output_labels‘ the LLM should choose from. These labels MUST be selected ONLY from the Project Label Space provided. Use comma-separated values. 4. Try your best to elaborate on the ‘prompt_text‘ to ensure it is clear and unambiguous, guiding the LLM to select from the specified ‘output_labels‘.

Task Description: task_description Project Label Space: label_space_str (e.g., "Entailment, Neutral, Contradiction") Input Fields: The actual fields for the current project will be listed here field_definitions

Refinement Strategy: Ensure prompts are clear, unambiguous, and guide the LLM to select only from the specified ‘output_labels‘. The template structure MUST correctly use the provided ‘Input Fields‘.

Output Format (XML): <templates> <template> <name>zsl_template_name_1</name> <prompt_text><![CDATA[[Optional preamble explaining the input fields based on their descriptions] Input Field Description 1: [[placeholder_for_field_1_from_above]] Input Field Description 2: [[placeholder_for_field_2_from_above]] — Question: [Your clear question related to the task, using the actual field placeholders like [[text]] or [[hypothesis]]] Instruction: [Optional: Specific instructions for the LLM] Choose from: [label_A, label_B] Example labels Answer:]]></prompt_text> <output_labels>label_A, label_B</output_labels> Example labels the LLM should output for this specific template </template> <!– Add more <template> blocks as needed –> </templates>

6.9.3 Heuristic Discovery Prompt

This prompt is employed by the `HeuristicService` to generate potential heuristics (labeling rules or patterns) relevant to the specified task description and label space.

Prompt template for discovering relevant heuristics

System: You are an expert at identifying useful heuristics for text classification.
Instruction: Generate num_heuristics diverse heuristics for the following task. For each heuristic: 1. Provide a descriptive name (use underscores). 2. Describe the pattern or indicator to look for. 3. Specify the primary project label it indicates. 4. Optionally list caveats or edge cases (semicolon-separated if multiple).
Task Description: task_description Project Label Space: label_space_str
Output Format (XML): <heuristics> <heuristic> <name>heuristic_name_1</name> <description>Pattern description 1</description> <primary_label>project_label_A</primary_label> <caveats>Caveat 1; Caveat 2</caveats> </heuristic> <!– Add more <heuristic> blocks –> </heuristics>

6.9.4 Heuristic-Based Labeler (HL) Generation Prompt

Used by the `LabelerService`, this prompt takes a specific heuristic (discovered previously) and asks the LLM to formulate a precise prompt template that verifies the presence of that heuristic's pattern in the input, outputting "Yes", "No", or "Unsure".

Prompt template for converting a heuristic into a verification prompt

System: You are an expert at converting natural language heuristics into precise LLM prompts for high-precision labeling, suitable for weak supervision systems. Instruction: For the given heuristic, design a prompt template asking the LLM to verify if the heuristic's pattern is present in the input. The LLM should answer ONLY with "Yes", "No", or "Unsure". The template MUST: 1. Use ONLY the placeholders provided in the 'Input Fields' section below (e.g., [[text]], [[premise]]). 2. Clearly state the specific question derived from the heuristic description. 3. Instruct the LLM to answer "Yes" ONLY if the pattern is clearly present, otherwise "No" or "Unsure". 4. End the 'prompt_text' with "Answer: ".

Heuristic Name: heuristic_name Heuristic Description: heuristic_description Task Description: task_description (Context for the overall goal) Input Fields: The actual fields for the current project will be listed here field_definitions

Refinement Strategy: The prompt must be highly specific to the heuristic pattern and correctly use the provided 'Input Fields'. Use precise language.

Output Format (XML):

<template> <name>hl_heuristic_name_underscores_template</name> <prompt_text><![CDATA[[Optional preamble explaining the input fields based on their descriptions] Input Field Description 1: [[placeholder_for_field_1_from_above]] Input Field Description 2: [[placeholder_for_field_2_from_above]] — Question: [Your precise question based *directly* on the heuristic description, ensuring it refers to the input using the actual field placeholders like [[text]]]? Instruction: Analyze the input based *only* on the question above. Does the input *clearly* match the condition described in the question? Answer ONLY with "Yes", "No", or "Unsure". Answer:]]></prompt_text> </template>

6.9.5 Template Refinement Prompt

This prompt facilitates the iterative improvement of existing labeler prompts (`LabelerService.refine_labeler`). It provides the LLM with the original prompt, task context, and specific user feedback, requesting a revised prompt and an explanation of the changes.

Prompt template for refining an existing labeler prompt based on user feedback

System: You are an expert prompt engineer specializing in refining prompts for clarity, robustness, and accuracy in weak supervision workflows. Instruction: Refine the following labeling prompt template based on the provided feedback and context. Incorporate the feedback to make the prompt clearer, handle edge cases better, or improve its alignment with the task goal. Crucially, ensure the 'refined_prompt_text' **continues to use the correct input field placeholders** (e.g., [[text]], [[question]]) as defined by the project's 'Input Fields'. Do NOT introduce incorrect placeholders like [[field_name_1]].

Original Labeler Details: - Name: labeler_name - Type: labeler_type (zero_shot or heuristic) - Task: task_description - Project Fields (Input Fields): The actual fields used in the original template fields_str - Current Output Labels: output_labels_str

Current Prompt Template: original_template_text

User Feedback for Refinement: user_feedback

Refinement Goals: Apply the feedback to improve the prompt's effectiveness. Maintain the use of the correct [[field_name]] placeholders from the 'Input Fields'. For zero-shot, ensure the output labels remain consistent unless feedback specifically requests changes (which should be handled carefully). For heuristic, the output must remain Yes/No/Unsure.

Output Format (XML): <refined_template> <explanation><![CDATA[Explanation of the changes made based on the feedback.]]></explanation> <refined_prompt_text><![CDATA[[[The complete refined prompt text, incorporating feedback AND using the correct field placeholders like [[text]] or [[question]]]] Answer:]]></refined_prompt_text> </refined_template>

6.10 Appendix III: AgentAlfred Interaction Prompts for Alfred-Apps Tasks

This appendix lists the sequences of natural language commands used to instruct AgentAlfred for generating labeling functions (LFs) for the different tasks within the Alfred-Apps benchmark suite during our experiments. These sequences simulate a user interaction aimed at creating both general zero-shot LFs and more specific heuristic-based LFs.

6.10.1 MedNLI (Medical Diagnosis Entailment)

Task Goal: Determine if a potential medical diagnosis logically follows from given medical observations.

1. Create a project for checking medical diagnosis entailment. Call it 'med_nli_checker'. Labels are entailment, contradiction, neutral. Use 'premise' for observations and 'hypothesis' for the diagnosis.
2. Discover 6 heuristics useful for medical reasoning, like symptom-diagnosis links.
3. Generate 3 zero-shot labelers that focus on medical logic and entailment.
4. Create heuristic labelers using the medical patterns found.
5. What's the current project info again?
6. Export the labelers to medical_nli_labelers.csv.

6.10.2 ContractNLI (Contract Clause Entailment)

Task Goal: Verify if a hypothesis about a contract clause accurately reflects the premise of the clause.

1. New project: 'Analyze contract clause entailment'. ID: contract_inferencer. La-

bels: entailment, contradiction, neutral. Fields: 'hypothesis' for the interpretation, 'premise' for the clause and statement.

2. Find 6 heuristics relevant to contract language, like obligations or definitions.
3. Create 3 zero-shot labelers for checking contract interpretation.
4. Generate the heuristic labelers using the contract patterns.
5. Show me the list of heuristics discovered for this project.
6. Export the labelers to contract_analysis_labelers.csv.

6.10.3 Toxic-Chat (Toxicity Detection)

Task Goal: Identify harmful, abusive, or toxic language in online comments.

1. I need a project to detect toxic online comments. Let's call it 'toxicity_detector'. The labels are simply toxic and non-toxic, using the 'text' field.
2. Can you find 6 heuristics useful for spotting toxic language?
3. Generate 3 zero-shot prompts that could identify toxicity.
4. Use the heuristics we discovered to create specific heuristic labelers.
5. List all the labelers we have, even the inactive ones.
6. Export the labelers list as a Python file to toxicity_labelers.py, please.

6.10.4 PrivacyPolicyQA (Privacy Policy Question Answering)

Task Goal: Assess relevance of policy segments to user questions (simplified to Yes/No in this example interaction).

1. Start a project for privacy policy question answering. ID: privacy_helper. Labels: yes, no. Fields are 'policy' for the text and 'question' for the user query.

2. Discover 6 heuristics for finding relevant info in privacy policies, like permission phrasing.
3. Create 3 zero-shot labelers to answer questions based on the policy.
4. Make labelers from the privacy policy heuristics.
5. List only the zero-shot labelers.
6. Save the labelers to privacy_qa_labelers.csv.

6.10.5 Emotions (Emotion Classification)

Task Goal: Recognize specific emotions (joy, sadness, anger, etc.) expressed in text.

1. New project: id=emotion_classifier, task is 'Classify emotion in social media posts', labels are joy, sadness, anger, fear, surprise, disgust. The input field is 'text'.
2. Discover 6 heuristics related to how emotions are expressed.
3. Create 4 zero-shot labelers for finding emotions in text.
4. Turn the heuristics you found into heuristic labelers.
5. Can you show me the current project information?
6. Export the active labelers to emotion_labelers.csv.

6.10.6 EnvironmentalClaims (Environmental Claim Detection)

Task Goal: Identify whether sentences from corporate reports contain claims related to environmental actions or policies.

1. Create a project called 'env_claim_detector'. The task is 'Classify sentences from corporate reports for environmental claims'. The labels are 'claim' and 'no_claim', and the input field is 'text'.

2. Discover 6 heuristics that might indicate an environmental claim in corporate text, like mentioning emissions, sustainability goals, or renewable energy.
3. Generate 3 zero-shot labelers for classifying environmental claims.
4. Create heuristic labelers based on the environmental claim patterns discovered.
5. Show me the list of active labelers for this project.
6. Export the labelers to a CSV file named env_claim_labelers.csv.

CHAPTER 7

Conclusion

The effective integration of human expertise into machine learning systems constitutes a persistent grand challenge in artificial intelligence. While deep learning has achieved remarkable success, its reliance on vast, meticulously labeled datasets creates significant bottlenecks, particularly in specialized domains where expert knowledge is paramount and labeled data is scarce, expensive, or inherently ambiguous **??**. Programmatic Weak Supervision (PWS) offers a compelling paradigm to mitigate this dependency by synthesizing training labels from higher-level, potentially noisy heuristics provided by domain experts. However, traditional PWS often necessitates translating nuanced domain knowledge into rigid, code-based labeling functions (LFs), limiting both accessibility for non-programmers and the expressivity of the captured knowledge.

The advent of large-scale foundation models, pretrained on diverse data modalities and exhibiting sophisticated language understanding, generation, and reasoning capabilities **?**, presents a transformative opportunity. These models enable a paradigm shift in how expert knowledge can be elicited and operationalized—moving from code to natural language prompts, thereby lowering technical barriers and allowing for richer, more flexible forms of supervision. This dissertation has explored this critical

intersection, developing a framework that synergizes the principled noise modeling of weak supervision with the expressive power and accessibility of foundation models. Our central thesis is that this synergy empowers domain experts to articulate their knowledge more naturally and effectively, leading to more efficient and robust machine learning workflows.

Our contributions advance this vision along three key axes. First, we enhanced the foundational theory of weak supervision itself. Recognizing the limitations of single-label LF outputs, **Chapter 2** introduced the *Noisy Partial Label Model (NPLM)*, a more expressive probabilistic generative model. NPLM permits LFs to output label subsets, thereby capturing inherent expert uncertainty and enabling the use of more nuanced knowledge sources, supported by theoretical identifiability guarantees and empirical validation **?**.

Second, acknowledging that leveraging foundation models requires efficient adaptation strategies, we developed novel parameter-efficient techniques. **Chapter 3** introduced *Compositional Soft Prompting (CSP)*, enabling zero-shot compositional generalization in vision-language models by learning discrete attribute and object prompt tokens, thus avoiding the need for exhaustive paired data **?**. Furthermore, **Chapter ??** presented *Neural Selective Fine-Tuning (NSFT)*, an approach that optimizes downstream performance and inference efficiency by selectively fine-tuning minimal submodule parameters within transformers, crucial for practical deployment **?**.

Third, and central to our thesis, we directly integrated foundation models into the weak supervision pipeline. **Chapter 4** introduced **Alfred**, a system demonstrating the feasibility and utility of *prompted weak supervision*, allowing experts to define LFs using natural language **?**. Building upon this, **Chapter 5** presented a structure-refining module that leverages foundation model embeddings to automatically infer dependencies between prompted LFs, enhancing the accuracy of the final label aggregation **?**. Finally, to catalyze further research and development, **Chapter 6** introduced the *Alfred-Apps*

benchmark suite and *Agent Alfred*, a prototype agentic system showcasing the potential for LLMs to automate aspects of the prompted supervision workflow itself **?**.

Collectively, these contributions delineate a pathway towards more accessible, expressive, and efficient machine learning development paradigms centered on human knowledge integration. By bridging the gap between the intuitive reasoning of domain experts and the structured modeling of PWS through the versatile interface of foundation models, this work offers a tangible methodology for harnessing expertise at scale.

Looking forward, the trajectory suggested by this research points towards increasingly automated and collaborative machine learning systems. While current systems like **Alfred** facilitate expert expression, future iterations could evolve into proactive *agentic partners*. We cautiously envision systems capable of not only interpreting natural language supervision but also suggesting novel LFs, identifying potential dataset biases reflected in prompts, interactively refining supervision strategies with the expert, and even autonomously managing downstream tasks like model selection, training, and monitoring based on the synthesized labels. Such systems could dynamically adapt to evolving data distributions or task requirements, guided by high-level expert intent rather than low-level implementation details. Realizing this vision will necessitate further research into robust agentic control, reliable uncertainty quantification in prompted models, methods for validating agent-proposed strategies, and frameworks for seamless human-AI collaboration in the loop. Nonetheless, the synergy between principled weak supervision and the reasoning capabilities of foundation models, as explored in this dissertation, lays crucial groundwork for a future where sophisticated AI systems can be more readily built, adapted, and guided by human expertise.

List of Figures

List of Tables

References

Alberto, T. C., Lochter, J. V., and Almeida, T. A. (2015). Tubespam: Comment spam filtering on youtube. In *2015 IEEE 14th international conference on machine learning and applications (ICMLA)*, pages 138–143. IEEE.

Allman, E. S., Matias, C., Rhodes, J. A., et al. (2009). Identifiability of parameters in latent structure models with many observed variables. *The Ann. of Stat.*, 37(6A):3099–3132.

Allman, E. S., Rhodes, J. A., Stanghellini, E., and Valtorta, M. (2015). Parameter identifiability of discrete bayesian networks with hidden variables. *J. of Causal Inference*, 3(2):189–205.

Anthropic (2024). The claude 3 model family: Opus, sonnet, haiku. `https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf`. Whitepaper.

Arachie, C. and Huang, B. (2021). A general framework for adversarial label learning. *J. of Mach. Learn. Research*, 22:1–33.

Arora, S., Narayan, A., Chen, M. F., Orr, L. J., Guha, N., Bhatia, K., Chami, I., Sala, F., and Ré, C. (2022). Ask me anything: A simple strategy for prompting language models. *arXiv preprint arXiv:2210.02441*.

Awasthi, A., Ghosh, S., Goyal, R., and Sarawagi, S. (2020a). Learning from rules generalizing labeled exemplars. In *ICLR*.

Awasthi, A., Ghosh, S., Goyal, R., and Sarawagi, S. (2020b). Learning from rules generalizing labeled exemplars. *arXiv preprint arXiv:2004.06025*.

Bach, S. H., He, B., Ratner, A., and Ré, C. (2017a). Learning the structure of generative models without labeled data. In *ICML*.

Bach, S. H., He, B., Ratner, A., and Ré, C. (2017b). Learning the structure of generative models without labeled data. In *International Conference on Machine Learning*, pages 273–282. PMLR.

Bach, S. H., Rodriguez, D., Liu, Y., Luo, C., Shao, H., Xia, C., Sen, S., Ratner, A., Hancock, B., Alborzi, H., et al. (2019a). Snorkel drybell: A case study in deploying weak supervision at industrial scale. In *Proceedings of the 2019 International Conference on Management of Data*, pages 362–375.

Bach, S. H., Rodriguez, D., Liu, Y., Luo, C., Shao, H., Xia, C., Sen, S., Ratner, A., Hancock, B., Alborzi, H., Kuchhal, R., Ré, C., and Malkin, R. (2019b). Snorkel DryBell: A case study in deploying weak supervision at industrial scale. In *SIGMOD*.

Bach, S. H., Sanh, V., Yong, Z.-X., Webson, A., Raffel, C., Nayak, N. V., Sharma, A., Kim, T., Bari, M. S., Fevry, T., Alyafeai, Z., Dey, M., Santilli, A., Sun, Z., Ben-David, S., Xu, C., Chhablani, G., Wang, H., Fries, J. A., Al-shaibani, M. S., Sharma, S., Thakker, U., Almubarak, K., Tang, X., Radev, D., Jiang, M. T.-J., and Rush, A. M. (2022a). PromptSource: An integrated development environment

and repository for natural language prompts. In *Meeting of the Association for Computational Linguistics (ACL) Demonstration*.

Bach, S. H., Sanh, V., Yong, Z.-X., Webson, A., Raffel, C., Nayak, N. V., Sharma, A., Kim, T., Bari, M. S., Fevry, T., Alyafeai, Z., Dey, M., Santilli, A., Sun, Z., Ben-David, S., Xu, C., Chhablani, G., Wang, H., Fries, J. A., Al-shaibani, M. S., Sharma, S., Thakker, U., Almubarak, K., Tang, X., Tang, X., Jiang, M. T.-J., and Rush, A. M. (2022b). Promptsource: An integrated development environment and repository for natural language prompts.

Balsubramani, A. and Freund, Y. (2015a). Optimally combining classifiers using unlabeled data. In *COLT*.

Balsubramani, A. and Freund, Y. (2015b). Scalable semi-supervised aggregation of classifiers. *Advances in Neural Information Processing Systems*, 28.

Balsubramani, A. and Freund, Y. (2016a). Optimal binary classifier aggregation for general losses. In *NeurIPS*.

Balsubramani, A. and Freund, Y. (2016b). Scalable semi-supervised aggregation of classifiers. In *NeurIPS*.

Bar, A., Herzig, R., Wang, X., Rohrbach, A., Chechik, G., Darrell, T., and Globerson, A. (2021). Compositional video synthesis with action graphs. In *ICML*.

Ben-Zaken, E., Ravfogel, S., and Goldberg, Y. (2022). Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *ACL*.

Blum, A. and Mitchell, T. (1998). Combining labeled and unlabeled data with co-training. In *Proceedings of the eleventh annual conference on Computational learning theory*, pages 92–100.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N. S., Chen, A. S., Creel, K., Davis, J., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Gillespie, L. E., Goel, K., Goodman, N. D., Grossman, S., Guha, N., Hashimoto, T., Henderson, P., Hewitt, J., Ho, D. E., Hong, J., Hsu, K., Huang, J., Icard, T. F., Jain, S., Jurafsky, D., Kalluri, P., Karamcheti, S., Keeling, G., Khani, F., Khattab, O., Koh, P. W., Krass, M. S., Krishna, R., Kuditipudi, R., Kumar, A., Ladhak, F., Lee, M., Lee, T., Leskovec, J., Levent, I., Li, X. L., Li, X., Ma, T., Malik, A., Manning, C. D., Mirchandani, S. P., Mitchell, E., Munyikwa, Z., Nair, S., Narayan, A., Narayanan, D., Newman, B., Nie, A., Niebles, J. C., Nilforoshan, H., Nyarko, J. F., Ogut, G., Orr, L., Papadimitriou, I., Park, J. S., Piech, C., Portelance, E., Potts, C., Raghunathan, A., Reich, R., Ren, H., Rong, F., Roohani, Y. H., Ruiz, C., Ryan, J., R'e, C., Sadigh, D., Sagawa, S., Santhanam, K., Shih, A., Srinivasan, K. P., Tamkin, A., Taori, R., Thomas, A. W., Tramèr, F., Wang, R. E., Wang, W., Wu, B., Wu, J., Wu, Y., Xie, S. M., Yasunaga, M., You, J., Zaharia, M. A., Zhang, M., Zhang, T., Zhang, X., Zhang, Y., Zheng, L., Zhou, K.,

and Liang, P. (2021a). On the opportunities and risks of foundation models. *ArXiv*, abs/2108.07258.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., et al. (2021b). On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*.

Bonifacio, L., Abonizio, H., Fadaee, M., and Nogueira, R. (2022). InPars: Data augmentation for information retrieval. *arXiv preprint arXiv:2202.05144*.

Breaux, T. D. (2013). Privacy requirements in an age of increased sharing. In *2013 35th International Conference on Software Engineering (ICSE)*, pages 1019–1020. IEEE.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H., editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Bucher, M., Herbin, S., and Jurie, F. (2017). Generating visual representations for zero-shot classification. In *ICCV Workshops*.

Cabannes, V., Bach, F., and Rudi, A. (2021). Disambiguation of weak supervision with exponential convergence rates. *arXiv preprint arXiv:2102.02789*.

Cabannnes, V., Rudi, A., and Bach, F. (2020). Structured prediction with partial labelling through the infimum loss. In *ICML*.

Cachay, S. R., Boecking, B., and Dubrawski, A. (2021). Dependency structure misspecification in multi-source weak supervision models. *arXiv preprint arXiv:2106.10302*.

Changpinyo, S., Chao, W.-L., Gong, B., and Sha, F. (2016). Synthesized classifiers for zero-shot learning. In *CVPR*.

Chao, W.-L., Changpinyo, S., Gong, B., and Sha, F. (2016). An empirical study and analysis of generalized zero-shot learning for object recognition in the wild. In *European conference on computer vision (ECCV)*.

Chapelle, O., Scholkopf, B., and Zien, A. (2009). *Semi-Supervised Learning*. MIT Press.

Chase, H. (2022). LangChain.

Chen, M. F., Fu, D. Y., Adila, D., Zhang, M., Sala, F., Fatahalian, K., and Ré, C. (2022). Shoring up the foundations: Fusing model embeddings and weak supervision. In *Uncertainty in Artificial Intelligence*, pages 357–367. PMLR.

Chen, V. S., Varma, P., Krishna, R., Bernstein, M., Re, C., and Fei-Fei, L. (2019). Scene graph prediction with limited labels. In *ICCV*.

Chia, Y. K., Bing, L., Poria, S., and Si, L. (2022a). Relationprompt: Leveraging prompts to generate synthetic data for zero-shot relation triplet extraction. *arXiv preprint arXiv:2203.09101*.

Chia, Y. K., Bing, L., Poria, S., and Si, L. (2022b). RelationPrompt: Leveraging prompts to generate synthetic data for zero-shot relation triplet extraction. In *Findings of the Association for Computational Linguistics*.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on information theory*, 2(3):113–124.

Cohan, A., Ammar, W., Van Zuylen, M., and Cady, F. (2019). Structural scaffolds for citation intent classification in scientific publications. In *NAACL*.

Cohn, D. A., Ghahramani, Z., and Jordan, M. I. (1996). Active learning with statistical models. *J. of Artificial Intelligence Research*, 4:129–145.

Corney, D. P., Albakour, D., Martinez-Alvarez, M., and Moussa, S. (2016). What do a million news articles look like? In *NewsIR@ ECIR*, pages 42–47.

Cour, T., Sapp, B., and Taskar, B. (2011). Learning from partial labels. *The J. of Mach. Learn. Research*, 12:1501–1536.

Dalvi, N., Dasgupta, A., Kumar, R., and Rastogi, V. (2013). Aggregating crowdsourced binary ratings. In *Proceedings of the 22nd international conference on World Wide Web*, pages 285–294.

Dawid, A. P. and Skene, A. M. (1979a). Maximum likelihood estimation of observer error-rates using the em algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28(1):20–28.

Dawid, A. P. and Skene, A. M. (1979b). Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society C*, 28(1):20–28.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. (2009). Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*.

Ding, N., Hu, S., Zhao, W., Chen, Y., Liu, Z., Zheng, H., and Sun, M. (2022). OpenPrompt: An open-source framework for prompt-learning. In Basile, V., Kozareva, Z., and Stajner, S., editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 105–113, Dublin, Ireland. Association for Computational Linguistics.

Dong, X., Bao, J., Zhang, T., Chen, D., Shuyang, G., Zhang, W., Yuan, L., Chen, D.,

Wen, F., and Yu, N. (2022). Clip itself is a strong fine-tuner: Achieving 85.7 *arXiv preprint arXiv:2212.06138*.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.

Drozdov, A., Schärli, N., Akyürek, E., Scales, N., Song, X., Chen, X., Bousquet, O., and Zhou, D. (2022). Compositional semantic parsing with large language models. *arXiv preprint arXiv:2209.15003*.

Durand, T., Mehrasa, N., and Mori, G. (2019). Learning a deep convnet for multi-label classification with partial labels. In *CVPR*.

Elazar, Y., Kassner, N., Ravfogel, S., Ravichander, A., Hovy, E., Schütze, H., and Goldberg, Y. (2021). Measuring and improving consistency in pretrained language models. *Transactions of the Association for Computational Linguistics*, 9:1012–1031.

Farhadi, A., Endres, I., Hoiem, D., and Forsyth, D. A. (2009). Describing objects by their attributes. In *CVPR*, pages 1778–1785. IEEE Computer Society.

Feigenbaum, E. A. (1984). Knowledge engineering. *Annals of the New York Academy of Sciences*, 426(1):91–107.

Feigenbaum, E. A., Buchanan, B. G., and Lederberg, J. (1970). On generality and problem solving: A case study using the dendral program. Technical report.

Felix, R., Reid, I., Carneiro, G., et al. (2018). Multi-modal cycle-consistent generalized zero-shot learning. In *ECCV*.

Feng, L., Kaneko, T., Han, B., Niu, G., An, B., and Sugiyama, M. (2020a). Learning with multiple complementary labels. In *ICML*.

Feng, L., Lv, J., Han, B., Xu, M., Niu, G., Geng, X., An, B., and Sugiyama, M. (2020b). Provably consistent partial-label learning. In *NeurIPS*.

Feng, L., Lv, J., Han, B., Xu, M., Niu, G., Geng, X., An, B., and Sugiyama, M. (2020c). Provably consistent partial-label learning.

Fodor, J. A. and Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28:3–71.

Fries, J., Varma, P., Chen, V., Xiao, K., Tejeda, H., Priyanka, S., Dunnmon, J., Chubb, H., Maskatia, S., Fiterau, M., Delp, S., Ashley, E., Ré, C., and Priest, J. (2019). Weakly supervised classification of rare aortic valve malformations using unlabeled cardiac MRI sequences. *Nature Communications*, 10(1).

Fries, J. A., Steinberg, E., Khattar, S., Fleming, S. L., Posada, J., Callahan, A., and Shah, N. H. (2021). Ontology-driven weak supervision for clinical entity classification in electronic health records. *Nature Communications*, 12(1):1–11.

Frome, A., Corrado, G. S., Shlens, J., Bengio, S., Dean, J., Ranzato, M., and Mikolov, T. (2013). Devise: A deep visual-semantic embedding model. In *NeurIPS*.

Fu, D. Y., Chen, M. F., Sala, F., Hooper, S. M., Fatahalian, K., and Ré, C. (2020). Fast and three-rious: Speeding up weak supervision with triplet methods. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, pages 3280–3291. PMLR.

Gao, C. and Zhou, D. (2013). Minimax optimal convergence rates for estimating ground truth from crowdsourced labels. *CoRR*, abs/1207.0016.

Gao, H., Barbier, G., and Goolsby, R. (2013). Multimedia knowledge quality assessment: Evaluating the quality of medical data annotated by crowdsourcing workers. *IEEE International Conference on Multimedia and Expo Workshops*, pages 1–6.

Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., et al. (2020). The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Gao, P., Geng, S., Zhang, R., Ma, T., Fang, R., Zhang, Y., Li, H., and Qiao, Y. (2021). Clip-adapter: Better vision-language models with feature adapters. *ArXiv preprint*, abs/2110.04544.

Gardner, M., Grus, J., Neumann, M., Tafjord, O., Dasigi, P., Liu, N. F., Peters, M., Schmitz, M., and Zettlemoyer, L. S. (2017). Allennlp: A deep semantic natural language processing platform.

Ge, C., Huang, R., Xie, M., Lai, Z., Song, S., Li, S., and Huang, G. (2022). Domain adaptation via prompt learning. *ArXiv preprint*, abs/2202.06687.

Gemma Team, G. D. (2024). Gemma 2. Lightweight open models ranging from 2 to 27 billion parameters.

Gómez Hidalgo, J. M., Bringas, G. C., Sánz, E. P., and García, F. C. (2006). Content based sms spam filtering. In *Proceedings of the 2006 ACM symposium on Document engineering*, pages 107–114.

Gulli, A. (2005). *AG's corpus of news articles*.

Guo, D., Rush, A., and Kim, Y. (2021). Parameter-efficient transfer learning with diff pruning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4884–4896, Online. Association for Computational Linguistics.

Hamburg, M. A. and Collins, F. S. (2012). Hipaa compliance and implementation in a medical imaging department. *Radiologic technology*, 83(5):447–456.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *CVPR*.

He, P., Liu, X., Gao, J., and Chen, W. (2020). Deberta: Decoding-enhanced bert with disentangled attention.

Herzig, R., Bar, A., Xu, H., Chechik, G., Darrell, T., and Globerson, A. (2020). Learning canonical representations for scene graph to image generation. In *European Conference on Computer Vision*, pages 210–227. Springer.

Hoffmann, R., Zhang, C., Ling, X., Zettlemoyer, L., and Weld, D. S. (2011). Knowledge-based weak supervision for information extraction of overlapping relations. In *Proceedings of the 49th annual meeting of the association for computational linguistics: human language technologies*, pages 541–550.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. (2019). Parameter-efficient transfer learning for NLP. In Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR.

Huang, L., Yu, W., Ma, W., Zhong, W., Feng, Z., Wang, H., Chen, Q., Peng, W., Feng, X., Qin, B., et al. (2023). A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*.

Huang, Y., Lv, T., Cui, L., Lu, Y., and Wei, F. (2022). Layoutlmv3: Pre-training for document ai with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia*, pages 4083–4091.

Hudson, D. A. and Manning, C. D. (2019). GQA: A new dataset for real-world visual reasoning and compositional question answering. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 6700–6709. Computer Vision Foundation / IEEE.

Hüllermeier, E. and Cheng, W. (2015). Superset learning based on generalized loss minimization. In *ECML PKDD*.

Hupkes, D., Dankers, V., Mul, M., and Bruni, E. (2020). Compositionality decomposed: How do neural networks generalise? *J. Artif. Intell. Res.*, 67:757–795.

Ipeirotis, P. G., Provost, F., and Wang, J. (2010). Quality management on amazon mechanical turk. *Proceedings of the ACM SIGKDD workshop on human computation*, pages 64–67.

Ishida, T., Niu, G., Hu, W., and Sugiyama, M. (2017). Learning from complementary labels. In *NeurIPS*.

Isola, P., Lim, J. J., and Adelson, E. H. (2015). Discovering states and transformations in image collections. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*, pages 1383–1391. IEEE Computer Society.

Jain, A., Guo, M., Srinivasan, K., Chen, T., Kudugunta, S., Jia, C., Yang, Y., and Baldridge, J. (2021). Mural: multimodal, multitask retrieval across languages. *ArXiv preprint*, abs/2109.05125.

Jayaraman, D. and Grauman, K. (2014). Zero shot recognition with unreliable attributes. In *NeurIPS*.

Jia, C., Yang, Y., Xia, Y., Chen, Y., Parekh, Z., Pham, H., Le, Q. V., Sung, Y., Li, Z., and Duerig, T. (2021). Scaling up visual and vision-language representation learning with noisy text supervision. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 4904–4916. PMLR.

Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., de las Casas, D., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L. R., Lachaux, M.-A., Stock, P., Scao, T. L., Lavril, T., Wang, T., Lacroix, T., and Sayed, W. E. (2023). Mistral 7b. Open-source language model with 7 billion parameters.

Jin, R. and Ghahramani, Z. (2002). Learning with multiple labels. In *NeurIPS*.

Jin, W., Cheng, Y., Shen, Y., Chen, W., and Ren, X. (2022). A good prompt is worth millions of parameters: Low-resource prompt-based learning for vision-language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Joglekar, M., Garcia-Molina, H., and Parameswaran, A. (2015). Comprehensive and reliable crowd assessment algorithms. In *2015 IEEE 31st International Conference on Data Engineering*, pages 195–206. IEEE.

Johnson, J., Hariharan, B., Van Der Maaten, L., Fei-Fei, L., Lawrence Zitnick, C., and Girshick, R. (2017). Clevr: A diagnostic dataset for compositional language and elementary visual reasoning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2901–2910.

Ju, C., Han, T., Zheng, K., Zhang, Y., and Xie, W. (2021). Prompting visual-language models for efficient video understanding. *ArXiv preprint*, abs/2112.04478.

Karamanolakis, G., Mukherjee, S., Zheng, G., and Awadallah, A. H. (2021). Self-training with weak supervision. In *NAACL*.

Kittur, A., Nickerson, J. V., Bernstein, M., Gerber, E., Shaw, A., Zimmerman, J., Lease, M., and Horton, J. (2013). The future of crowd work. *Proceedings of the 2013 conference on Computer supported cooperative work*, pages 1301–1318.

Koller, D. (2009). *Probabilistic Graphical Models: Principles and Techniques*. The MIT Press.

Krishna, R., Hata, K., Chen, S., Kravitz, J., Shamma, D. A., Fei-Fei, L., and Bernstein,

M. S. (2016). Embracing error to enable rapid crowdsourcing. In *Proceedings of the 2016 CHI conference on human factors in computing systems*, pages 3167–3179.

Kruskal, J. B. (1977). Three-way arrays: rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics. *Linear algebra and its applications*, 18(2):95–138.

Kuang, Z., Arachie, C. G., Liang, B., Narayana, P., DeSalvo, G., Quinn, M. S., Huang, B., Downs, G., and Yang, Y. (2022). Firebolt: Weak supervision under weaker assumptions. In *International Conference on Artificial Intelligence and Statistics*, pages 8214–8259. PMLR.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626.

Lake, B. M. and Baroni, M. (2018). Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In Dy, J. G. and Krause, A., editors, *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 2879–2888. PMLR.

Lampert, C. H., Nickisch, H., and Harmeling, S. (2009). Learning to detect unseen object classes by between-class attribute transfer. In *CVPR*.

Lang, H., Agrawal, M., Kim, Y., and Sontag, D. (2022). Co-training improves prompt-based learning for large language models. *arXiv preprint arXiv:2202.00828*.

Lester, B., Al-Rfou, R., and Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Lhoest, Q., del Moral, A. V., Jernite, Y., Thakur, A., von Platen, P., Patil, S., Chaumond, J., Drame, M., Plu, J., Tunstall, L., et al. (2021). Datasets: A community library for natural language processing. *arXiv preprint arXiv:2109.02846*.

Li, C., Li, X., and Ouyang, J. (2020a). Learning with noisy partial labels by simultaneously leveraging global and local consistencies. In *CIKM*.

Li, X. and Roth, D. (2002). Learning question classifiers. In *COLING*.

Li, X. L. and Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4582–4597, Online. Association for Computational Linguistics.

Li, Y., Liang, F., Zhao, L., Cui, Y., Ouyang, W., Shao, J., Yu, F., and Yan, J.

(2021). Supervision exists everywhere: A data efficient contrastive language-image pre-training paradigm. *ArXiv*, abs/2110.05208.

Li, Y., Xu, Y., Mao, X., and Lu, C. (2020b). Symmetry and group in attribute-object compositions. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pages 11313–11322. IEEE.

Liang, C., Yu, Y., Jiang, H., Er, S., Wang, R., Zhao, T., and Zhang, C. (2020). Bond: Bert-assisted open-domain named entity recognition with distant supervision. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1054–1064.

Lison, P., Hubin, A., Barnes, J., and Touileb, S. (2020). Named entity recognition without labelled data: A weak supervision approach. *arXiv preprint arXiv:2004.14723*.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. (2024). Visual instruction tuning. *Advances in neural information processing systems*, 36.

Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., and Raffel, C. (2022). Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *ArXiv preprint*, abs/2205.05638.

Liu, L. and Dietterich, T. (2014). Learnability of the superset label learning problem. In *ICML*.

Liu, L. and Dietterich, T. G. (2012). A conditional multinomial mixture model for superset label learning. In *NeurIPS*.

Liu, L., Zhou, T., Long, G., Jiang, J., and Zhang, C. (2020). Attribute propagation network for graph zero-shot learning. In *AAAI*.

Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H., and Neubig, G. (2021). Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ArXiv*, abs/2107.13586.

Liu, Y., Guo, J., Cai, D., and He, X. (2019a). Attribute attention for semantic disambiguation in zero-shot learning. In *ICCV*.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., and Stoyanov, V. (2019b). Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*.

Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *ICLR*.

Luo, J. and Orabona, F. (2010). Learning from candidate labeling sets. Technical report.

Mahabadi, R. K., Henderson, J., and Ruder, S. (2021). Compacter: Efficient low-rank hypercomplex adapter layers. In *NeurIPS*.

Mallinar, N., Shah, A., Ugrani, R., Gupta, A., Gurusankar, M., Ho, T. K., Liao, Q. V.,

Zhang, Y., Bellamy, R. K., Yates, R., et al. (2019). Bootstrapping conversational agents with weak supervision. In *AAAI*.

Mancini, M., Naeem, M., Xian, Y., and Akata, Z. (2021a). Open world compositional zero-shot learning. In *34th IEEE Conference on Computer Vision and Pattern Recognition*. IEEE.

Mancini, M., Naeem, M. F., Xian, Y., and Akata, Z. (2021b). Learning graph embeddings for open world compositional zero-shot learning. *ArXiv*, abs/2105.01017.

Marcus, G. F. (2003). *The algebraic mind: Integrating connectionism and cognitive science*. MIT press.

Mason, W. and Suri, S. (2012). Conducting behavioral research on amazon's mechanical turk. *Behavior research methods*, 44(1):1–23.

Mazzetto, A., Cousins, C., Sam, D., Bach, S. H., and Upfal, E. (2021a). Adversarial multiclass learning under weak supervision with performance guarantees. In *ICML*.

Mazzetto, A., Sam, D., Park, A., Upfal, E., and Bach, S. H. (2021b). Semi-supervised aggregation of dependent weak supervision sources with performance guarantees. In *AISTATS*.

McCarthy, J. (1959). Programs with common sense.

Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., and Galstyan, A. (2019). A survey on bias and fairness in machine learning. *arXiv preprint arXiv:1908.09635*.

Meinshausen, N. and Bühlmann, P. (2006). High-dimensional graphs and variable selection with the lasso.

Meng, Y., Shen, J., Zhang, C., and Han, J. (2018). Weakly-supervised neural text classification. In *proceedings of the 27th ACM International Conference on information and knowledge management*, pages 983–992.

Meta (2024a). Llama 3.1. Multilingual large language model with 405 billion parameters.

Meta (2024b). Llama 3.2. Multimodal AI model with text and image processing capabilities.

Microsoft (2024). Phi-3 technical report: A highly capable language model locally on your phone. 3.8 billion parameter language model optimized for mobile devices.

Misra, I., Gupta, A., and Hebert, M. (2017). From red wine to red tomato: Composition with context. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*, pages 1160–1169. IEEE Computer Society.

Mitchell, E., Lin, C., Bosselut, A., Finn, C., and Manning, C. D. (2022). Fast model editing at scale. In *International Conference on Learning Representations*.

Naeem, M. F., Xian, Y., Tombari, F., and Akata, Z. (2021). Learning graph embeddings

for compositional zero-shot learning. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 953–962.

Nagarajan, T. and Grauman, K. (2018). Attributes as operators. *ECCV*.

Narayan, S., Gupta, A., Khan, F. S., Snoek, C. G., and Shao, L. (2020). Latent embedding feedback and discriminative features for zero-shot classification. In *ECCV*.

Nawhal, M., Zhai, M., Lehrmann, A., Sigal, L., and Mori, G. (2020). Generating videos of zero-shot compositions of actions and objects. In *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XII*, page 382–401, Berlin, Heidelberg. Springer-Verlag.

Newell, A., Shaw, J. C., and Simon, H. A. (1959). Report on a general problem solving program. In *IFIP congress*, volume 256, page 64. Pittsburgh, PA.

Nguyen, N. and Caruana, R. (2008). Classification with partial labels. In *KDD*.

Nitzan, S. and Paroush, J. (1982). Optimal decision rules in uncertain dichotomous choice situations. *International Economic Review*, 23(2):289–97.

Norouzi, M., Mikolov, T., Bengio, S., Singer, Y., Shlens, J., Frome, A., Corrado, G. S., and Dean, J. (2013). Zero-shot learning by convex combination of semantic embeddings. *arXiv preprint arXiv:1312.5650*.

Orr, L. (2022). Manifest. `https://github.com/HazyResearch/manifest`.

Palatucci, M., Pomerleau, D., Hinton, G. E., and Mitchell, T. M. (2009). Zero-shot learning with semantic output codes. In *NeurIPS*.

Pan, S. J. and Yang, Q. (2009). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering*, 22(10):1345–1359.

Parkhi, O. M., Vedaldi, A., Zisserman, A., and Jawahar, C. (2012). Cats and dogs. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3498–3505. IEEE.

Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., and Lerer, A. (2017). Automatic differentiation in PyTorch. In *NeurIPS Autodiff Workshop*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E. B., and Garnett, R., editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 8024–8035.

Pearl, J. (1988). Probabilistic reasoning in intelligent systems: Networks of plausible inference. morgan kaufmann, san mateo, california.

Pennington, J., Socher, R., and Manning, C. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.

Pourpanah, F., Abdar, M., Luo, Y., Zhou, X., Wang, R., Lim, C., and Wang, X. (2020). A review of generalized zero-shot learning methods. *ArXiv*, abs/2011.08641.

Purushwalkam, S., Nickel, M., Gupta, A., and Ranzato, M. (2019). Task-driven modular networks for zero-shot compositional learning. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pages 3592–3601. IEEE.

Qin, G. and Eisner, J. (2021). Learning how to ask: Querying LMs with mixtures of soft prompts. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5203–5212, Online. Association for Computational Linguistics.

Quinn, A. J. and Bederson, B. B. (2011). Crowdflow: Integrating machine learning with mechanical turk for speed-cost-quality flexibility. *Technical Report HCIL-2011-09*.

Qwen Team, A. G. (2024). Qwen2 technical report. Comprehensive suite of language models ranging from 0.5 to 72 billion parameters.

Rabiner, L. R. (1989). A tutorial on hidden markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286.

Radenović, F., Sinha, A., Gordo, A., Berg, T. L., and Mahajan, D. K. (2021). Large-scale attribute-object compositions. *ArXiv*.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. (2021a). Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021b). Learning transferable visual models from natural language supervision. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR.

Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I. (2022). Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Ratner, A., Bach, S. H., Ehrenberg, H., Fries, J., Wu, S., and Ré, C. (2017). Snorkel: Rapid training data creation with weak supervision. In *Proceedings of the VLDB Endowment. International Conference on Very Large Data Bases*, volume 11, page 269. NIH Public Access.

Ratner, A., Hancock, B., Dunnmon, J., Sala, F., Pandey, S., and Ré, C. (2019a). Training complex models with multi-task weak supervision. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4763–4771.

Ratner, A. J., De Sa, C. M., Wu, S., Selsam, D., and Ré, C. (2016a). Data programming: Creating large training sets, quickly. In *NeurIPS*.

Ratner, A. J., De Sa, C. M., Wu, S., Selsam, D., and Ré, C. (2016b). Data programming: Creating large training sets, quickly. *Advances in neural information processing systems*, 29.

Ratner, A. J., Hancock, B., Dunnmon, J., Sala, F., Pandey, S., and Ré, C. (2019b). Training complex models with multi-task weak supervision. In *AAAI*.

Ré, C., Niu, F., Gudipati, P., and Srisuwananukorn, C. (2019). Overton: A data system for monitoring and improving machine-learned products. *arXiv preprint arXiv:1909.05372*.

Romera-Paredes, B. and Torr, P. (2015). An embarrassingly simple approach to zero-shot learning. In *ICML*.

Ruis, F., Burghouts, G. J., and Bucur, D. (2021). Independent prototype propagation for zero-shot compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International J. of Computer Vision*.

Saab, K., Dunnmon, J., Ré, C., Rubin, D., and Lee-Messer, C. (2020). Weak supervision as an efficient approach for automated seizure detection in electroencephalography. *NPJ Digital Medicine*, 3(1):1–12.

Safranchik, E., Luo, S., and Bach, S. H. (2020). Weakly supervised sequence tagging from noisy rules. In *AAAI*.

Saleiro, P., Kuester, B., Hinkson, L., London, J., Stevens, A., Anisfeld, A., Rodolfa, K. T., and Ghani, R. (2018). Aequitas: A bias and fairness audit toolkit. *arXiv preprint arXiv:1811.05577*.

Sanh, V., Webson, A., Raffel, C., Bach, S., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Raja, A., Dey, M., Bari, M. S., Xu, C., Thakker, U., Sharma, S. S., Szczechla, E., Kim, T., Chhablani, G., Nayak, N., Datta, D., Chang, J., Jiang, M.

T.-J., Wang, H., Manica, M., Shen, S., Yong, Z. X., Pandey, H., Bawden, R., Wang, T., Neeraj, T., Rozen, J., Sharma, A., Santilli, A., Fevry, T., Fries, J. A., Teehan, R., Scao, T. L., Biderman, S., Gao, L., Wolf, T., and Rush, A. M. (2022). Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

Sanh, V., Webson, A., Raffel, C., Bach, S. H., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Scao, T. L., Raja, A., et al. (2021). Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207*.

Sariyildiz, M. B. and Cinbis, R. G. (2019). Gradient matching generative networks for zero-shot learning. In *CVPR*.

Schapire, R. E. and Freund, Y. (2013). Boosting: Foundations and algorithms. *Kybernetes*, 42(1):164–166.

Schick, T. and Schütze, H. (2021a). Exploiting cloze-questions for few-shot text classification and natural language inference. In Merlo, P., Tiedemann, J., and Tsarfaty, R., editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 255–269, Online. Association for Computational Linguistics.

Schick, T. and Schütze, H. (2021b). Generating datasets with pretrained language models. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t., editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6943–6951, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Schick, T. and Schütze, H. (2021c). Generating datasets with pretrained language models. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Scialom, T., Chakrabarty, T., and Muresan, S. (2022). Fine-tuned language models are continual learners. In *Conference on Empirical Methods in Natural Language Processing*.

Settles, B. (2012). Active learning. *Synthesis Lectures on Artificial Intelligence and Machine Learning*, 6(1):1–114.

Shin, C., Li, W., Vishwakarma, H., Roberts, N., and Sala, F. (2021). Universalizing weak supervision. *arXiv preprint arXiv:2112.03865*.

Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E., and Singh, S. (2020). AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4222–4235, Online. Association for Computational Linguistics.

Shortliffe, E. H., Davis, R., Axline, S. G., Buchanan, B. G., Green, C. C., and Cohen, S. N. (1975). Computer-based consultations in clinical therapeutics: explanation and rule acquisition capabilities of the mycin system. *Computers and biomedical research*, 8(4):303–320.

Smith, R., Fries, J. A., Hancock, B., and Bach, S. H. (2022). Language models in the loop: Incorporating prompting into weak supervision. *arXiv preprint arXiv:2205.02318*.

Snow, R., O'Connor, B., Jurafsky, D., and Ng, A. Y. (2008). Cheap and fast—but is it good?: evaluating non-expert annotations for natural language tasks. In *Proceedings of the 2008 conference on empirical methods in natural language processing*, pages 254–263.

Socher, R., Ganjoo, M., Manning, C. D., and Ng, A. (2013). Zero-shot learning through cross-modal transfer. In *NeurIPS*.

Song, J., Shen, C., Yang, Y., Liu, Y., and Song, M. (2018). Transductive unbiased embedding for zero-shot learning. In *CVPR*.

Sung, Y.-L., Nair, V., and Raffel, C. (2021). Training neural networks with fixed sparse masks. In *NeurIPS*.

Suri, S., Chanda, R., Bulut, N., Narayana, P., Zeng, Y., Bailis, P., Basu, S., Narlikar, G., Ré, C., and Sethi, A. (2020). Leveraging organizational resources to adapt models to new data modalities. *arXiv preprint arXiv:2008.09983*.

Sylvain Gugger, Lysandre Debut, T. W. P. S. Z. M. S. M. (2022). Accelerate: Training and inference at scale made simple, efficient and adaptable. `https://github.com/huggingface/accelerate`.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Van Engelen, J. E. and Hoos, H. H. (2020). A survey on semi-supervised learning. *Machine Learning*, 109(2):373–440.

Varma, P., He, B. D., Bajaj, P., Khandwala, N., Banerjee, I., Rubin, D., and Ré, C. (2017). Inferring generative model structure with static analysis. *Advances in neural information processing systems*, 30.

Varma, P., Sala, F., He, A., Ratner, A., and Ré, C. (2019a). Learning dependency structures for weak supervision models. In *ICML*.

Varma, P., Sala, F., He, A., Ratner, A., and Ré, C. (2019b). Learning dependency structures for weak supervision models. In *International Conference on Machine Learning*, pages 6418–6427. PMLR.

Varma, P., Sala, F., Sagawa, S., Fries, J., Fu, D., Khattar, S., Ramamoorthy, A., Xiao, K., Fatahalian, K., Priest, J., et al. (2019c). Multi-resolution weak supervision for sequential data. *Advances in Neural Information Processing Systems*, 32.

Verma, V. K., Arora, G., Mishra, A., and Rai, P. (2018). Generalized zero-shot learning via synthesized examples. In *CVPR*.

Vu, T., Lester, B., Constant, N., Al-Rfou, R., and Cer, D. M. (2021). Spot: Better frozen model adaptation through soft prompt transfer. *ArXiv*.

Wan, Z., Chen, D., Li, Y., Yan, X., Zhang, J., Yu, Y., and Liao, J. (2019). Transductive zero-shot learning with visual structure constraint. In *NeurIPS*.

Wang, H., Liu, W., Zhao, Y., Hu, T., Chen, K., and Chen, G. (2020). Learning from multi-dimensional partial labels. In *IJCAI*.

Wang, J., Kraska, T., Franklin, M. J., and Feng, J. (2012). Crowder: Crowdsourcing entity resolution. In *Proceedings of the VLDB Endowment*, volume 5, pages 1483–1494.

Wang, W., Zheng, V. W., Yu, H., and Miao, C. (2019). A survey of zero-shot learning: Settings, methods, and applications. *ACM Transactions on Intelligent Systems and Technology*, 10(2):1–37.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., and Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., and Zhou, D. (2022). Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (2020). Transformers: State-of-the-art natural language processing. In Liu, Q. and Schlangen, D., editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Wu, Y., Gardner, M., Stenetorp, P., and Dasigi, P. (2022a). Generating data to mitigate spurious correlations in natural language inference datasets. In Muresan, S., Nakov, P., and Villavicencio, A., editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2660–2676, Dublin, Ireland. Association for Computational Linguistics.

Wu, Y., Gardner, M., Stenetorp, P., and Dasigi, P. (2022b). Generating data to mitigate spurious correlations in natural language inference datasets. In *Meeting of the Association for Computational Linguistics (ACL)*.

Xian, Y., Akata, Z., Sharma, G., Nguyen, Q., Hein, M., and Schiele, B. (2016). Latent embeddings for zero-shot classification. In *CVPR*.

Xian, Y., Lampert, C. H., Schiele, B., and Akata, Z. (2018). Zero-shot learning: A comprehensive evaluation of the good, the bad and the ugly. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9):2251–2265.

Xian, Y., Sharma, S., Schiele, B., and Akata, Z. (2019). f-vaegan-d2: A feature generating framework for any-shot learning. In *CVPR*.

Xie, M.-K. and Huang, S.-J. (2021). Partial multi-label learning with noisy label identification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Xu, G., Kordjamshidi, P., and Chai, J. (2021). Zero-shot compositional concept learning. In *Proceedings of the 1st Workshop on Meta Learning and Its Applications to Natural Language Processing*, pages 19–27, Online. Association for Computational Linguistics.

Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., and Zhou, M. (2020a). Layoutlm: Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1192–1200.

Xu, Y., Xu, Y., Lv, T., Cui, L., Wei, F., Wang, G., Lu, Y., Florencio, D., Zhang, C., Che, W., et al. (2020b). Layoutlmv2: Multi-modal pre-training for visually-rich document understanding. *arXiv preprint arXiv:2012.14740*.

Yan, Y. and Guo, Y. (2020). Partial label learning with batch label correction. In *AAAI*.

Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., and Narasimhan, K. (2024). Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36.

Ye, J., Gao, J., Li, Q., Xu, H., Feng, J., Wu, Z., Yu, T., and Kong, L. (2022a). Zerogen: Efficient zero-shot learning via dataset generation. *arXiv preprint arXiv:2202.07922*.

Ye, J., Gao, J., Li, Q., Xu, H., Feng, J., Wu, Z., Yu, T., and Kong, L. (2022b). ZeroGen: Efficient zero-shot learning via dataset generation. *arXiv preprint arXiv:2202.07922*.

Ye, Y., Singh, M., Gupta, A., and Tulsiani, S. (2019). Compositional video prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10353–10362.

Yu, A. and Grauman, K. (2014). Fine-grained visual comparisons with local learning. In *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*, pages 192–199. IEEE Computer Society.

Yu, P. and Bach, S. (2023). Alfred: A system for prompted weak supervision. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 479–488, Toronto, Canada. Association for Computational Linguistics.

Yu, P., Ding, T., and Bach, S. H. (2022). Learning from multiple noisy partial labelers. In *Artificial Intelligence and Statistics (AISTATS)*.

Yuan, L., Chen, D., Chen, Y.-L., Codella, N., Dai, X., Gao, J., Hu, H., Huang, X., Li, B., Li, C., et al. (2021). Florence: A new foundation model for computer vision. *arXiv preprint arXiv:2111.11432*.

Yuan, X., Côté, M.-A., Fu, J., Lin, Z., Pal, C., Bengio, Y., and Trischler, A. (2019). Interactive language learning by question answering. *arXiv preprint arXiv:1908.10909*.

Yun, T., Bhalla, U., Pavlick, E., and Sun, C. (2022). Do vision-language pretrained models learn primitive concepts? *arXiv preprint arXiv:2203.17271*.

Zelikman, E., Wu, Y., and Goodman, N. D. (2022). Star: Bootstrapping reasoning with reasoning. *arXiv preprint arXiv:2203.14465*.

Zhang, J., Hsieh, C.-Y., Yu, Y., Zhang, C., and Ratner, A. (2022a). A survey on programmatic weak supervision. *arXiv preprint arXiv:2202.05433*.

Zhang, J., Wang, B., Song, X., Wang, Y., Yang, Y., Bai, J., and Ratner, A. (2021a). Creating training sets via weak indirect supervision. *arXiv preprint arXiv:2110.03484*.

Zhang, J., Yu, Y., Li, Y., Wang, Y., Yang, Y., Yang, M., and Ratner, A. (2021b). WRENCH: A comprehensive benchmark for weak supervision. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Zhang, R., Yu, Y., Shetty, P., Song, L., and Zhang, C. (2022b). Prboost: Prompt-based rule discovery and boosting for interactive weakly-supervised learning. *arXiv preprint arXiv:2203.09735*.

Zhang, R., Yu, Y., Shetty, P., Song, L., and Zhang, C. (2022c). PRBoost: Prompt-based rule discovery and boosting for interactive weakly-supervised learning. In *Meeting of the Association for Computational Linguistics (ACL)*.

Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level convolutional networks for text classification. In *NeurIPS*.

Zhang, Y., Chen, X., Zhou, D., and Jordan, M. I. (2016). Spectral methods meet em: A provably optimal algorithm for crowdsourcing. *The Journal of Machine Learning Research*, 17(1):3537–3580.

Zhang, Z.-R., Zhang, Q.-W., Cao, Y., and Zhang, M.-L. (2021c). Exploiting unlabeled data via partial label assignment for multi-class semi-supervised learning. In *AAAI*.

Zhao, B., Fu, Y., Liang, R., Wu, J., Wang, Y., and Wang, Y. (2019). A large-scale attribute dataset for zero-shot learning. In *CVPR Workshops*.

Zhao, Z., Wallace, E., Feng, S., Klein, D., and Singh, S. (2021). Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, pages 12697–12706. PMLR.

Zhou, K., Yang, J., Loy, C. C., and Liu, Z. (2021). Learning to prompt for vision-language models. *ArXiv preprint*, abs/2109.01134.

Zhou, K., Yang, J., Loy, C. C., and Liu, Z. (2022). Conditional prompt learning for vision-language models. In *CVPR*.

Zhu, J., Lao, N., and Xing, E. P. (2010). Grafting-light: fast, incremental feature selection and structure learning of markov random fields. In *Proceedings of the 16th*

ACM SIGKDD international conference on Knowledge discovery and data mining, pages 303–312.

Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., and He, Q. (2020). A comprehensive survey on transfer learning. *Proceedings of the IEEE*, 109(1):43–76.